

分 类 号_____

学校代码 10388

密级 公开

学号 2211708001



福建理工大学
FUJIAN UNIVERSITY OF TECHNOLOGY

硕士专业学位研究生学位（毕业）论文

基于微服务架构的在线学习平台
设计与实现

专 业 名 称： 计算机技术

研 究 方 向： 信息管理与智能决策

研 究 生 姓 名： 刘亚辉

指 导 教 师 、 职 称： 李杰 教授

校 外 指 导 老 师： 曾开发

所 在 学 院： 管理学院

答辩委员会主席签名：_____

二〇二四年 六 月

基于微服务架构的在线学习平台设计与实现

摘要

近年来,“上网课”这个词越来越流行,其背后反映的是在线学习平台的迅猛发展。随着互联网技术的不断进步和普及,越来越多的高校和企业开始将教学资源搬到线上,打造各类在线学习平台,以满足广大学习者多样化的学习需求。本文设计并实现了一个在线学习平台,该平台是应笔者所在单位的需求而开发的,专注于专业技能培训。其主要目的是为工程领域的学员提供优质的专业课程,以满足他们的学习需求。

早期开发的在线学习平台大多都是采用单体架构,这种架构将所有业务逻辑和功能集成在一个大型应用程序中,从而提高了平台在开发、部署和维护方面的效率。然而,随着业务规模 and 需求的不断增长,单体架构的在线学习平台逐渐暴露出其局限性。首先,功能模块的增多使得代码间的耦合度不断提升,进而导致了维护成本的显著增加。其次,业务伸缩性差的问题愈发明显,当某个功能需要扩展时,整个应用都需要进行扩展,这不仅增加了维护成本,还可能导致系统的不稳定。此外,系统并发能力的不足也已成为制约平台进一步发展的关键因素,无法满足大量用户同时在线学习的需求。

为解决单体架构带来的模块耦合度高、业务伸缩性差、高并发处理能力不足等问题,本文采用微服务架构对在线学习平台进行开发。微服务架构将整个系统拆分为多个独立、轻量级的服务,每个服务都能够独立地开发、部署、扩展和运维,降低了模块之间的耦合度,提高了系统的可扩展性和并发处理能力。

本文主要研究内容:首先根据微服务拆分准则,按业务逻辑划分出 4 个主要的微服务,即课程服务、视频服务、订单服务和用户服务,然后利用 UML 用例图来分析各个服务的功能需求;其次是设计出系统的整体架构,并基于此对系统各功能模块进行概要设计;通过分析 E-R 图中各实体间的关系对系统数据库进行物理设计;然后基于 ASP.NET Core 框架,使用 Ocelot 作为网关组件,Consul 作为服务注册组件,并运用 gRPC 服务通信机制实现服务间的通信调用;通过这些技术的组合来实现基于微服务架构的在线学习平台;最后对在线学习平台进行功能性测试和非功能性测试,确保系统运行时的稳定性和可靠性。

经过完整的开发流程和系统测试,本文设计开发的基于微服务的在线学习平台已经能够稳定运行。目前,该平台已正式进入公司的内测阶段,之后将结合实际情况,对平台进行必要的优化和改进,为用户提供优质的学习体验,推动公司的业务发展。

关键词: 在线学习平台,微服务架构,ASP.NET Core 框架

Design and Implementation of Online Learning Platform Based on Microservice Architecture

Abstract

In recent years, the phrase "online courses" has become increasingly popular, reflecting the rapid development of online learning platforms. With the continuous advancement and popularization of Internet technology, more and more universities and enterprises have begun to move teaching resources online, creating various online learning platforms to meet the diverse learning needs of learners. This thesis designs and implements an online learning platform, which is developed in response to the needs of the author's unit and focuses on professional skills training. Its main purpose is to provide high-quality professional courses for students in the engineering field to meet their learning needs.

Early developed online learning platforms mostly adopted a monolithic architecture, which integrates all business logic and functions into a large application, thus improving efficiency in development, deployment, and maintenance. However, as business scale and demands continue to grow, the limitations of the monolithic architecture in online learning platforms have gradually become apparent. Firstly, the increasing number of functional modules has led to a rise in code coupling, resulting in a significant increase in maintenance costs. Secondly, the poor scalability of the business has become increasingly evident. When a certain function needs to be expanded, the entire application requires expansion, which not only increases maintenance costs but may also lead to system instability. Additionally, the insufficient concurrent processing capability of the system has become a crucial factor restricting the further development of the platform, unable to meet the demand for a large number of users learning online simultaneously.

To address the issues of high module coupling, poor business scalability, and insufficient high-concurrency processing capabilities brought by the monolithic architecture, this thesis adopts a microservices architecture for the development of the online learning platform. The microservices architecture divides the entire system into multiple independent and lightweight services, enabling each service to be independently developed, deployed, scaled, and maintained. This reduces the coupling between modules and improves the scalability and concurrent processing capabilities of the system.

The main research content of this dissertation is as follows: Firstly, according to the microservice splitting criteria, four major microservices are identified based on business logic, namely, course service, video service, order service, and user service. Then, UML use case

diagrams are employed to analyze the functional requirements of each service. Secondly, the overall architecture of the system is designed, and based on this, the conceptual design of each functional module of the system is conducted. The physical design of the system database is achieved by analyzing the relationships between entities in the E-R diagram. Subsequently, using ASP.NET Core framework, Ocelot as the gateway component, Consul as the service registration component, and gRPC service communication mechanism, the communication and invocation among services are realized. Through the combination of these technologies, an online learning platform based on microservice architecture is implemented. Finally, functional and non-functional tests are conducted on the online learning platform to ensure its stability and reliability during operation.

After a complete development process and system testing, the microservices-based online learning platform designed and developed in this dissertation has been able to operate stably. Currently, the platform has officially entered the internal testing phase of the company. Subsequently, based on actual conditions, necessary optimizations and improvements will be made to the platform to provide users with a quality learning experience and promote the company's business development.

Keywords: Online Learning Platform, Microservice Architecture, ASP.NET Core Framework

目 录

第一章 绪论.....	1
1.1 研发背景和意义.....	1
1.1.1 研发背景.....	1
1.1.2 研发意义.....	2
1.2 国内外研究与开发现状.....	3
1.2.1 微服务架构研究与开发现状.....	3
1.2.2 在线学习平台研究与开发现状.....	5
1.2.3 微服务架构在学习平台中的应用实践	6
1.3 开发研究的思路.....	7
1.4 论文主要内容.....	8
第二章 相关理论与技术	9
2.1 微服务架构.....	9
2.1.1 微服务架构发展历程.....	9
2.1.2 微服务架构的特点.....	12
2.1.3 微服务架构的优势.....	13
2.1.4 服务拆分准则.....	13
2.2 微服务相关技术.....	14
2.2.1 Consul 服务注册管理	15
2.2.2 Ocelot 网关	16
2.2.3 Apollo 配置中心	17
2.3 Identity Server4 统一鉴权中心.....	17
2.4 gRPC 通信机制	18
2.5 本章小结.....	19
第三章 在线学习平台需求分析	20
3.1 可行性分析.....	20
3.1.1 技术可行性.....	20
3.1.2 经济合理性.....	20
3.1.3 操作便捷性.....	21
3.2 用户需求分析.....	21
3.2.1 用户角色分类.....	21
3.2.2 用例分析.....	21
3.2.3 用户权限分配.....	22
3.3 功能性需求分析.....	23
3.3.1 课程管理微服务功能需求分析.....	23

3.3.2 视频管理微服务功能需求分析.....	25
3.3.3 用户管理微服务功能需求分析.....	25
3.3.4 订单管理微服务功能需求分析.....	26
3.4 非功能性需求分析.....	27
3.5 本章小结.....	27
第四章 在线学习平台概要设计	28
4.1 系统整体架构设计.....	28
4.2 系统功能模块设计.....	32
4.2.1 课程管理微服务功能设计.....	32
4.2.2 视频管理微服务功能设计.....	34
4.2.3 用户管理微服务功能设计.....	36
4.2.4 订单管理微服务功能设计.....	37
4.3 数据库设计.....	38
4.3.1 数据库架构设计.....	39
4.3.2 数据库表结构设计.....	39
4.4 本章小结.....	45
第五章 在线学习平台实现与测试	46
5.1 微服务基础组件设计实现	46
5.1.1 系统开发环境介绍.....	46
5.1.2 服务注册中心实现.....	47
5.1.3 服务网关设计实现.....	47
5.2 课程管理微服务实现.....	49
5.2.1 课程管理功能时序图.....	49
5.2.2 课程管理类图设计与接口说明.....	50
5.2.3 课程管理功能页面展示.....	54
5.3 视频管理微服务实现.....	56
5.3.1 视频管理功能时序图.....	56
5.3.2 视频管理类图设计与接口说明.....	57
5.3.3 视频管理功能页面展示.....	59
5.4 订单管理微服务实现.....	61
5.4.1 订单管理时序图.....	61
5.4.2 订单管理类图设计与接口说明.....	62
5.4.3 订单管理页面展示.....	64
5.5 用户管理微服务实现.....	66
5.5.1 用户服务功能时序图.....	66
5.5.2 用户服务类图设计与接口说明.....	67
5.5.3 用户服务页面展示.....	69
5.6 系统测试.....	72

5.6.1 系统测试环境.....	72
5.6.2 系统功能测试.....	72
5.6.3 非功能性测试.....	75
5.7 本章小结.....	77
第六章 总结与展望	78
6.1 总结.....	78
6.2 创新点.....	79
6.3 展望.....	79
参考文献.....	80

第一章 绪论

本章将介绍论文选题背景和在线教育领域的发展趋势，并分析论文的实际意义；再阐述国内外关于微服务架构和在线学习平台的研究与开发现状；最后阐述论文的主要研究内容并对论文的组织结构进行介绍。

1.1 研发背景和意义

1.1.1 研发背景

本文研究的在线学习平台是在企业实习期间所参与开发的，该企业长期致力于建设工程领域信息化、数字化研究与应用，是集工程软件研发、数字工程教育培训、工程信息咨询服务为一体的地方高新技术企业。由于该企业开发的一系列信息化的产品，如 BIM 建模系统、工程计量与计价系统等，建筑业相关用户在使用这些专业软件前也需要信息化产品开发单位进行教学培训，为了降低培训成本，需要开发一套线上教学系统为公司培训业务作为支撑，用户可以直接在教学平台中学习信息化和数字化产品使用方法。此外，该系统还为工程领域的工作者提供学习资源，包括一些职业技能以及工程建设领域相关职业资格，如一、二级造价师、监理工程师和建造师等证书考试的培训课程，让学员通过互联网直接进行线上学习。

随着互联网信息技术的持续革新与深入发展，直接推动了在线教育领域的蓬勃发展。在此背景下，在线学习平台的建设呈现出迅猛增长的态势，特别是 2020 年疫情刚开始时，全国所有学校都实行“停课不停学”政策^[1]，在线学习的用户猛烈增长，相较于上一年足足提升了 20%，由于在线教育的便捷性和低成本，近年来在线学习人数还呈现上升趋势，不仅在各类基础教育和学历教育中得到应用，也在企业产品推广和职业培训等领域得以推广。根据《中国互联网发展报告（2022 年）》预测，2023 年在线教育行业依然会保持稳定增长，市场规模将达到 5901.9 亿元，用户规模将达到 3.52 亿人^[2]，其预测数据如图 1-1 所示。在此背景下，在线教育迅速地发展起来，产生了很多新的教学方式，如慕课、微课、翻转课堂等，传统教育模式受到前所未有的冲击和挑战。与传统教育相比，在线教育有着一定的优势^[3]：首先，它推动了教育资源的高效整合，互联网放大了优质教育资源的影响，使得一名授课教师能够服务成百上千，乃至数万名学生，超越了地域限制；其次，互联网的广泛连通性促进了跨地域、跨领域、跨时间的合作学习，减少了无效的重复工作，加速了整体教育质量的提升；最后，互联网打破了知识获取的壁垒，全球知识体系迅速扩展，丰富的在线教育资源不仅提升了知识获取的速度，也显著降低了学习成本，使人人都能随时随地获得想要的学习资源，从而逐步弥补了由地理位置、时间及师资不平衡造成的教育鸿沟。相信在市场、

技术、政策的多重推动下，在线教育产业将愈发活跃，成为教育行业用于教学的重要方式^[4]。



图1-1 在线教育市场规模预测图

在技术手段上，通过互联网接受线上教学的用户逐渐增多，对于系统服务器的压力会越来越大，系统崩溃发生的次数也越来越多^[5]，例如之前疫情时期的超星尔雅学习网、爱课程网都因为同时在线人数过多，出现过服务器宕机的现象，其原因除了当时系统访问量过大之外，还有可能是因为系统架构设计不合理导致的，如像超星、爱课程、慕课等这些早期开发的在线教学平台基本都是基于单体式架构开发的，该架构的特点是所有的功能模块运行在同一个进程中，对应的数据库也只有一个，导致无法对服务器和数据库做到灵活扩展，在系统访问量突增的情况下，性能方面很容易遇到瓶颈。基于以上问题，本文使用微服务架构来替代单体式架构对系统进行开发，微服务架构的出现解决了单体架构产生的问题，其核心概念是将应用程序的功能拆分为多个独立的服务模块，从而来实现业务逻辑的分离和解耦^[6]。其中每个微服务只负责明确且单一的业务功能，因此即使其中一个服务出现宕机问题，并不会影响其他微服务的运行，整个平台也不会崩溃，从而提高了系统的可靠性和可用性。基于微服务架构开发的好处是系统可拓展性高、可维护性强，能够满足未来在线学习、技能培训与职业教育的发展需求。

1.1.2 研发意义

本文旨在设计并实现一个基于微服务架构的在线教学平台，该系统主要提供一些

与工程建设领域相关的课程，面对的群体大多也都是建筑行业的学生和专业人士，帮助他们学习使用一些工程软件，还可以学习工程造价等相关的专业课程。以下是设计该系统的几点实际意义：

(1)可以提升教学质量

传统学习方式主要依赖于线下的课堂教学，依靠传统工具如黑板和教科书来展开，教师的一次性讲解可能无法满足所有学生的需求，导致部分学生理解困难或复习不易，影响学习效果。网络课程则提供了灵活的学习方式，视频课程可重复观看，使学生能按需多次学习。教师还能制作针对重难点的微课程，优化教学时间，激发学生的自主学习热情。

(2)一个更专业和实用的学习平台

该在线教学平台不同于慕课网这种涵盖多领域的课程系统，慕课中的课程种类齐全，几乎涉及所有的专业，课程的数量也及其庞大，但事物总有两面性，它的优点就可能成为其缺点，由于课程数据较多，涉猎范围较广，会导致课程数据查找困难，有时通过搜索所查到的课程并不是用户自己想要学习的；相关课程过多导致课程质量参差不齐，也会让用户选择困难。本文所开发的系统专注于工程领域的教学，同时与相关的高校教师合作开办课程，将该系统打造成专业化，实用性强的学习平台。

(3)高性能、可扩展性强

系统基于微服务架构进行开发，将整个系统拆分成多个相互独立的服务模块，每个微服务模块都单独开发和部署，降低耦合度，可扩展性强，可以直接通过增加微服务的方式为系统添加新的功能。每个微服务都运行在单独的进程中，进程中的资源会相对宽裕，对访问处理更快速，系统性能也会更高。

1.2 国内外研究与开发现状

1.2.1 微服务架构研究与开发现状

2014 年微服务架构提出以来，已经从理论构想演化为业界广泛采纳和实践的一种软件架构模式。在这个过程中，微服务架构不仅解决了传统单体应用面临的诸多挑战，还推动了软件开发领域的一系列创新和变革^[7]。随着云计算、容器化技术的快速发展，微服务架构在国内外的研究与实践呈现出蓬勃发展的态势。

关于微服务架构，国外的研究者和实践者们对其进行了广泛而深入的研究，推动了微服务架构在理论和应用层面的不断进步。国外的学者主要对微服务架构的服务的治理与管理、微服务架构性能评估以及在开发中的实际应用等方面进行研究。Tokmak

等人^[8]研究如何提高微服务架构中服务的可见性,由于在部署时一些微服务可能会同时复制出多个一起运行,请求如何选择正确的微服务至关重要,Tokmak 团队利用机器学习算法来评估预测出最合适的微服务,该算法的预测准确率可达 90.4%。Blinowski 等人^[9]研究了在 Web 应用程序上比较单体架构和微服务架构的性能和可扩展性,研究结论表明在应用复杂度大的系统中,微服务架构在性能方面和可扩展性方面相较于单体架构都有很大的优势。Chaieb 等人^[10]在研究如何将整体应用程序自动迁移到微服务架构中时,提出了一种方法,该方法可以从给定的整体应用程序中识别潜在的微服务,将源应用程序的源代码作为输入,以便应用不同的机器学习算法将一体化应用程序设计成不同的微服务。Hasan 等人^[11]提出了一种评估微服务架构可维护性的软件体系结构度量指标,这些指标考虑了耦合性、复杂性、内聚性和大小等因素,有助于确保软件体系结构的长期可维护性。Delowar 等人^[12]深入分析了在边缘计算环境中使用微服务的最新研究方向,并探索了其在不同领域的应用潜力,总结出边缘计算和微服务技术的结合在数据处理、性能提升、减少延迟和资源利用等方面具有较大优势。Teresa 等人^[13]提出了一种新的面向微服务架构的交易平台来管理 NoSQL 数据库集群中的大数据处理,并通过模拟伦敦公共汽车服务的实验,验证了该交易平台在微服务体系结构中能够有效处理事务,保证数据的可靠性和一致性。目前许多主流网站都有集成智能聊天机器人以回答用户的疑惑,但独立开发一个聊天机器人非常复杂,于是 Vahdati 等人^[14]提出了一种基于模型驱动的方法论,可以系统地管理智能会话代理的创建,同时与微服务架构相结合,直接将智能会话服务添加到微服务中来实现系统的聊天功能。

在国内,微服务架构的研究与实践正在不断深入。国内研究者主要针对微服务架构的服务治理、服务拆分、服务故障检测和负载均衡算法等方面进行了大量的研究和创新。李艳等人^[15]提出了一个指导微服务拆分的准则和框架,并首次从可维护性角度给出判定准则和方法。通过对某微服务项目的治理为例,验证了该框架和方法的有效性,提高了数据共享性和系统可扩展性。江郑等人^[16]研究出了一种微服务架构下的服务划分策略。此策略首先依据系统服务与属性的关联数据,构建实体与属性的关系图。然后结合微服务架构的特性以及目标系统的需求信息,制定出相应的服务划分准则,对两类顶点间的关联信息进行量化处理,进而形成实体-属性加权图。最终利用加权的 GN 算法,自动实现系统的微服务划分。实验结果充分证明,该方法在提升服务划分时效性方面效果显著,并且在评估指标上,生成的微服务划分方案表现更为优异。王璐等人^[17]从微服务故障检测领域中分析出四个主要的研究方向,分别是软件监控、故障诊断、故障预测、故障复现和测试,基于此提出了一个微服务故障检测的基础框架来阐述四个研究方向之间的联系,为微服务故障检测的发展提供参考。杨乾龙等人^[18]设计了一种基于 Xgboost 的最短预测响应时间负载均衡算法。该算法通过集成学习预测新任务的响应时间,并将任务分配给预测响应时间最短的服务器节点,从而实现了服

务器节点间的负载均衡。实验结果显示,该算法在吞吐量、截止率和平均响应时间等方面均有改进,特别适用于高并发环境下的微服务集群,为微服务架构的负载均衡问题提供了有效的解决方案。张卓等人^[19]在研究中提出了一种基于改进逼近理想及排序法(TOPSIS)的动态权重负载均衡算法,用于解决微服务架构下服务器集群在高并发场景中负载失衡的问题。实验结果显示,在高并发场景下,该算法相比主流负载均衡算法具有更低的响应时间和更高的吞吐量,有效提升了微服务架构下服务器集群的性能。王平等人^[20]提出了一种基于多模态数据和多特征提取(MADMM)的新型微服务系统故障检测方法,该方法从空间和时间角度对指标和日志进行联合分析,同时引入了多粒度对比学习方法,从模态间和模态内两个方面学习稳健的跨模态特征,通过这些特征信息实现微服务系统的自动异常检测。

综上所述,微服务架构在国内外都得到了广泛的研究和应用。随着技术的不断进步和业务的快速发展,微服务架构将继续引领软件开发的创新和发展。同时,我们也需要关注微服务架构在实践中面临的挑战和问题,如服务治理、安全性、性能优化等,并不断探索新的技术和方法来解决这些问题。

1.2.2 在线学习平台研究与开发现状

相较国内而言,国外对于在线学习研究的更早一些,从1994年CALCampus首次推出在线学习课程并提出这一概念开始,在线学习凭借其低成本和灵活性等特点进入快速发展期^[21]。近年来,随着网络技术的不断进步,在线学习成为教育领域的研究热点。国外存在众多知名的在线学习平台,每个平台都有其独特的特点和优势。比如在2012年由吴恩达等人创办的Coursera在线学习平台^[22],提供了来自全球各地的优质教学资源,该平台积极与各高校合作,推出的认证课程可以为学生提供学习成果认证的机会。由麻省理工学院和哈佛大学联合推出的在线教育平台edX^[23],它不仅提供了大量的在线课程,还积极探索互动式学习模式,使学生能够获得更加深入的学习体验。还有像Duolingo(多邻国)这种移动端的学习软件^[24],它主要以语言学习为主,专业性较强;通过游戏化学习方式使得学习过程变得有趣而轻松。

随着网络技术的不断进步和普及,国内在线学习平台呈现出蓬勃发展的态势,市场规模持续扩大,越来越多的学习者选择通过在线学习平台获取知识和技能。2014年5月,高等教育出版社旗下的爱课程网和网易合作推出了中国第一个拥有自主知识产权的慕课平台——“中国大学慕课”。这一平台的出现标志着慕课在中国进入了快速发展阶段,并为广大学习者提供了丰富、优质的在线学习资源。随着越来越多的企业和高校进入在线教育领域,推动了市场的多元化和竞争,这种竞争态势促使在线教育平台不断创新和发展。比如由清华大学发起建立的“在线学堂”、腾讯公司推出的“腾讯课堂”^[25]、还可以包括“哔哩哔哩”平台^[27],它虽然以娱乐性著称,但其

中也包括非常多的学习视频，几乎所有专业领域的课程都能在这找到。

以上这些都属于综合类的学习平台，涵盖多个领域和学科。与之相对应的还有专业技能培训类的学习平台，它们通过与行业内的专业机构或高校合作，提供最新的行业资讯和实践课程，帮助学习者提升专业技能和竞争力。这一类别中还可以划分出很多专业领域，比如以语言类为主的“沪江网校”，它提供英日法韩等 13 种外语在线课程；以 IT 技术为主的“蓝桥云课”^[28]、“我要自学网”等平台，提供了大量优质的 IT 编程设计课程。还有专注于建筑行业培训的在线学习平台，如广联达的“建筑课堂”^[29]，通过其丰富的课程以及优秀的教师团队，帮助学员提升建筑行业的专业能力和素养；还有鲁班软件公司开发的“鲁班学堂”，提供了专业且丰富的 BIM 技术培训、工程造价的课程。本文开发的学习平台借鉴“建筑课堂”和“鲁班学堂”的模式，并结合公司所开发的软件产品以及专业培训业务，在课程方面进行专业设计，为软件使用者提供系统性的软件操作课程，以线上线下相融合(OMO)的教学模式^[30]，为专业培训的学员提供优质的专业课程。目前国内外一些主流的在线学习平台包括多学科综合类学习平台和专业学科教学培训平台，具体如图 1-2 所示。



图1-2 主流在线学习平台

1.2.3 微服务架构在学习平台中的应用实践

在线学习平台在其发展初期主要依赖于 HTML 静态页面，其功能仅限于展示文字和图片，缺乏一定的交互性，导致用户体验不佳。随着 Web 技术的发展，特别是 JSP 和 Servlet 等技术的出现^[31]，单体架构得以应用，使得在线教育网站能够实现用户与网站的动态交互，极大地提升了用户体验。然而，随着在线学习需求的不断增长，传统单体架构在可维护性、扩展性和稳定性方面的局限性逐渐暴露出来，在高并发场景下的

性能表现也不尽如人意^[33]。这些问题严重制约了在线学习系统的发展,无法满足日益增长的系统承载需求。

通过对现有的在线学习系统的调查研究,对慕课网、学堂在线等这些早期上线的学习系统来说,它们的系统架构发展都至少经历过两到三个阶段^[33]。慕课网作为如今高校内最主流的学习平台,其早期使用传统的单体式架构进行开发,将所有的功能模块都运行在同一个服务中,耦合度较高;虽然首次开发比较容易,但随着后续系统业务量不断提升,想要增加其他功能时会非常的复杂,而且在系统出现问题时,定位查询错误代码位置时会消耗大量时间,在后续的运维工作会面临诸多难题,在一定程度上增加了开发和管理成本。之后,慕课网开始将系统架构迁移到微服务架构上,对基础设施的系统进行改造和升级,将业务服务、数据源进行了拆分,并通过服务发现机制和服务注册机制对网站数十个微服务进行管理。

而腾讯课堂系统初期采用的是 SOA 面向服务的架构,虽然它已经从单体架构分离出了多个服务,但每个服务的体量依然很庞大,服务间的耦合性依然很高,而且当一个服务运行时出现故障会导致系统崩溃;后期开发的腾讯课堂对系统架构进行升级,使用微服务架构的思想将系统的业务划分更加细致,使用服务注册组件中的健康检测机制对每个微服务进行监控,当服务出现问题时可以立即重新启动新的服务,通过微服务架构在一定程度上解决了早期学习平台遇到的困难。

目前,还有很多网站在不断的技术更新,逐步升级成了微服务架构模式。而且最新开发的在线教学系统,如网易云课堂等就是直接使用微服务架构进行开发的。在未来,微服务架构将成为众多在线教育平台的优选模式之一。

1.3 开发研究的思路

本文通过使用微服务架构,设计并实现了一个在线学习平台。该平台的开发以某公司项目为背景,其目的是为了与该公司的教学培训业务相结合,将该业务做成线上线下相融合(OMO)的发展模式。在线学习平台的前端设计选用了 Vue.js 与 Axios 等前沿技术,后端开发则依托于 ASP.NET Core 开发框架以及 MySQL 数据库管理技术等。本文主要工作如下:

通过查阅大量有关微服务架构开发的文献和书籍,对比了一些主流的微服务框架,并结合自身的编程技术,确定将 C#作为该系统的主要编程语言。并对微服务架构相关组件进行分析,包括服务网关、服务注册与发现、服务间通信框架等,最终确定了本系统的具体解决方案。

其次,对在线学习平台进行可行性分析、用户需求分析、功能性需求分析以及非功能性需求分析。通过分析,将用户分成前台用户和管理员用户;再根据微服务的划分理念,将本系统分为四个核心微服务,分别是课程管理微服务、视频管理微服

务、订单管理微服务以及用户管理微服务，依据系统整体业务逻辑，并对这些微服务模块实施服务治理，从而构建出一个全面且功能完善的在线学习平台。

然后，对系统的数据库进行设计，分析出系统存在哪些实体，这些实体应该包括用户、课程、视频、订单等，对这些实体之间的关系进行分析并生成出 E-R 图。再分析实体的属性，通过 ORM 技术将实体映射到数据库中，生成数据库表。

最后，设计实现在线学习平台，并对其进行系统测试。通过对微服务架构和相关组件的研究，结合对系统进行的服务划分结果，设计出了整个系统的微服务运行结构，并分别对各个微服务进行具体功能代码的实现。根据实际的业务场景，对各微服务模块进行了全面的功能测试。同时还对系统进行了非功能测试，确保系统的整体性能、兼容性和可靠性。

1.4 论文主要内容

本论文的组织结构旨在系统地阐述基于微服务架构的在线学习平台的设计、实现与测试过程，从绪论开始，逐步深入到相关理论技术、需求分析、系统设计、系统实现与测试来展示系统开发过程，最后进行总结与展望。

第一章，绪论。本章介绍了论文研究背景与意义，阐述在线学习平台的兴起与发展趋势，以及微服务架构在现代软件开发中的应用价值。同时，分析国内外研究现状，指出基于微服务架构的在线学习平台的必要性。最后介绍了本论文的主要研究内容。

第二章，相关理论与技术。本章详细介绍了微服务架构的基本概念和发展历程，还分析了微服务架构使用到的相关技术组件。

第三章，在线学习平台需求分析。本章首先对系统可行性进行分析，然后对在线学习平台的用户需求、功能性需求和非功能性需求进行详细分析。

第四章，系统概要设计。本章详细介绍系统整体架构设计，然后依据微服务模块划分对各个功能模块进行设计，最后对系统数据库进行设计。

第五章，在线学习平台详细设计、实现与测试。本章首先介绍系统的开发环境，然后分析各微服务功能时序图来展示各微服务之间的交互过程，介绍了一些核心方法同时对各个微服务的功能界面进行展示。最后对系统进行功能性和非功能性测试。

第六章，总结与展望。本章对论文的主要研究内容进行总结，指出本文所研究的在线学习系统的不足之处，针对这些不足之处提出相应的改进措施，并展望未来的研究方向和发展趋势。

第二章 相关理论与技术

本章将介绍微服务架构的基本概念和发展历程，对单体架构和微服务架构进行对比分析，然后对系统中所使用到的微服务相关技术组件进行介绍，分析其功能和工作原理。

2.1 微服务架构

微服务架构的核心理念源于 2014 年 Martin Fowler 在其文章中提出的微服务概念，这一概念是在面向服务架构(SOA)的基础上进一步发展而来的^[34]。在大型系统中，传统的单体应用架构往往导致代码结构复杂、维护困难以及扩展性受限。为了解决这些问题，微服务架构通过将大型系统拆分为一系列独立且微小的服务，其中每个微服务聚焦于实现一个特定的业务功能，具有独立的代码库、数据库和部署环境，从而实现了高内聚、低耦合的设计原则^[35]。微服务架构的关键特性之一是其独立性和可扩展性，每个微服务都可以独立部署和扩展，根据业务需求进行灵活的资源分配。在微服务架构中，配置管理和服务发现是两大核心问题，配置中心负责存储和管理各个微服务的配置信息，确保服务的配置一致性和动态更新；注册中心则负责服务的自动发现和注册，使得服务之间可以相互调用和通信；这些机制共同构成了微服务架构的基础设施，为服务的稳定运行提供了保障。通信协议在微服务架构中也起着至关重要的作用，REST 和 gRPC 等协议因其轻量级和跨语言的特性，成为了微服务之间通信的常用选择，这些协议允许不同语言和技术栈的服务之间进行互操作，促进了服务的解耦和复用。此外，为了保证微服务架构的可靠性和稳定性，链路追踪、服务熔断和服务降级等机制被广泛应用。链路追踪技术可以记录服务调用的详细过程，帮助开发者定位问题和优化性能；服务熔断机制可以在服务故障时快速切断调用链，防止故障扩散；而服务降级机制则可以在系统压力过大时，通过简化功能或降低性能来保障系统的基本运行。基于以上微服务的基本组件造就了微服务架构的流行与广泛使用。

2.1.1 微服务架构发展历程

微服务架构的发展经历了从单体架构到面向服务(SOA)架构，再到微服务架构的演变过程，这是一个逐步演进、优化和适应业务需求的发展历程，该过程不仅展示了技术架构的不断进步，也反映了信息行业对于软件可维护性、可扩展性和灵活性的持续追求。

(1)单体架构

在软件开发的初期阶段，为追求快速开发效率，项目往往倾向于选择单体架构作为解决方案。单体架构的核心在于其三层架构设计思想，即表示层、业务逻辑层和数据访问层，将全部功能集中在一个工程中发布。单体架构之所以流行的原因，主要归因于其开发、调试和部署的便捷性，以及相对较低的运维成本。

然而，随着项目规模的扩大，功能需求的增长以及业务逻辑的日益复杂化，单体架构的局限性逐渐显现^[37]。不断增加的系统插件和工具使得代码库日益庞大，导致代码结构混乱，模块间耦合度过高，这种结构上的缺陷使得系统的升级和维护变得愈发困难，不仅影响了开发效率，也增加了运维的复杂性。同时，随着系统复杂性的提升，开发和运维之间的协作难度也逐渐增大。单体架构图如图 2-1 所示。

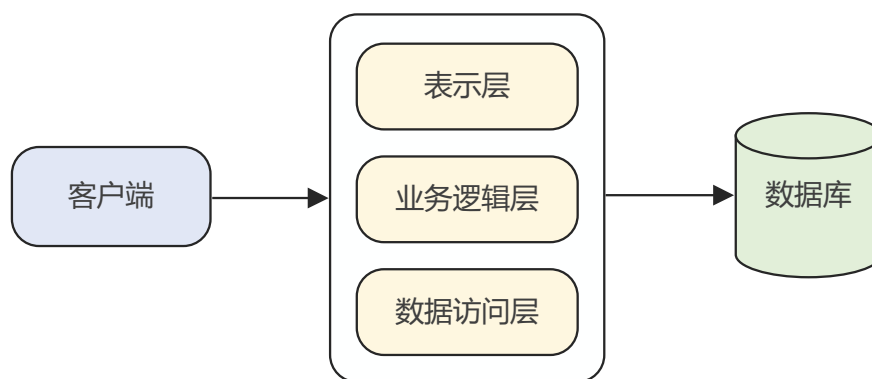


图2-1 单体架构图

(2)面向服务(SOA)架构

在软件架构的演进历程中，面向服务的架构（SOA）作为对单体架构的改进方案，其出现标志着软件架构向更高级别的模块化与解耦发展。在 SOA 中，系统被分解为多个独立的服务单元，每个服务依据功能或业务需求进行明确定义，并通过共享统一的数据存储实现数据一致性。这种分解方式有效降低了系统的复杂性，提高了服务的可复用性和可维护性。服务总线作为 SOA 架构的核心组件，负责协调和管理各服务间的通信。通过服务总线，服务之间能够相互依赖、协同工作，从而完成复杂的业务任务。这种通信机制不仅提高了系统的灵活性和可扩展性，还使得系统能够更好地应对业务变化^[38]。

然而，尽管 SOA 在一定程度上解决了单体架构的存在问题，但它仍然面临诸多挑战，服务模块过大的问题依然存在，这导致服务的拆分和部署仍然较为困难。快速部署小型服务单元在 SOA 中难以实现，这限制了系统的快速迭代和升级；而且服务间的耦合度仍然较高，这使得服务的独立性和可替换性受到一定限制。面向服务(SOA)架构图如图 2-2 所示。

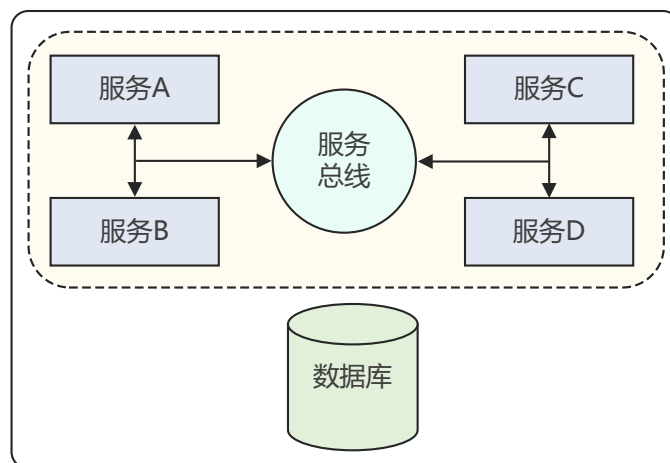


图2-2 面向服务(SOA)架构图

(3)微服务架构

微服务架构作为面向服务架构(SOA)的深化与扩展，进一步推动了软件系统的模块化与解耦。在该架构中，整个系统被划分为多个独立微小的服务，其中每个服务只负责特定的业务功能，其自身可以独立部署和运行。这种服务组件的细分使得每个服务都具备了高度的独立性和可维护性，从而极大地提高了系统的灵活性和可扩展性。

微服务架构中的服务组件可以通过轻量级的通信协议（如 gRPC）进行交互，这种通信方式不仅降低了服务间的耦合度，还提高了系统的响应速度和可靠性。此外，微服务架构支持完全自动化的服务器独立部署，使得服务的部署、升级和扩展变得更加高效和便捷。与 SOA 相比，微服务架构更加强调服务的独立性和轻量化，每个服务都是一个独立的业务单元，可以独立地进行开发、测试和部署^[39]，提高了开发团队的协同效率。同时，服务的轻量化使得系统能够更快速地响应业务变化，并降低了系统的维护成本。微服务架构图如图 2-3 所示。

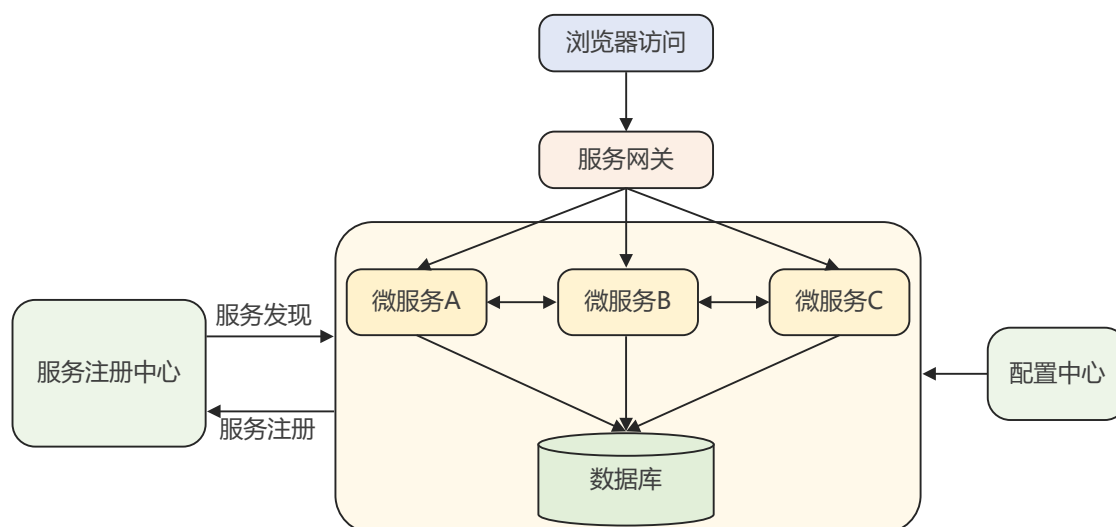
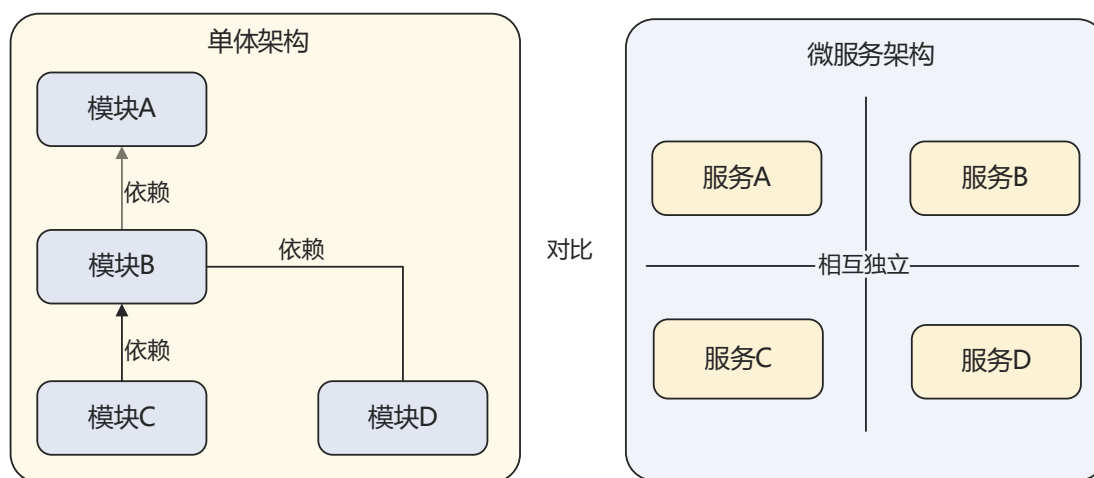


图2-3 微服务架构图

2.1.2 微服务架构的特点

(1) 功能模块划分

单体架构和微服务架构这两种架构模式，都是通过对系统功能进行模块化划分来梳理各个组件间的联系。然而，单体架构的各个模块间存在紧密的相互依赖，代码互相引用，随着业务增长，模块间的耦合度会显著增强，这给系统的更新和维护带来了困难。相比之下，微服务架构的每个服务都是独立的，拥有自己的业务逻辑，还可以通过远程调用接口来实现服务间的通信^[40]，从而保持服务间的独立性，有利于整体架构的灵活性。模块划分对比如图 2-4 所示。



2-4 模块划分对比图

(2) 独立部署

在部署方面，单体架构将所有模块打包成一个执行程序，统一在同一个进程中运行。若需更新某个模块，必须停止整个应用进行修改和重新部署。相反，微服务架构的每个服务可以在不同的端口上独立部署^[41]，一般一个端口对应一个服务。因此，在开发或维护某个特定的服务时，只需要停止该服务，不用去影响系统中的其他服务。部署方式对比图如图 2-5 所示。

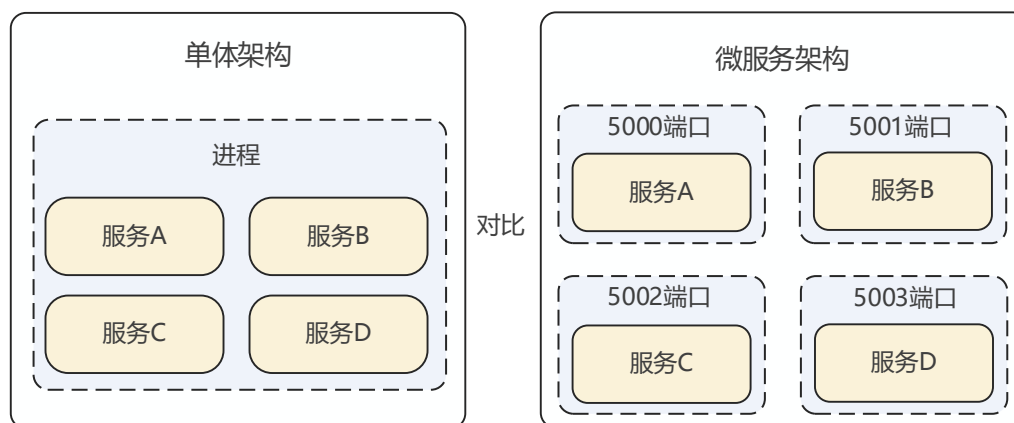


图2-5 部署方式对比图

(3)技术多样性

单体架构要求统一的开发语言和框架，而微服务架构则允许每个服务根据自身需求选择最适合的编程语言、数据库技术和框架，突出了微服务架构灵活性的特点^[42]。例如，一个服务可能使用 C#和 ASP.NET Core，而另一个服务则可能选择 Node.js 和 Express，这种多样性使团队能够针对特定服务选择最佳的工具，从而优化性能和效率。

此外，微服务架构的技术多样性还体现在它支持多种通信协议和接口标准，如 REST 和 gRPC 等，使得服务之间能够以灵活的方式进行交互。每个服务都可以有自己的数据库，这意味着可以根据服务的特定需求选择 SQL 或 NoSQL 解决方案。

总的来说，微服务架构的技术多样性为开发团队提供了很好的灵活性和创新空间，使他们能够为每个服务量身定制技术解决方案，以满足其独特的业务需求和性能目标。这种多样性是现代软件开发中推动快速迭代和持续改进的关键因素之一。

2.1.3 微服务架构的优势

采用微服务架构的优点不仅体现在系统的可维护性和可扩展性上，还体现在团队协作和产品开发效率上^[36]。首先，微服务架构显著降低了系统的复杂度。在传统的单体应用中，随着功能的不断增加，代码堆积问题日益严重，导致系统变得庞大而难以维护；而微服务架构通过将系统拆分为一系列独立且微小的服务，使得每个服务都专注于实现特定的业务功能，使得系统的复杂度变得可控，当有新功能需求时，可以选择将其添加到已有的微服务中，或者根据需要开发新的微服务，这种方式有效避免了无止境的代码堆积，有效降低了系统的复杂度。其次，微服务架构促进了团队协作和分工。在微服务架构中，每个微服务都可以由一个小团队进行独立开发和维护，团队成员可以更加专注于自己的领域，提高了工作效率。同时，当系统出现问题时，可以精准地定位到具体的微服务，从而提出针对性的解决方案，这种精准定位问题的能力大大缩短了解决问题的周期，保证了产品迭代的迅速进行。此外，微服务架构还提供了灵活的技术选型空间，在传统的单体应用中，由于整个系统使用统一的技术栈，技术选型的灵活性受到限制；而微服务架构允许每个服务独立选择适合自己的开发语言和技术框架，这意味着团队可以根据行业发展现状和团队成员的专长，为每个微服务选择最合适的技术方案，这种灵活性不仅有助于发挥开发者的专长，还能提高系统的性能和稳定性。

2.1.4 服务拆分准则

微服务架构的核心理念在于将大型系统拆分为多个独立的微服务，这些微服务各自具备独立运行和提供特定功能服务的能力，它们通过轻量级的通信机制进行信息传递，来满足系统用户的整体需求^[43]。尽管微服务的拆分策略有许多，但总体而言都需

遵循以下几个原则：

(1)业务功能导向原则

每个微服务应有明确的业务边界，负责处理特定的业务领域内的业务逻辑和数据操作。服务内部的功能应紧密相关，以实现高内聚性，从而确保服务的独立性和可维护性。服务之间应保持低耦合性，降低服务间的依赖关系，提高系统的灵活性和可扩展性。

(2)自治与独立部署原则

每个微服务应有其独立的版本控制和生命周期管理，以便于独立地进行开发和维护。服务应能够独立于其他服务进行部署，以确保服务的快速迭代和上线；还应该可以根据其负载情况独立地进行伸缩，以满足系统的性能需求。

(3)服务粒度适中原则

微服务的粒度应适中，既不能过于庞大也不应过于细小，过大的服务会导致复杂性增加，降低系统的可维护性和可扩展性；过小的服务可能导致过多的服务间通信和协调，增加系统的复杂性和性能开销。在确定服务粒度时，需要综合考虑业务需求和技术实现，找到最佳的平衡点。

(4)数据自治原则

每个微服务应拥有自己独立的数据存储和访问权限，确保数据的完整性和安全性。尽量避免不同微服务间直接共享数据，以减少数据一致性和并发访问的问题；服务的数据存储应能够根据其业务需求独立地进行扩展和优化，以满足性能和数据容量的要求。

2.2 微服务相关技术

本文开发的教学系统是基于.NET 平台，采用 ASP.NET Core+Consul+Ocelot+Identity Server 4+gRPC+Apollo+Swagger 为微服务开发技术栈。.NET 是一个由微软开发的开源软件开发框架，支持跨平台，包括 Windows、Linux 和 macOS，它提供了一个全面的编程模型，帮助开发者构建各种类型的应用程序，如桌面应用、Web 应用、移动应用、游戏、物联网应用等。该平台内置了依赖注入功能，并拥有轻量级和模块化的 HTTP 请求管道。.NET 平台在性能方面一直备受赞誉，其运行时环境经过高度优化，提供了出色的执行速度和内存管理；此外，.NET 平台支持异步编程模型，这使得开发者能够编写出响应迅速且能够高效处理并发请求的应用程序。在 TechEmpower 的 Web 框架性能测试排行榜中，与 Java、Python 等框架相比，ASP.NET Core 在主流框架的各项性

能对比中都处于领先地位^[44]。

基于 ASP.NET Core 框架进行学习平台的开发具有一定的优势，各主流编程语言的 Web 框架性能对比如图 2-6 所示，其中排名前两位主流框架的分别是 C++ 的 Drogon 框架和 Rust 的 Actix 框架，ASP.NET Core 排名第三，但与前两位相差并不是很大，而使用最多的 Spring 框架性能只有 ASP.NET Core 的三分之一不到。

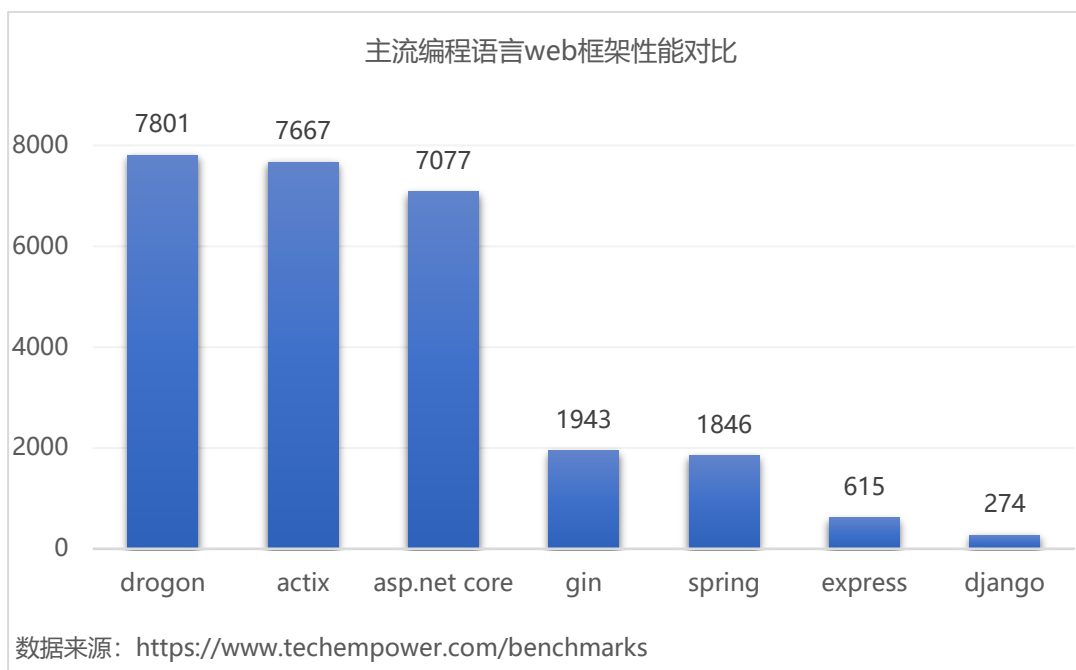
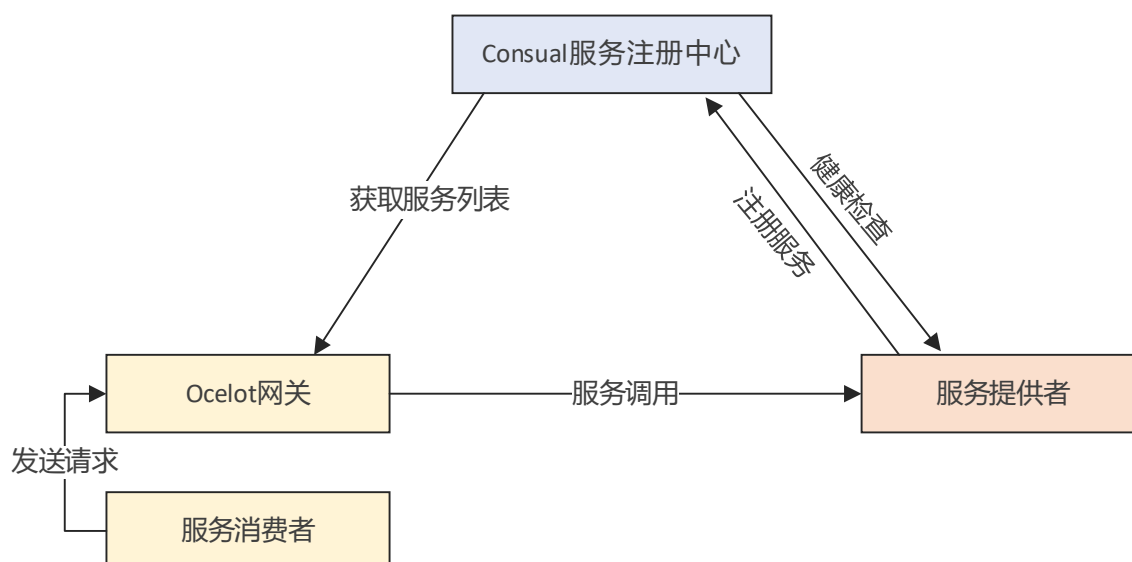


图2-6 主流Web框架性能对比

2.2.1 Consul 服务注册管理

Consul 作为一款由 HashiCorp 公司打造的开源工具，具备高效且灵活的特性，旨在解决分布式系统中的服务发现与配置问题。在微服务架构中，服务发现是一个核心组件，它允许服务相互发现并进行通信，而无需知道每个服务的具体位置。Consul 提供了一个服务注册表，服务可以在其中注册自己的信息^[45]，如 IP 地址和端口号。

Consul 具有健康检查功能，它可以监控服务的状态，确保请求只会被转发到健康的服务实例。此外，Consul 还提供了键值存储功能，用于存储和管理配置信息。各个服务在启动时会向 Consul Server 申请注册，然后将服务信息存储进 Consul Server 中，服务消费者拿到服务注册表后就可以调用其他服务中的方法。Consul 服务注册中心原理图如下图 2-7 所示。



2.2.2 Ocelot 网关

Ocelot 是基于 .NET Core 实现的一款开源的轻量级 API 网关，常用于微服务架构中，它的主要作用涵盖了请求转发、动态路由、访问控制、认证鉴权、负载均衡以及限流熔断等多个方面。

Ocelot 网关作为客户端和微服务系统之间的中间层，承担着请求转发的核心任务。它接收来自客户端的请求，并根据预先配置的规则，将这些请求转发给相应的微服务，这种机制不仅简化了客户端的编程工作，还降低了客户端与多个微服务直接通信的复杂性。同时还支持动态路由配置，展现了较高的灵活性，可以根据不同的请求条件，将请求路由到不同的微服务^[46]。这种动态路由能力使得系统能够轻松应对复杂的业务场景，并随着业务需求的变化而灵活调整。

此外，访问控制和认证鉴权是 Ocelot 网关提供的另一重要功能。通过对进入系统的请求进行身份验证和权限检查，网关确保了微服务的安全性，可以有效防止未经授权的访问和恶意请求。在微服务架构中，通常会有多个相同类型的微服务实例运行在不同的服务器上，网关可以根据负载均衡算法（如轮询、随机、最少连接数等）将请求分发到这些实例上，以实现负载均衡和故障转移。

最后，Ocelot 网关还具备限流和熔断机制，以应对突发流量和故障情况。限流功能可以限制请求的频率或并发数，防止系统因过载而崩溃；而熔断机制则能在某个微服务出现故障时，迅速将请求转发到其他可用的服务或备用方案上，从而保障系统的持续可用性和稳定性。

2.2.3 Apollo 配置中心

Apollo（阿波罗）配置中心是一个分布式配置管理系统，专为微服务架构和大规模应用场景设计，它在架构设计、功能特性、性能优化以及可靠性保障等方面均展现出了其优势之处。

它采用了高可用、高可扩展的架构，确保了系统在面对复杂环境和不断增长的需求时仍能保持稳定和高效。通过实时性配置更新、命名空间管理、灰度发布以及细粒度的权限控制等核心功能，Apollo 不仅提升了配置管理的灵活性和效率，还显著降低了配置变更带来的风险。此外，该系统在性能优化和可靠性保障方面也表现出色，通过多项技术手段确保了系统在高并发场景下的良好性能以及数据的安全性和可用性。Apollo 配置中心以其全面的功能特性、卓越的性能和坚实的可靠性，为现代软件系统的配置管理提供了强有力的支持。

2.3 Identity Server4 统一鉴权中心

系统内将 Identity Server4 独立实现为认证鉴权服务，以完成统一的认证鉴权功能。在 IdentityServer4 项目中，实现了 API 资源的定义、客户端访问资源范围的配置、账户密码验证以及数据返回格式等功能。然后在 API 网关项目中进行了统一认证的集成，而且 Ocelot 网关对 Identity Server4 进行了良好的集成，因此只需在网关中添加统一的认证代码即可，各个微服务实例则只需关注各自的业务逻辑代码。

在客户端尝试访问 API 资源时，如果缺乏有效的认证凭证，网关会返回 HTTP 状态码 401，表示未经认证的请求。此时，客户端需要先通过网关向 Identity Server4 请求一个 Token。获取到 Token 后，客户端再次携带 Token 通过网关请求相同的 API 资源，则可以正确获取所需资源。

首先，客户端（Authorization Client）向资源所有者（Resource Owner）发出一个授权请求，希望获取对受保护资源的访问权限。资源所有者在这个过程中可能需要进行身份验证或确认授权。随后，资源所有者向客户端返回一个授权许可，这个授权许可通常是一个临时的凭证，表明资源所有者已经同意客户端的访问请求。接着，客户端使用这个授权许可向认证服务器（Authorization Server）请求一个访问令牌（AccessToken）。这个过程可能涉及客户端与认证服务器之间的交互，其中客户端可能需要证明自己的身份，并展示由资源所有者给予的授权许可。认证服务器验证客户端的身份和授权许可的有效性后，生成一个访问令牌，并将这个令牌返回给客户端。这个访问令牌是客户端访问受保护资源的凭证。最后，客户端使用这个访问令牌向资源服务器（Resource Server）请求访问受保护的资源。资源服务器会验证访问令牌的有效性，如果有效，则允许客户端访问相应的资源。箭头的方向表示数据流的走向，即

从请求发起方流向处理方，再返回结果给请求发起方，整个授权交互流程如图 2-8 所示。

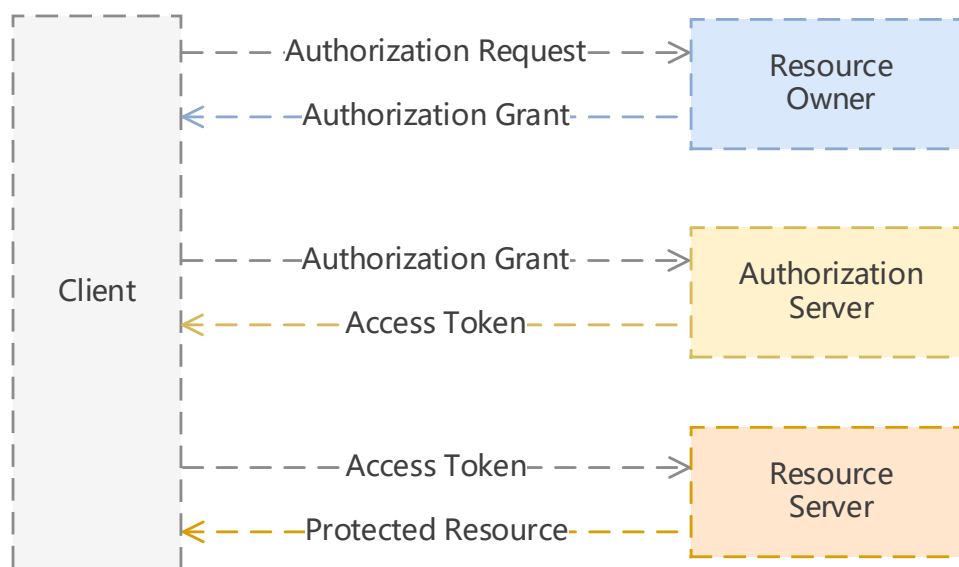


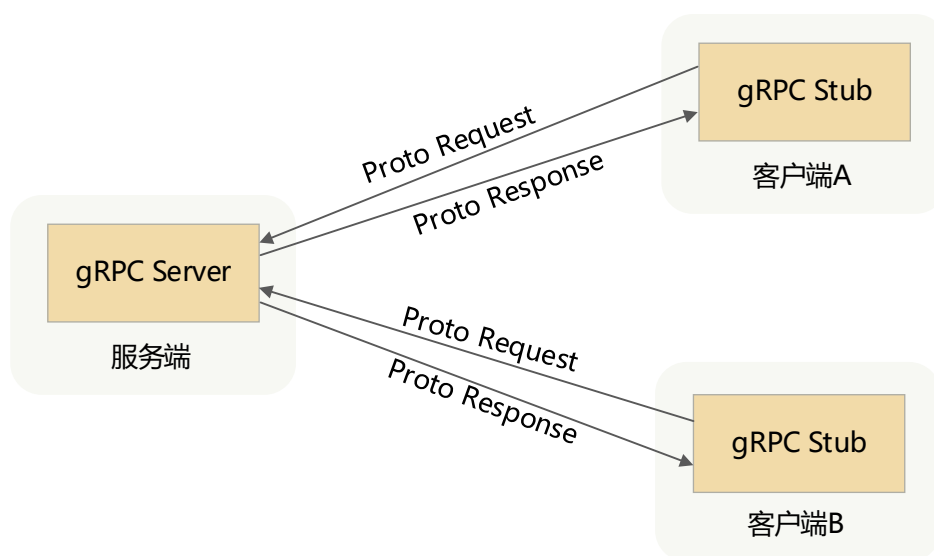
图2-8 授权Token交互流程图

2.4 gRPC 通信机制

gRPC 是一种高性能、跨平台、跨语言的远程过程调用(RPC)框架，它由 Google 开发并基于 HTTP 2.0 传输层协议，这是一种全双工通信协议，支持多路复用和头部压缩等技术，能够更好地处理并发请求，减少连接开销，并在单个连接上实现双向通信^[47]，这对于微服务架构中的服务间通信至关重要。

gRPC 使用 Protocol Buffers（简称 ProtoBuf）作为数据交换格式，其中 ProtoBuf 是一种轻量级且高效的结构化数据存储格式，开发者可以通过它来定义消息类型和服务接口。与 JSON 或 XML 等传统数据交换格式相比，ProtoBuf 具有更小的数据体积和更快的序列化/反序列化速度，非常适合用于微服务之间的数据传输。gRPC 通信机制还支持多种通信模式，如一元调用、服务器流、客户端流和双向流等，这些模式为微服务架构中的不同业务场景提供了灵活的通信方式，满足了复杂业务逻辑的需求。

gRPC 调用示例图如图 2-9 所示，在服务端，通过 gRPC 的编译器工具将 ProtoBuf 格式的文件编译成相应的代码来实现服务接口；服务端接收来自客户端的请求，并根据请求的方法调用相应的逻辑进行处理，然后返回响应结果。在客户端，同样根据生成的代码实现接口定义的服务，客户端通过 Stub 类创建 RPC 请求，并发送给服务端；客户端还负责处理服务端返回的响应结果。



2-9 gRPC调用示例图

2.5 本章小结

介绍了微服务架构的基本概念，并分析了从单体架构到面向服务架构，再到微服务架构这一发展历程，然后对单体架构和微服务架构进行对比分析，得出了微服务架构的优势和特点；并简要阐述了微服务的拆分准则。之后还介绍了微服务相关技术组件，确定了本系统所使用到的技术栈。

第三章 在线学习平台需求分析

本章将根据在线学习平台的实际研究背景进行详细的需求分析。首先从技术、经济和系统操作方面进行可行性分析。再根据系统的实际需求对平台用户和系统管理员这两类关键用户群体的功能性和非功能性需求进行分析，然后对在线学习平台的主要业务功能进行详细的需求分析。通过对平台的多维度论证和需求分析，为系统的整体设计和实施提供了有力的理论支撑和实践指导，确保了平台设计的科学性和实用性。

3.1 可行性分析

在软件开发的初期阶段，需要制定一份详尽且精准的预测性报告，这份报告的核心内容就包括可行性分析^[48]，其中涵盖了技术适用性、经济效益和用户体验等多个角度的评估，旨在确定项目是否可行以及预期达成的目标。可行性分析是软件工程的关键步骤，它决定着项目是否有意义，项目成功实施的可能性，以及是否能充分满足用户的需求。

3.1.1 技术可行性

在开发过程中，选择并运用合理的开发技术非常重要，其中涉及如何挑选合适的技术，以及如何评估这些技术是否符合项目需求，开发者需结合自身的技术实力，确保选用的技术能有效地推动软件开发目标的实现。在国内，各类在线学习平台的技术选配各具特色，本文在项目中则选择了 ASP.NET Core 框架作为后端支持，配合 MySQL 和 Redis 进行数据持久化存储；同时舍弃了传统的本地存储方案，转而拥抱云服务，采用阿里云的 OSS 对象存储和 VOD 视频点播服务对课程视频文件进行管理，这些都是近几年软件开发的主流技术，且通常有丰富的开发文档，便于开发团队快速学习和掌握，从而保障项目开发进度按计划推进。

3.1.2 经济合理性

经济合理性分析是基于经济利益的视角，全面考量系统开发的成本及后续维护成本，以评估其合理性与经济效益。本文中所设计的在线学习平台，采用互联网技术并通过集群部署少量服务器，就能实现对线上培训课程和企业优质课程资源的高效整合，为用户提供便捷、灵活的学习服务。用户在闲暇之余，能够随时随地通过平板或电脑进行学习，有效巩固线上课堂的学习成果。同时，该在线学习平台还能够与线下课程相融合，形成了良好的互补关系。总而言之，该平台在经济合理性方面表现出色，通过技术创新和资源整合，实现了成本的有效控制和经济效益的提升。

3.1.3 操作便捷性

操作便捷性分析关注于从用户操作友好性的视角来审视系统，评估其基本操作是否满足普遍标准，以确保各个模块功能的正常使用。本系统在设计过程中，采纳了由“饿了么”公司开发的 Element-UI 组件库，其可以为用户带来一致的界面风格、丰富的交互组件、良好的响应性能、易于理解的信息展示等好处。因此，经过综合评估我们可以确认，该项目在操作便捷性方面已经达到了预期的标准，为用户提供了优质、便捷的使用体验。

3.2 用户需求分析

3.2.1 用户角色分类

(1)前台用户角色分析

前台用户作为在线学习平台的直接使用者，他们对系统功能上的需求主要包括：注册并登录系统、浏览系统页面、查看首页信息、管理个人信息、试看免费视频内容、利用分类搜索找到相关课程、选择购买付费课程。

(2)后台管理员角色分析

作为后台系统的管理者，后台管理员的工作主要针对系统数据进行管理，例如一些增删改查的操作，主要包括管理用户信息、管理系统课程、管理课程分类、管理订单交易等。

3.2.2 用例分析

本节运用 UML（统一建模语言）来构建分析前后台系统的用户用例图^[49]。前台系统包含了前台用户功能、前台课程操作以及前台课程展示等多个功能，其中前台用户功能包括了用户登录、查看个人信息、查看订单等操作，前台课程操作包括了加入课程、购买付费课程等操作，前台课程展示包括了首页课程展示、查看课程详情等。前台用户用例图如下图 3-1 所示。

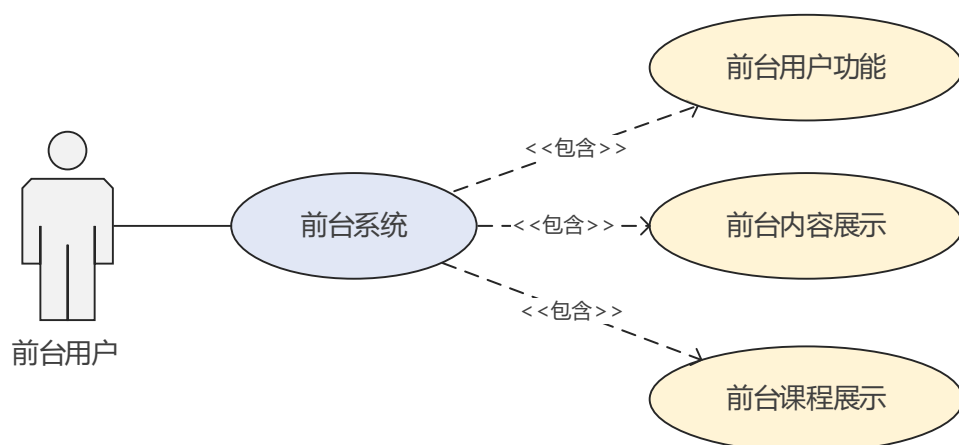


图3-1 前台用户用例图

后台管理员用例图包括了课程管理、课程分类管理、交易订单管理与用户信息管理等功能。课程管理包括发布课程、查看课程列表与管理课程等操作；视频管理包括上传视频、添加视频到课程、删除视频等操作；交易订单管理包括查询订单、修改订单以及对订单进行隐藏等操作。用户信息管理包括查询用户、锁定隐藏用户等操作。后台管理员用例图如下图 3-2 所示。

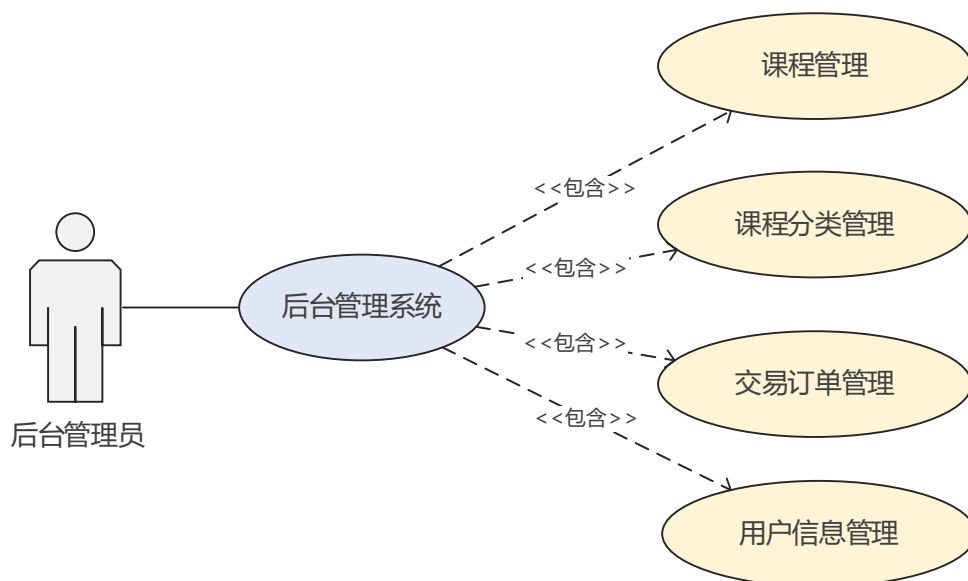


图3-2 后台管理员用例图

3.2.3 用户权限分配

权限设置主要涉及两方面：首先，对数据权限进行控制，明确用户需要访问的数据范围，确保敏感数据不被未经授权的用户获取或篡改。其次，对用户功能权限进行限制，根据用户的角色和职责，将功能权限分配给对应的用户，确保每个用户只能执行其被授权的操作^[50]。系统划分了两种角色，即前端用户和后台管理员。前端用户需通过手机短信验证注册并登录，登录后将获得前台用户功能、前台课程操作及前台课

程展示的权限。而后台管理员则利用账户密码登录后台管理系统，成功登录后，将拥有课程管理、视频管理、交易订单管理和用户信息管理的权限。具体的权限分配原则如表 3-1 所示，其中“○”表示用户有权限使用该功能，“×”表示没有该功能权限。

表3-1 系统角色权限分配表

功能模块 \ 用户角色	前台用户	后台管理员
前台用户功能	○	×
前台课程操作	○	×
前台课程展示	○	×
课程管理	×	○
视频管理	×	○
交易订单管理	×	○
用户信息管理	×	○

3.3 功能性需求分析

3.3.1 课程管理微服务功能需求分析

为了清晰阐述各微服务功能需求，本小节将采用用例图进行分析。课程管理微服务主要包括课程管理和课程展示。从后台管理员角度分析，课程管理涉及课程分类和课程处理。课程管理微服务后台管理员用例图如图 3-3 所示。

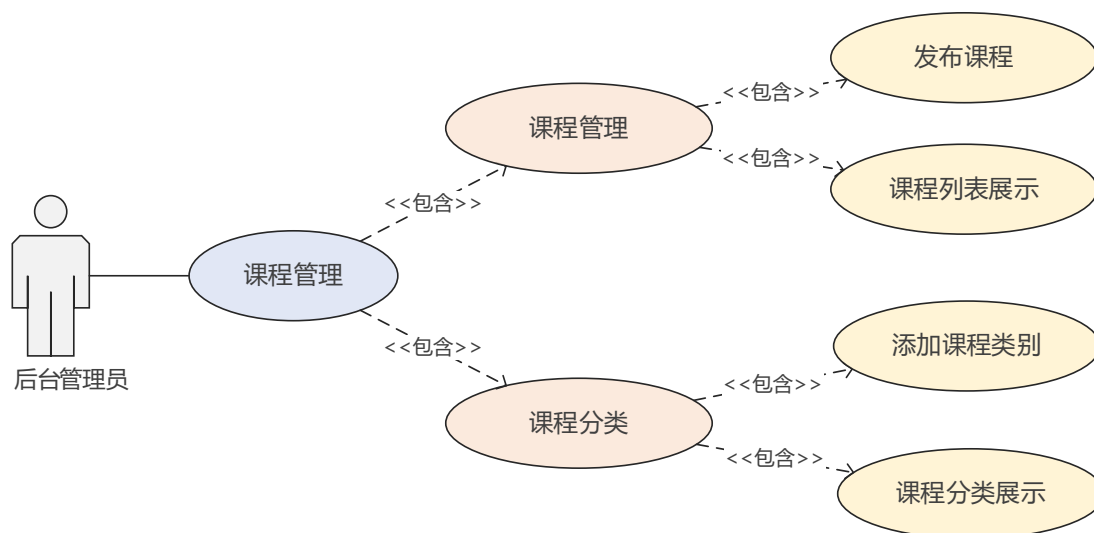


图3-3 课程管理微服务后台管理员用例图

(1)课程管理功能

课程管理功能分为发布课程与课程列表展示两个子功能。发布课程功能分为三个步骤，首先点击添加新课程，填写课程基本信息，包括课程封面、课时、课程价格及课程描述等信息，然后设置课程大纲，包括了添加章节信息、上传对应的课程视频等操作，最后检查确认填写的课程信息无误后即可发布课程。课程发布完成后，后台管理员可以检索已发布课程，若发现信息填写有误可以对其进行修改。

(2)课程分类功能

课程管理微服务在后台系统的课程类别页面中使用树形结构展示课程分类情况，将软件学习、线上课堂、线下精品三个方向作为课程的一级分类，它们的子类作为其所属的二级分类，可以通过点击分类项中的添加子级分类按钮实现课程类别添加功能，也可以点击删除按钮将某课程分类信息进行删除。

从前台用户角度出发，该部分具体分为首页课程展示、课程分类展示两个功能，课程管理微服务前台用户用例图如图 3-4 所示。

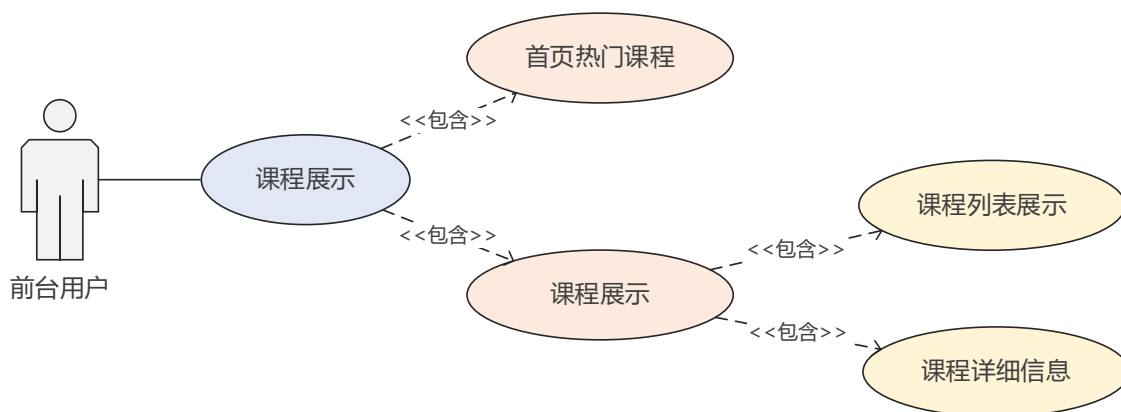


图3-4 课程管理微服务前台用户用例图

(3)首页课程展示

用户访问系统，在网站较为显眼的中心位置，就能看见首页展示的课程。该功能会从课程数据库中按一级分类分别检索出 8 个相关课程进行展示，若前台用户想要学习这些首页展示的课程，可以直接点击进入该课程简介界面，如果是免费课程，用户可以直接加入该课程进行学习。

(4)课程分类展示

课程展示功能可分为课程详情和课程列表展示两部分。当前端用户在首页未能找到所需课程时，他们可以点击课程选项进入分类展示页。在这里，他们能按课程类别初步筛选，选择一级分类后，会出现相对应的二级分类列表，用户点击感兴趣的二级

分类，系统就会展示相应的课程视频。课程详情子功能是前台用户点击某课程后，会跳转到课程详情页面，在该页面中用户可以查看课程封面、课程章节内容、课程价格等信息。

3.3.2 视频管理微服务功能需求分析

视频管理微服务涵盖了视频点播配置和视频管理两大功能。该服务原理是将云存储技术引入了微服务架构。在视频点播配置中，主要任务是引入阿里云视频点播 SDK，集成播放插件，并获取播放凭证和地址等关键信息。视频管理功能的作用主要是通过 API 接口上传视频、执行视频删除等操作。视频管理微服务用例图如图 3-5 所示。

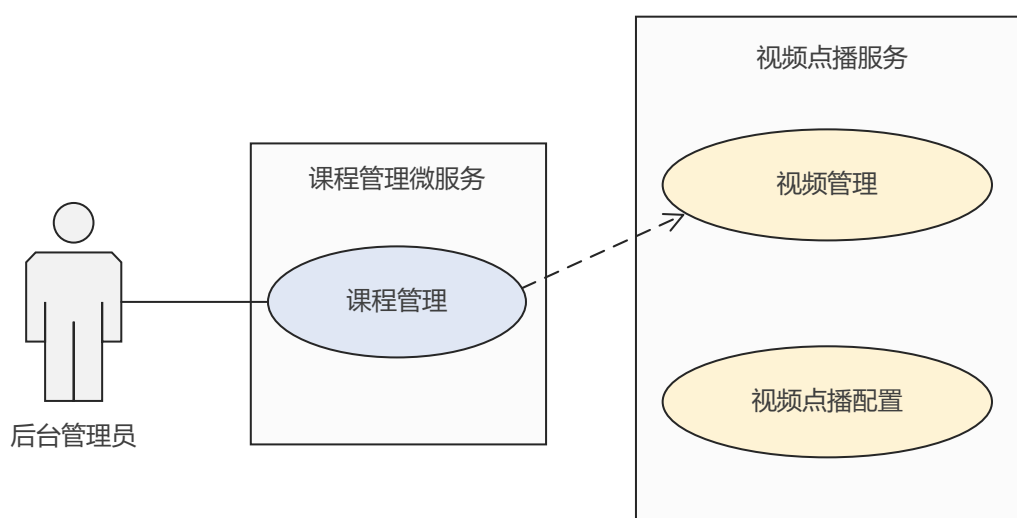


图3-5 视频管理微服务用例图

3.3.3 用户管理微服务功能需求分析

用户管理微服务主要功能包括单点登录管理和用户注册。在微服务系统中，用户访问系统时会使用到多个微服务模块的功能，而每个微服务都有不同的端口号，服务间的访问可能存在跨域问题，而此时的单点登录 SSO(Single Sign On)机制正好能解决跨域问题^[50]。单点登录管理子功能主要用于系统全局的 Token 安全管理，未登录的用户需先登录账户才能使用各项功能。如果用户没有账号，则需要跳转到用户注册中心，在这用户需要填写账户信息并发送短信验证码进行账号注册。用户管理微服务用例图如图 3-6 所示。

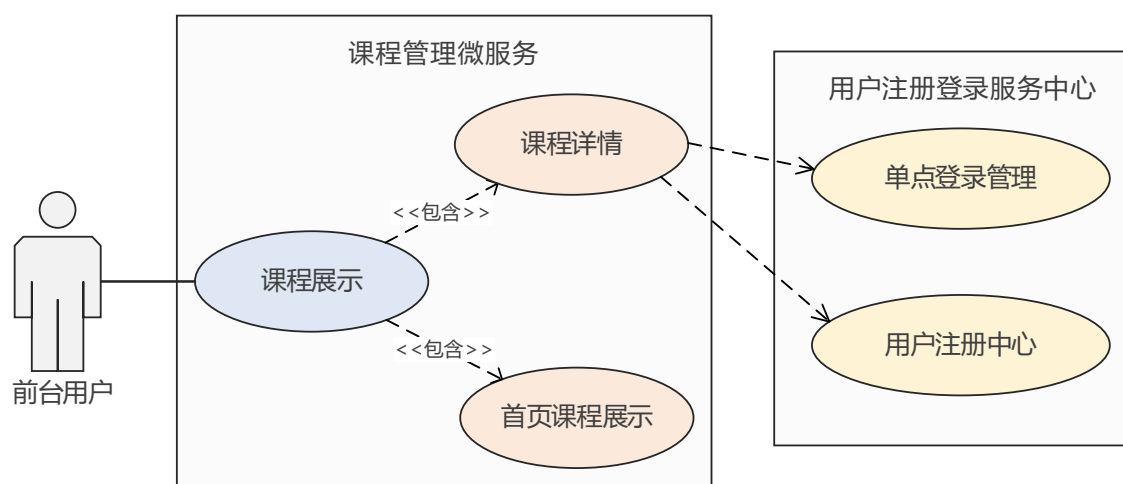


图3-6 用户管理微服务用例图

3.3.4 订单管理微服务功能需求分析

订单管理微服务的核心是订单管理和第三方支付。一旦前台用户选择付费课程下单，订单管理功能将根据用户的登录信息追踪订单状态，生成订单详情信息，并在支付完成后引导用户进入课程学习页面，便于他们开始视频学习。第三方支付管理子功能的核心工作在于将微信支付功能整合进系统，它一开始在订单管理子功能中设定统一的下单接口。下单后，支付流程正式启动，由后台服务器向微信服务器发送请求。然后微信支付服务器会产生一个支付二维码，用户通过微信扫描此码，核实课程信息并支付。同时，此操作也会同步更新后台数据库的状态，从而完成全部的订单处理和支付流程。订单管理微服务用例图如图 3-7 所示。

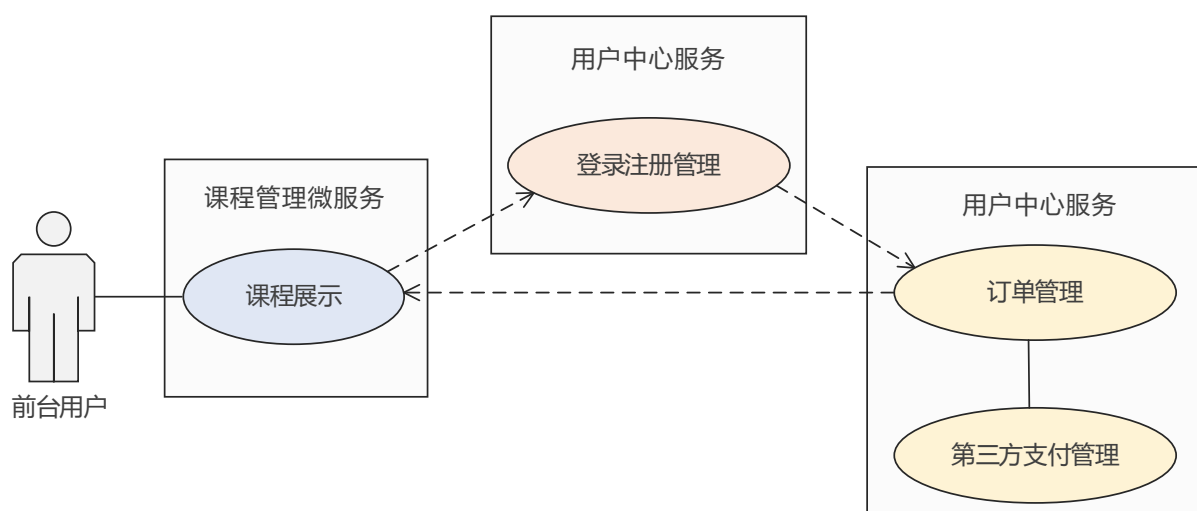


图3-7 订单管理微服务用例图

3.4 非功能性需求分析

非功能性需求分析关注的是除实际业务需求之外的其他关键特性，是确保系统质量的关键环节。通过对可靠性、安全性、可维护性和易用性等方面进行分析，可以识别和定义系统所需的关键非功能属性，从而确保系统能够满足用户的期望和需求。

(1)系统的可靠性

可靠性要求在规定的运行时间内和特定的操作环境下，系统不仅能够稳定、连续地执行其预定的功能，而且应具备强大的错误预防和容错能力。本文的在线学习系统使用微服务架构开发，利用 Consul 服务注册发现组件来对系统服务进行管理，当服务出现异常时，该组件会立即启动新的服务来代替异常服务，确保系统的正常运行。

(2)系统的安全性

安全性需求分析是确保系统能够抵御各种潜在威胁，保护数据完整性和机密性。本文对不同用户的功能访问权限进行了控制，通过设定严格的身份验证和授权机制，防止未经授权的数据访问和泄露。本系统将视频文件和图片通过云存储的方式保存，其提供了强大的数据备份和加密功能，来应对数据丢失损坏和被窃取的风险。

(3)系统的可维护性

本系统通过将整个应用程序拆分成多个小型、独立的服务，每个服务负责单独的业务功能，这种形式使得服务之间的耦合度降低，对服务的修改或扩展不会对其他服务产生影响。当需要修复 Bug 时，只需要专注于特定的服务，而无需担心对整个系统的影响，这显著提高了本系统的可维护性。

(4)系统的易用性

易用性分析主要关注用户在使用系统时的便捷性、直观性以及用户的学习成本等因素。本文的在线学习系统采用简洁明了的界面设计，功能按钮和导航菜单清晰易见，系统操作流程简单，用户只需一次点击即可进入课程学习页面。系统提供实时反馈和提示功能，在用户完成操作后显示成功或失败的提示信息，方便用户了解操作结果。

3.5 本章小结

本章节主要探讨的是在线教学系统的总体需求分析。首先是对系统的可行性进行评估，以保证项目的实用性和开发可行性。之后借助 UML 用例图来分析两类核心用户的需求。然后详细分析了各个微服务的功能需求，对它们进行明确的功能划分。最后阐述了系统的一些非功能需求。

第四章 在线学习平台概要设计

本章将根据需求分析并结合实际业务场景进行系统概要设计，首先对系统整体架构进行设计，然后根据每个微服务的功能模块图对各微服务进行功能划分。最后从数据库架构设计、和表结构设计方面对在线学习平台的数据库进行分析与设计。

4.1 系统整体架构设计

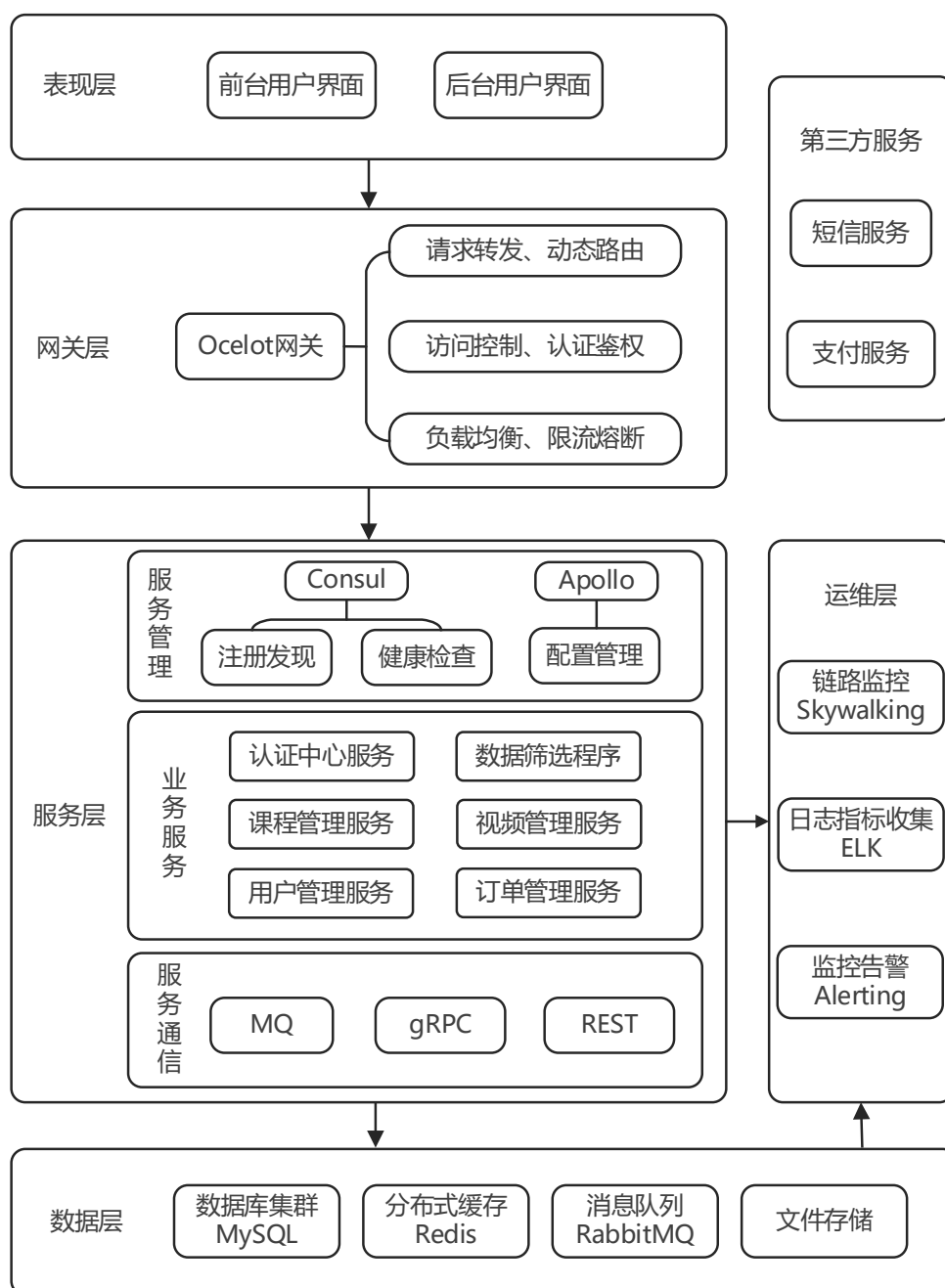


图 4-1 系统整体架构图

基于微服务架构的在线学习平台的整体架构图如图 4-1 所示，系统架构主要分为表现层、网关层、服务层、数据层、第三方服务和运维层六个部分，下面对各个部分进行详细介绍。

(1)表现层

表现层是用户与系统交互的直接界面，通常包括 PC 端的前台用户界面和后台管理界面，本文的在线教学平台的前端都是基于 Vue.js 开发的。在表现层，用户可以通过界面进行各种操作，如用户登录、课程学习、视频上传等。表现层负责发送用户的请求，并将结果以可视化的方式呈现给用户，是用户与后端服务进行交互的重要窗口。

(2)网关层

网关层提供了所有微服务的统一入口，用户请求通过网关发送到可处理该请求的微服务，从而简化了客户端与服务之间的交互；网关还能实现动态路由，灵活调度服务，降低客户端与服务的耦合度；还可以通过对请求进行鉴权校验来增强系统的安全性。本文系统架构中的网关层使用了 Ocelot 网关，所有来自客户端的请求都需要通过这个网关进行转发；它还支持多种路由策略，如路径映射、请求转发和重定向等。这些策略可以根据具体的业务需求进行灵活配置，以实现请求的精确路由和高效处理。在认证和鉴权方面，Ocelot 网关同样表现出色，它拥有多种认证方式，如 JWT、OAuth2 等，以确保只有经过授权的请求才能访问后台服务。Ocelot 网关还提供了一系列附加功能，如请求聚合、负载均衡、限流熔断等，这些功能可以根据实际需求进行选择和配置，以进一步增强系统的稳定性和可靠性。通过请求聚合功能，网关可以将多个请求合并为一个请求进行处理，从而提高请求的处理效率；负载均衡功能可以将请求均匀地分配到多个后端服务上，从而提高系统的整体性能；而限流熔断功能则可以在系统压力过大时自动断开连接，以保护后台服务免受过载影响。

(3)服务层

服务层，作为整个微服务系统的核心要素，涵盖了服务管理、业务服务以及服务通信这三大关键模块。其中服务管理模块的主要功能包括服务注册发现、服务健康检查和系统的全局配置管理。本系统采用 Consul 作为服务注册发现的工具，具有安全、高可用、分布式、可扩展性强等特点。提供了一系列核心功能，主要包括服务注册发现、健康检查、分布式键值存储以及多数据中心部署等功能，有效地解决了分布式和微服务架构系统中的服务治理和管理问题。在本系统中，使用 Apollo 工具对系统全局配置进行管理，它具有高性能、可靠性强和功能全面的特点，使得开发人员能够高效地管理应用的配置信息，提高系统的灵活性和可维护性。

业务服务模块实际上是由在线教学系统中的核心业务微服务和其他业务服务程序所构成，其中核心业务微服务主要包括课程管理服务、视频管理服务、订单管理服务和用户管理服务。其他业务服务程序包括采用了 Identity Server4 框架来构建的认证中心服务，Identity Server4 是一个专为 ASP.NET Core 设计的中间件，它实现了 OpenId

Connect 和 OAuth2.0 协议，为整个系统提供了一个统一的认证授权中心。这些服务的协同工作，确保了整个在线教学平台高效稳定的运行。

服务通信模块的作用是通过使用一些通信传输协议或工具，实现了微服务之间的数据交互和信息共享，本系统主要使用的是 gRPC 通信机制，它基于 HTTP 2.0 协议和 Protocol Buffers 序列化协议，是一个开源、高性能、可靠和灵活的微服务通信框架。能够很好的满足微服务架构中服务间通信的需求。

(4)数据层

数据从主要负责业务数据、文件、消息和缓存等各类资源的存储与访问。为实现高效且稳定的数据管理，本文的系统使用关系型数据库 MySQL 来进行数据的存取，并建立数据库集群，即一主库两从库的模式，同时使用选主机制来确保数据一致性，通过读写分离策略优化了数据库的读写效率。此外，系统还引入了分布式缓存 Redis 来提供高效的缓存服务，以及开源工具 RabbitMQ 作为消息队列，实现了异步通信与任务处理。

(5)第三方服务

第三方服务指的是那些由外部厂商提供，可通过购买或免费使用来增强系统功能的服务。这些服务通过厂商提供的 API 接口集成到系统中，为系统增添了特定的功能。在本系统中，采用了阿里云的短信服务，通过调用其 API 接口，实现了用户手机号验证和短信注册新用户的功能。还引入了第三方支付功能，为用户提供了更加便捷和安全的支付方式。

(6)运维层

运维层负责整个系统的监控、管理和维护。其中的链路监控 Skywalking 是一个开源的应用性能监控系统（APM），专为云原生、微服务和基于容器的架构设计，它具备链路追踪、指标收集、服务和实例的拓扑分析等功能，有助于开发者和运维人员更好地理解 and 优化分布式系统的性能，在本系统中用于监控微服务架构中各服务间的调用关系、性能瓶颈及异常问题，以便团队快速定位和解决生产环境中的性能问题。日志指标收集 ELK 是 Elasticsearch、Logstash 和 Kibana 三个开源软件的组合，常用于构建日志管理和分析系统，Elasticsearch 是分布式搜索和数据分析引擎，Logstash 负责数据收集、转换和发送，Kibana 则用于搜索、查看和交互存储在 Elasticsearch 中的数据。本系统用它集中收集、存储和分析包括应用程序日志、系统日志、安全日志在内的各种日志数据，帮助运维人员实时监控系统的运行状态，发现潜在问题，并进行优化调整。监控告警 Alerting 指的是监控系统中的告警功能，它是基于预设规则和阈值对监控数据进行实时分析的。在系统运维中，Alerting 帮助团队及时发现系统故障、性能下降等问题，以便迅速采取行动，避免或减少损失。

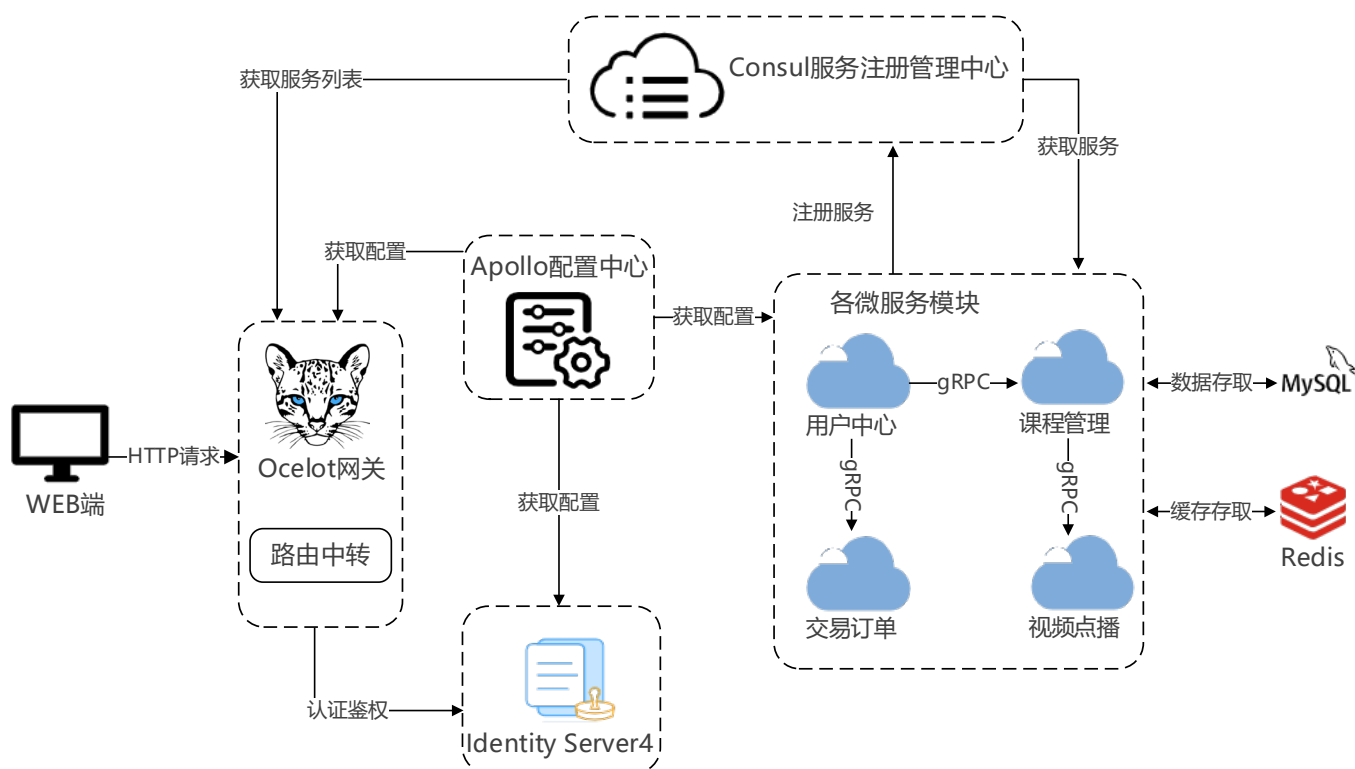


图 4-2 请求处理流程图

微服务系统请求处理的详细流程图如图 4-2 所示。在微服务系统启动时，首要步骤是启动 Consul 服务，随后启动 IdentityServer4 认证服务。紧接着，各个业务微服务相继启动。在启动过程中，每个微服务实例会向 Consul 发出请求，拉取全局配置信息。根据这些配置，各实例向 Consul 注册其服务名、IP 地址、端口号等信息。注册成功后，Consul 会对每个微服务进行健康检查，确保每个服务实例的正常运行。最后，开启 Ocelot 网关，为整个系统提供统一的入口和路由管理。

在微服务架构中，Web 端发出的 HTTP 请求会先经过 Ocelot 网关进行接收和处理。当请求试图访问系统内的受保护资源时，Ocelot 网关会对请求头中是否包含有效的授权令牌 Token 进行检查，如果请求头中包含 Token，Ocelot 网关会进一步与 Identity Server4 认证服务进行交互，发起一个验证请求，确保该 Token 的有效性。若验证未通过或请求头中不包含 Token 信息，Ocelot 网关会阻止请求的访问，此时系统会直接返回 401（Unauthorized）或 403（Forbidden）状态码，告知用户无权访问该内容，并提示用户需完成登录并获取有效 Token 后，才能重新发起请求进行认证鉴权。用户通过认证鉴权之后，Ocelot 网关便会依据请求的路由信息，从而确定目标微服务的名称，然后和 Consul 服务交互，获取到相应微服务的服务信息，再通过负载均衡策略挑选出一个服务实例并将本次请求发送给它。当服务实例接收到请求时，系统会再次进行角色权限的验证，通过权限验证后请求会被发送到相应的控制器，触发相应的请求处理方法进行逻辑运算。在处理过程中可能会涉及到与 MySQL 数据库进行数据读取或写

入操作，也可能需要利用 Redis 缓存服务进行数据的快速存取。若业务逻辑需要与其他微服务进行通信，系统将遵循 gRPC 微服务通信机制进行交互。最终，服务实例会将处理结果返回给 Ocelot 网关，并由其转发至原始请求的客户端，从而完成整个请求处理流程。

4.2 系统功能模块设计

4.2.1 课程管理微服务功能设计

课程管理微服务是本文在线教学平台的核心部分，由图 4-3 功能模块图所知，后台管理员在课程管理微服务中拥有课程管理和课程分类功能，而前台用户拥有查看课程详情、查看课程首页等功能。课程管理微服务功能模块图如 4-3 所示。

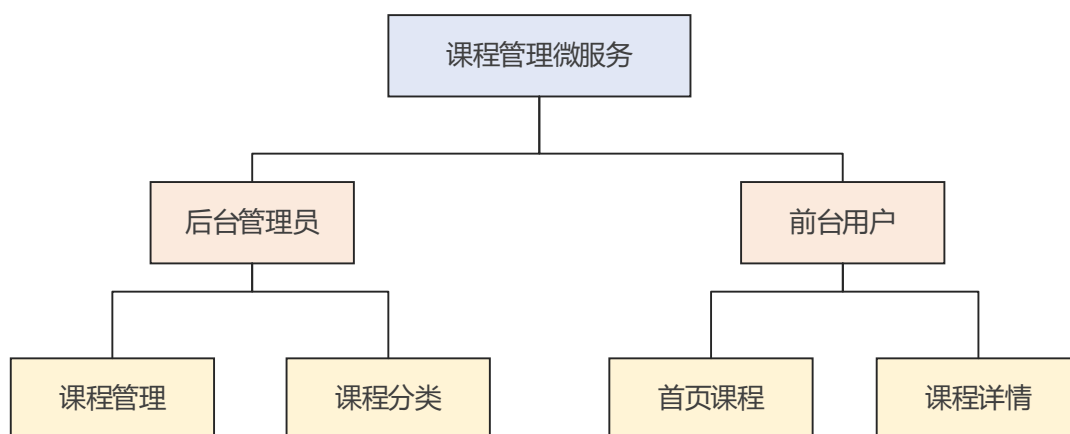


图4-3 课程管理微服务功能模块图

(1)后台管理员模块

其主要的核心任务是录入各种系统所需的信息，主要包括课程信息和课程分类等，这些信息经过管理员的审核和系统处理后，将作为前端系统的数据基础，支撑用户的操作和查询。管理员通过点击“添加课程”功能，填写课程的基本详情，接着设定课程的章节和内容，最后确认无误后保存，系统会将这些信息存储到数据库中，添加课程操作完成，添加新课程流程如图 4-4 所示。

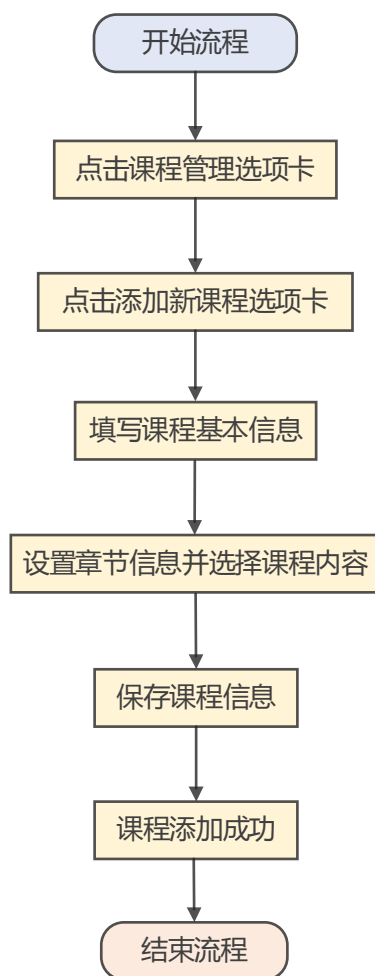


图4-4 后台管理员添加新课程流程图

(2)前台用户模块

前台用户的主要任务则是浏览课程资料，可以在系统首页查看各类课程，包括课程分类、课程概述和详细内容等。这些内容都是由后台管理员上传并公开的。用户进入前端界面后，首页就能看到一些学习课程。如若这些课程不符合用户需求，可以通过课程分类标签来查找，选择感兴趣的类别，进一步点击想学的课程，进入课程详情页，这里会有课程简介和详细的章节安排等信息。如果该课程是免费的，则可以直接选择加入课程，然后就可以选择课程中某一章节的视频内容开始学习，该操作流程如图 4-5 所示。

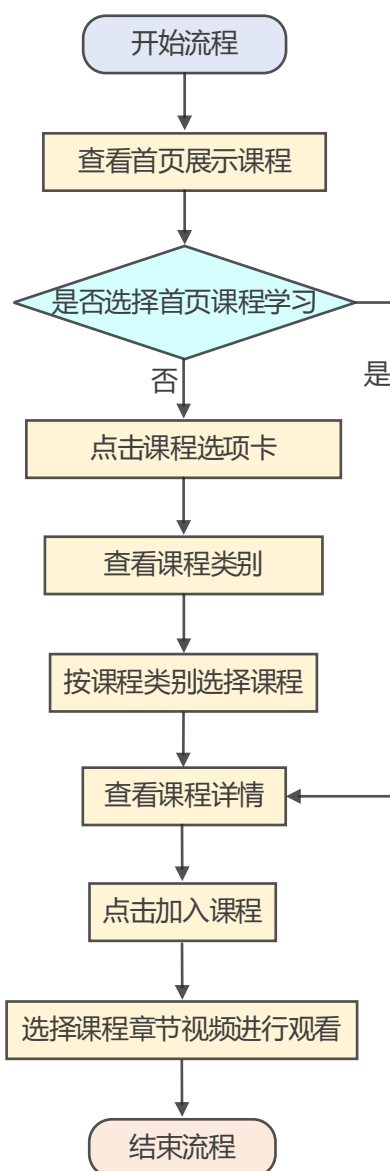


图4-5 前台用户课程展示流程图

4.2.2 视频管理微服务功能设计

视频管理微服务的核心功能在于提供全面的视频管理。它借助阿里云视频点播的 API 接口，将课程视频资源上传至云端存储。鉴于视频作为大量非结构化数据，其存储需求量很大。利用阿里云的视频点播 API，视频能够被上传至点播平台，使得管理员能根据需求执行基础操作，如存储优化、格式转换和安全加密。随后，借助播放凭证，阿里云播放器能被激活，从而提升用户的视频观看体验。视频管理微服务功能模块图如图 4-6 所示。

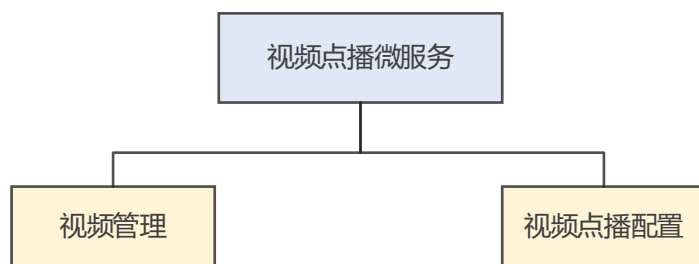


图4-6 视频管理微服务功能模块图

视频点播配置流程图如图 4-7 所示。首先，需启用阿里云的视频点播服务，配置存储空间，声明好视频转码方式和加密策略。在系统内引入视频点播的依赖，通过 API 获取播放授权，同时整合阿里云播放器的脚本。此播放器支持自动用播放凭证换取播放地址，简化了接入步骤，增强了安全性。课程管理微服务通过调用这些视频点播功能来实现视频播放。

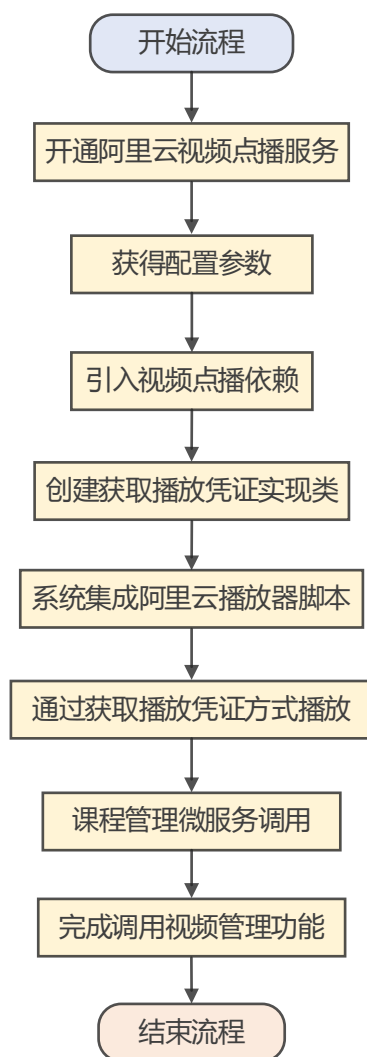


图4-7 视频点播配置流程图

4.2.3 用户管理微服务功能设计

用户管理微服务专注于用户管理功能，用户可以通过手机号进行注册和登录。由于系统由多个具有不同端口号的微服务组成，当请求的 URL 端口与当前微服务不符时，就会出现跨域问题。为解决这一问题，可以采用单点登录 SSO(Single Sign On)的方式进行登录，当用户尝试使用特定业务功能时，若尚未登录，系统会引导他们进入统一身份认证系统，该系统会从数据库检索并验证用户信息，一旦成功登录，用户的登录状态会被存储到 Redis 缓存，并设定一个有效期。此后，无论用户浏览哪个业务板块，系统都会检查统一身份认证系统中的浏览器缓存以确认其登录状态，实现全系统的单点登录验证，避免重复登录。该模块通过调用第三方短信服务，实现短信注册功能。用户管理微服务功能模块图如图 4-8 所示。

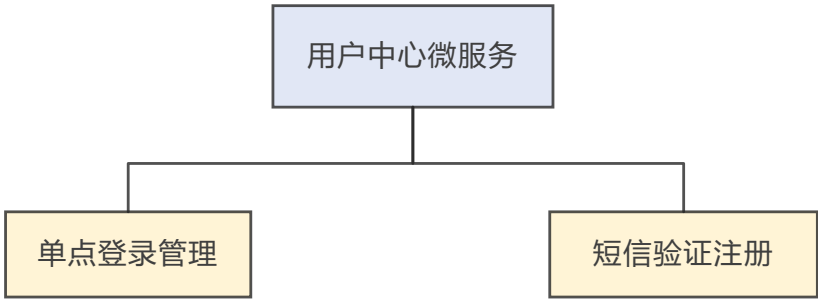


图4-8 用户管理微服务功能模块图

单点登录系统流程图如图 4-9 所示。当用户执行课程观看的功能时，必须首先进行登录和注册步骤。若用户已登录，他们可直接继续使用该功能去完成业务流程。若用户未登录，系统会自动引导他们至登录页面。已注册用户只需输入账户和密码，并发送登录请求到服务器，接着服务器会对这些信息进行验证。一旦验证成功，服务器会反馈登录成功的响应，同时在 Redis 缓存中存储用户的登录状态，并设定一个有效期，在有效期内用户可无障碍地使用系统内的所有功能，过期后用户需重新登录。

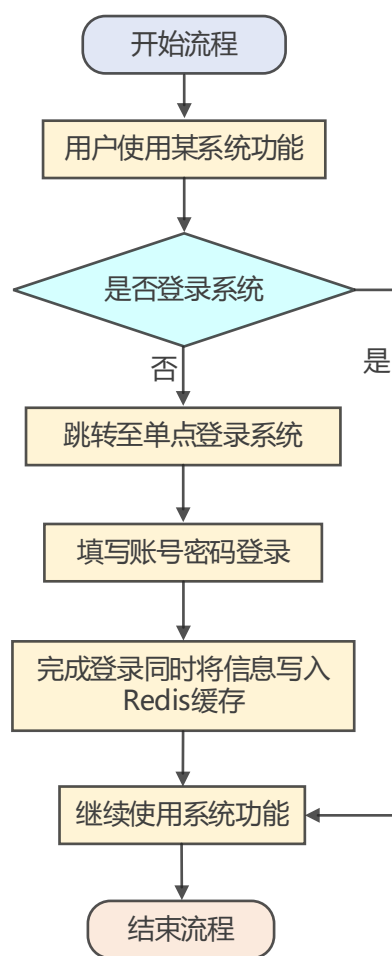


图4-9 登录注册系统流程图

4.2.4 订单管理微服务功能设计

订单管理微服务的主要任务是处理用户的订单操作，包括对第三方支付平台的调用。在本系统中，课程分为免费和付费两类。免费课程允许用户直接访问观看，但若需要购买付费课程，用户需点击购买按钮生成课程订单，此过程会触发对第三方支付接口的调用。如果是使用微信支付接口，系统生成一个对应课程价格的支付码，然后用户通过手机扫描二维码，确认交易信息后支付订单金额，即可完成订单购买操作，从而可以观看到付费视频内容。订单管理微服务功能模块图如图 4-10 所示。

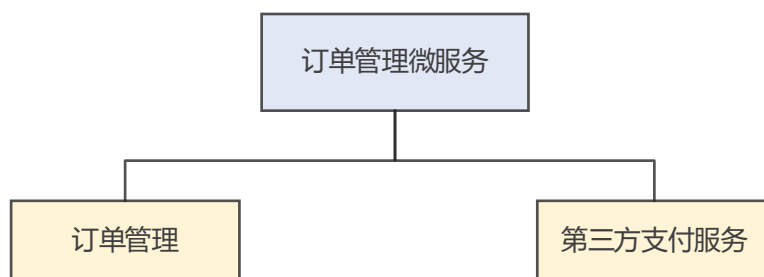


图4-10 订单管理微服务功能模块图

订单管理功能流程图如图 4-11 所示。首先用户访问课程首页，点击课程图标后会判断用户是否登录，未登录则跳转至登录页面，若已登录则会进入课程详情页面。其中课程如果是免费的则可以直接加入我的课程；付费课程则需要用户在详情页中点击“立即购买”，之后会生成课程订单信息，确认订单后点击“立即支付”会跳转至支付界面，选择微信支付后系统会调用支付接口，生成微信支付二维码，扫描二维码后即可完成支付，之后用户就能在“我的课程”中学习该课程。

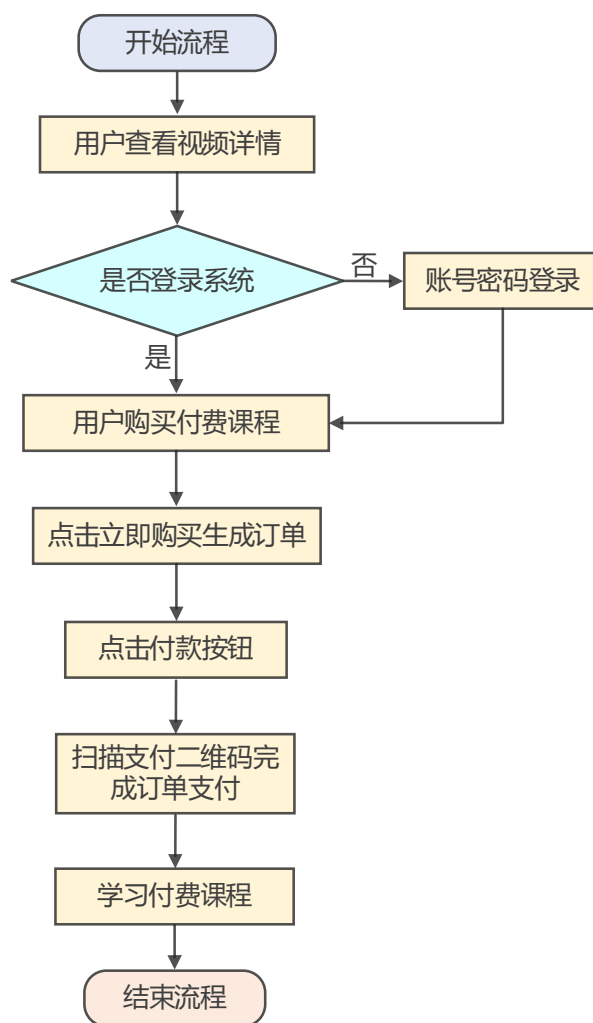


图4-11 订单管理功能流程图

4.3 数据库设计

数据库作为整个在线学习平台的核心部分，它不仅是数据的存储和管理中心，更是确保系统高效、稳定、安全运行的基石。合理的数据库设计能够为系统带来性能上的提升和数据完整性及安全性的提高。本系统使用 MySQL 数据库管理系统对数据进行存储和管理，并结合 Redis 内存数据库管理系统^[53]，为整个在线学习系统提供高效的数据查询和处理能力。

4.3.1 数据库架构设计

本系统数据库架构设计采用了主从架构(Master-Slave Architecture)模式,是一种常见的数据库架构,它涉及到两个或多个数据库服务器,其中一个被指定为主数据库(Master),而其他则被配置为从数据库(Slave)。在这种架构中,主数据库负责处理所有的写操作,而从数据库则复制主数据库上的数据,并通常用于处理读操作^[53]。下图 4-12 为主从数据库复制原理图。

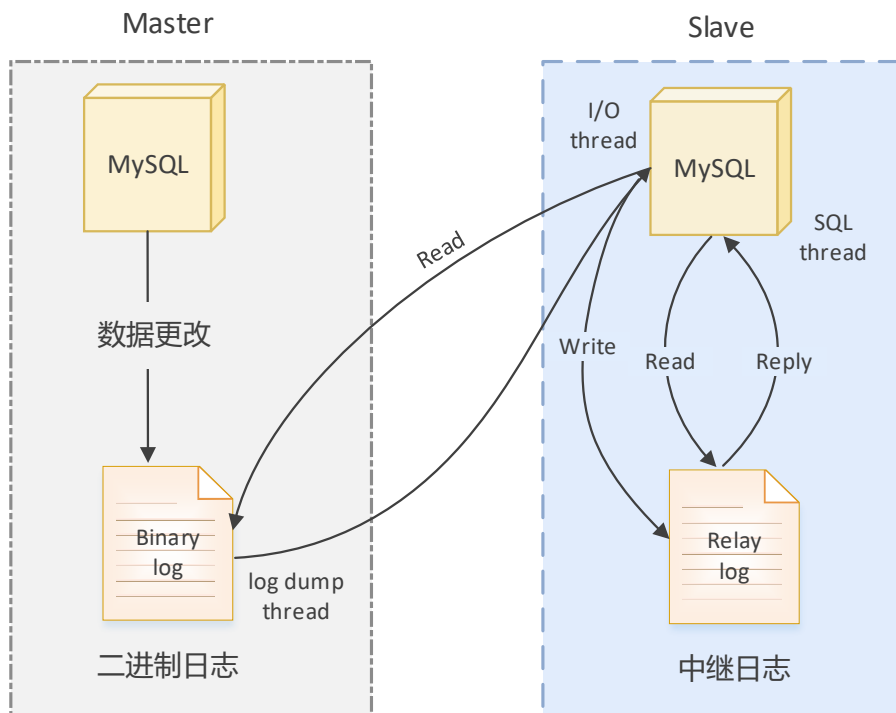


图4-12 主从数据库复制原理图

在整个主从复制过程中,从数据库(Slave)的 IO 线程负责向主数据库(Master)发送请求以获取二进制日志(Binary log)信息。Master 端的 binlog dump 线程响应这些请求,提供所需的二进制日志信息给 Slave 端的 IO 线程。随后,Slave 端的 IO 线程将接收到的内容写入本地的 Relay log 中,并将 Master 端的二进制日志文件名和位置记录到 master.info 文件中。当 Relay log 中的内容有更新,Slave 端的 SQL 线程会检测到这些变化,解析并执行其中的操作,从而确保了主从数据库中数据的一致性。

4.3.2 数据库表结构设计

经过上述对系统的需求分析,可以清晰地识别出系统中主要的实体包括课程、视频、订单和用户。根据这些实体各自承载的信息和它们之间存在的联系,设计出合理的数据库表结构。本文实现的在线学习平台的数据库实体关系(E-R)图如下图 4-13 所示。

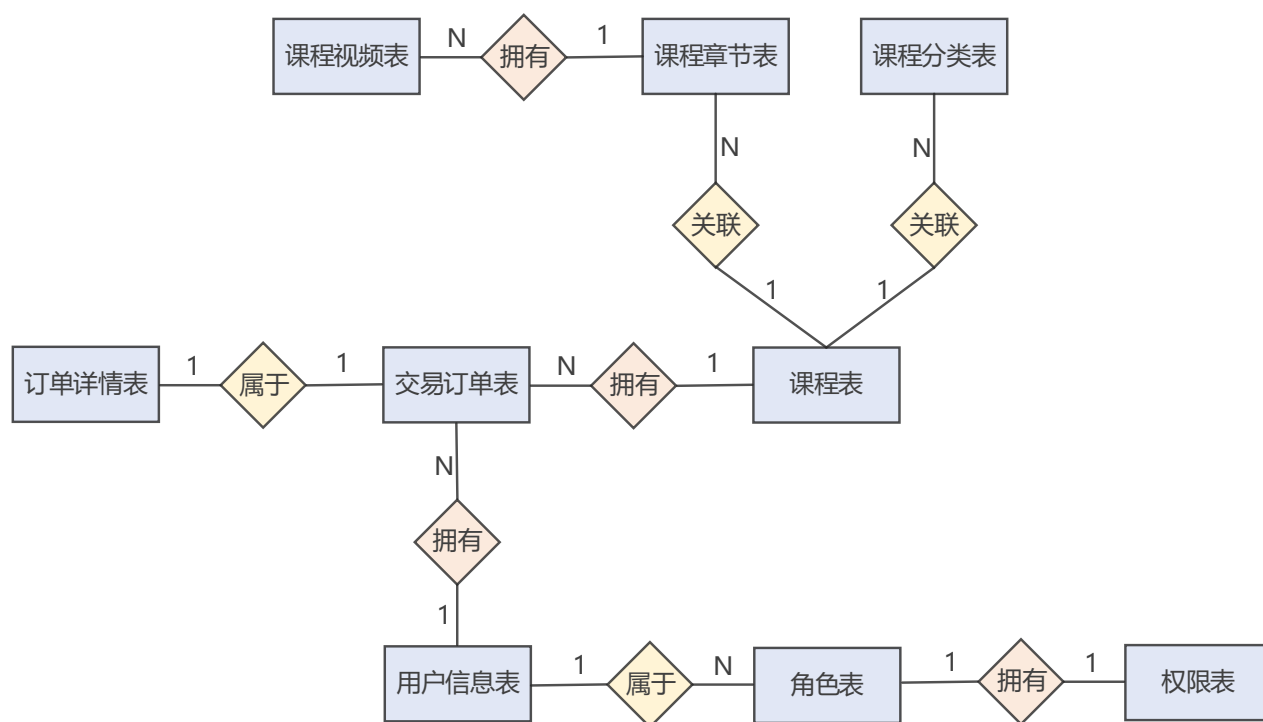


图4-13 数据库E-R图

根据数据库 E-R 图可知，系统中主要业务功能的数据库表有 9 张，其中与课程相关的表有课程表、课程分类表和课程章节表，与视频实体所对应的有课程视频表，与订单实体相关的有交易订单表和订单详情表，与用户实体相关的包含用户成员表、角色表和权限表。这些表是整个系统数据库表中最核心的部分，其他非核心数据库表不在本文体现。

(1)课程表(t_courses)

课程表中存储的是具体课程的相关信息，包括课程标题、课程描述、课程视频地址以及课程的价格等信息。该表将 `course_id` 字段作为课程表的主键，是课程数据的唯一标识，可以用该字段与课程章节表进行关联；其中 `class_id` 字段保存的是该课程所属分类的 id，其作为外键关联了课程分类表；`public_state` 字段表示课程发布状态，0 表示未发布，1 表示已发布。课程表的其它字段信息如表 4-1 所示。

表4-1 课程表(t_courses)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	course_id	课程编号	int	主键，非空
2	class_id	分类编号	int	外键，非空
3	course_title	课程标题	nvarchar(200)	非空
4	description	课程描述	text	可空
5	course_imgurl	图片地址	varchar(200)	可空

续表4-1 课程表(t_courses)

序号	字段名称	字段说明	字段类型及长度	字段约束
6	course_linkurl	课程视频地址	varchar(200)	可空
7	price	价格	decimal(18,3)	非空
8	order_no	排序	int	非空
9	remark	备注	text	可空
10	public_state	发布状态, 默认为 0	tinyint(1)	非空
11	public_time	发布时间	datetime	可空
12	is_hide	是否隐藏, 默认为 0	tinyint(1)	非空
13	add_time	添加时间	datetime	可空
14	update_time	修改时间	datetime	可空

(2)课程分类表(t_categories)

课程分类表存储的是课程类别信息, 其中 class_name 字段表示分类名称; class_id 作为分类表的主键, 为自增字段, 是类别信息的唯一标识; parent_class_id 是一个表内关联字段, 保存了上一级分类的编号, 默认值为 0, 表示该类别就为一级分类。课程分类表的其它字段信息如表 4-2 所示。

表4-2 课程分类表(t_categories)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	class_id	分类编号	int	主键, 非空
2	class_name	分类名称	nvarchar(100)	非空
3	parent_class_id	父分类编号, 默认为 0	tinyint(1)	可空
4	order_no	排序	int	非空
5	remark	备注	text	可空
6	is_hide	是否隐藏, 默认为 0	tinyint(1)	非空
7	add_time	添加时间	datetime	可空
8	update_time	修改时间	datetime	可空

(3)课程章节表(t_chapters)

课程章节表存储的是某课程的章节信息, 其中 chapter_name 字段表示章节名称; chapter_id 作为章节表的主键, 为自增字段, 是类别信息的唯一标识; course_id 作为章节表的外键, 与课程表进行关联, 明确了该章节的所属课程。课程章节表的其它字段信息如表 4-3 所示。

表4-3 课程章节表(t_chapters)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	chapter_id	章节编号	int	主键, 非空
2	chapter_name	章节名称	nvarchar(50)	非空
3	course_id	所属课程	int	外键, 非空
4	order_no	排序	int	非空
5	is_hide	是否隐藏, 默认为 0	tinyint(1)	非空
6	add_time	添加时间	datetime	可空
7	update_time	修改时间	datetime	可空

(4)课程视频表(t_videos)

课程视频表存储的是课程视频信息, video_id 作为视频表的主键, 为自增字段, 是课程视频数据的唯一标识; course_id 和 chapter_id 字段作为视频表的外键, 与课程表和章节表进行关联, 来明确该条视频数据归属于哪门课程中的哪一章节。由于本系统采用了阿里云存储服务来保存课程视频, 所以 video_original_url 字段记录的是视频在云存储中的地址信息, 用户可以通过阿里云的点播服务从该地址中获取视频内容。课程视频表的其它字段信息如表 4-4 所示。

表4-4 课程视频表(t_videos)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	video_id	视频编号	int	主键, 非空
2	course_id	课程编号	int	外键, 非空
3	chapter_id	章节编号	int	外键, 非空
4	title	视频标题	varchar(50)	非空
5	video_original_url	视频源地址	varchar(100)	非空
6	order_no	排序	int	非空
7	play_count	播放次数	bigint	非空
8	is_audition	是否试听	tinyint(1)	非空
9	audition_minute	试听分钟数	int	非空
10	duration	视频时长	float	非空
11	is_hide	是否隐藏, 默认为 0	tinyint(1)	非空
12	add_time	创建时间	datetime	可空
13	update_time	修改时间	datetime	可空

(5)交易订单表(t_orders)

交易订单表存储的是用户交易订单信息，order_id 字段作为主键，表示订单编号，user_id 字段存储该订单的用户编号；其中 order_state 字段表示订单的状态，0 表示未付款，1 表示已完成，2 表示已取消。交易订单表的其它字段信息如表 4-5 所示。

表4-5 交易订单表(t_orders)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	order_id	订单编号	varchar(20)	主键，非空
2	user_id	订单用户	int	外键，非空
3	phone	联系方式	varchar(11)	非空
4	user_name	联系人	nvarchar(50)	非空
5	qq	QQ 号码	nvarchar(50)	非空
6	order_state	订单状态，默认为 0	tinyint(1)	非空
7	total_cast	支付金额	decimal(18,0)	可空
8	transaction_id	交易流水	varchar(20)	可空
9	pay_time	订单支付时间	datetime	可空
10	description	描述	nvarchar(500)	可空
11	is_hide	是否隐藏，默认为 0	tinyint(1)	非空
12	add_time	添加时间	datetime	可空
13	update_time	修改时间	datetime	可空

(6)订单详情表(t_orderdetails)

订单详情表存储订单中的详细信息，包括购买的课程编号，课程价格和数量。在该表中使用 order_id 和 course_id 作为外键来关联交易订单表和课程表，以此可以查询到订单的所属用户和详细的课程信息；支付方式 pay_type 字段有两种，1 表示微信支付，2 表示支付宝支付。订单详情表的其它字段信息如表 4-6 所示。

表4-6 订单详情表(t_orderdetails)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	order_detail_id	订单详情编号	int	主键，非空
2	order_id	订单编号	int	外键，非空
3	course_id	课程编号	int	外键，非空
4	course_title	课程标题	nvarchar(200)	非空
5	product_price	产品价格	decimal(18,0)	非空
6	product_count	产品数量	int	非空
7	total_fee	订单金额	decimal(18,0)	非空

续表4-6 订单详情表(t_orderdetails)

序号	字段名称	字段说明	字段类型及长度	字段约束
8	pay_type	支付方式	tinyint(1)	非空
9	pay_time	支付时间	datetime	可空
10	is_hide	是否隐藏, 默认为 0	tinyint(1)	非空
11	add_time	添加时间	datetime	可空
12	update_time	修改时间	datetime	可空

(7)用户信息表(t_users)

用户信息表存储的是用户注册时所填写的用户信息, 可用于登录时进行账号密码校验, password 用户密码使用 MD5 算法进行加密存储, 其中 user_type 字段表示用户类型, 0 代表普通用户, 1 代表管理员。用户信息表的其它字段信息如表 4-7 所示。

表4-7 用户信息表(t_users)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	user_id	用户编号, 主键	int	主键, 非空
2	user_name	用户名称	varchar(50)	非空
3	password	用户密码, MD5 加密	varchar(50)	非空
4	user_type	用户类型	char(1)	非空
5	user_pic_url	用户头像地址	varchar(200)	非空
6	birth	用户生日	DateTime	非空
7	gender	用户性别	char	可空
8	email	邮箱	varchar(50)	可空
9	phone	电话	varchar(50)	可空
11	is_hide	是否隐藏, 默认为 0	tinyint(1)	非空
10	add_time	添加时间	DateTime	可空
11	update_time	销毁时间	DateTime	可空

(8)角色表(t_roles)

角色表是用于存储系统中定义的角色信息, 其中 role_id 是唯一标识角色的主键; role_code 用于存储角色的唯一编码或标识, 用于程序中的特定处理或查询; status 表示角色状态, 可以是“激活”或“冻结”状态。角色表的其它字段信息如表 4-8 所示。

表4-8 角色表(t_roles)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	role_id	角色编号, 主键	int	主键, 非空
2	role_name	角色名称	varchar(50)	非空
3	role_code	角色代码	varchar(50)	可空
4	description	角色描述信息	varchar(200)	可空
5	status	角色状态: 1 激活、2 冻结	int	非空
6	add_time	添加时间	datetime	可空
7	update_time	销毁时间	datetime	可空

(9)用户权限表(t_user_permissions)

用户权限表通常用于存储和管理系统中用户的权限信息, 其中 permission_id 作为主键, 是权限表中唯一标识权限的字段, user_id 作为外键与用户表中的主键相对应。status 字段用于标识权限关系是否有效, 可以是“启用”或“禁用”状态。用户权限表的其它字段信息如表 4-9 所示。

表4-9 用户权限表(t_user_permissions)

序号	字段名称	字段说明	字段类型及长度	字段约束
1	permission_id	权限编号, 主键	int	主键, 非空
2	user_id	用户编号	int	外键, 非空
3	role_id	角色编号	int	外键, 非空
4	description	权限描述信息	varchar(200)	可空
5	status	权限状态: 1 启用、2 禁用	int	非空
6	add_time	添加时间	DateTime	可空
7	update_time	销毁时间	DateTime	可空

4.4 本章小结

本章节分析了在线教学平台的初步设计和规划。首先通过构建在线学习系统的整体架构, 清晰地呈现了整个系统的工作流程。然后分析出各个微服务的功能模块, 并介绍了主要功能的操作流程。最后, 通过主从架构模式对系统数据库进行设计, 并根据实体间的关系生成 E-R 图, 分析出了 9 个主要的数据库表, 然后对它们进行表结构设计, 其中每个字段都根据数据存储需求配置了合理的数据类型和存储长度。

第五章 在线学习平台实现与测试

继需求分析、数据库设计和系统概述之后,本章将对在线学习平台进行详细设计、实现与测试。首先介绍系统的开发环境,然后针对拆分出的四个主要微服务进行详细设计与实现,并展示出每个服务所实现的功能界面。最后对在线学习平台进行功能性和非功能性测试。

5.1 微服务基础组件设计实现

5.1.1 系统开发环境介绍

在软件开发工具的选择上,使用 Windows 10 专业版 22H2 作为操作系统。在系统后端开发方面,使用 Visual Studio Community 2022 作为开发平台,而在系统前端开发方面,则选用了 Visual Studio Code,主要的开发语言是 C#。

在数据库的选择上,采用了 MySQL 作为关系型数据库,并使用 Navicat 作为管理工具;同时还选用了 Redis 来支持非关系型数据的存储。

在系统架构方面,本系统基于 ASP.NET Core 构建基础框架。在微服务领域,使用了 Consul 作为注册中心,Ocelot 作为基础网关。

本系统开发环境使用的组件与工具如下表 5-1 所示:

表5-1 开发工具与组件配置表		
	工具组件名称	软件/框架名称及版本
开发工具	操作系统	Windows10 专业版 22H2
	虚拟机	CentOS 7
	系统前端开发平台	Visual Studio Code 1.85.0
	系统后端开发平台	Visual Studio Community 2022
	开发语言	C# 11
数据库管理	关系型数据库	MySQL 8.0.33
	非关系型数据库	Redis 6.2.1
	数据库可视化工具	Navicat 15.0.26
系统架构/组件	系统前端基础框架	Vue 3
	系统后端基础框架	ASP.NET Core
	微服务基础网关	Ocelot
	微服务注册中心	Consul

5.1.2 服务注册中心实现

本文开发的在线学习平台采用 Consul 作为服务注册中心，为微服务系统提供服务注册与发现功能以及服务健康检查能力。在系统中，各个微服务实例在启动时会自动向 Consul 注册中心注册自己的服务名称、IP 地址、端口号等信息，其他微服务在需要调用某个服务时，就可以通过 Consul 查询该服务的注册信息，从而实现服务的自动发现，使得微服务架构中的服务能够动态地相互发现和通信。此外，Consul 还具备服务健康检查机制，通过定期执行开发人员定义的健康检查规则来验证服务实例的运行状态，当某个服务实例出现故障时，Consul 会将其从服务列表中移除，并触发相应的故障转移机制，将请求转发到其他可用的服务实例上，这种容错处理机制确保了系统的稳定性和可用性。Consul 还提供可视化界面，可以查看服务的健康状态和运行情况，本系统的服务管理界面如图 5-1 所示，其中包含所有已注册服务的状态信息。

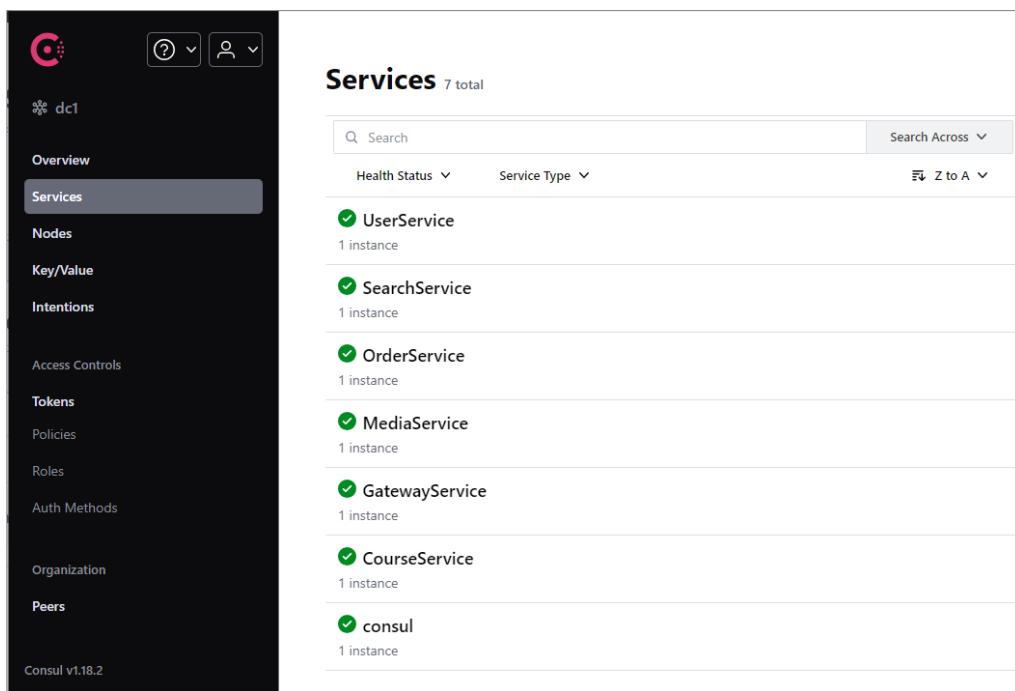


图5-1 服务注册中心实现

5.1.3 服务网关设计实现

服务网关是客户端和微服务集群之间的唯一入口点，其作用是给微服务系统提供一个统一的对外访问地址^[54]，客户端请求首先发送到服务网关地址，再根据网关的路由配置规则将请求转发至相对应的微服务实例进行处理。本文设计的微服务平台采用 Ocelot 作为服务网关，下面将介绍 Ocelot 网关在系统中的具体实现。

首先建立一个服务网关项目 GatewayService，在项目中添加 Ocelot 工具包；然后创建服务网关配置文件 ocelot.json，配置文件内容如图 5-2 所示。

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/{url}",
      "DownstreamScheme": "http",
      "UseServiceDiscovery": true,
      "UpstreamPathTemplate": "/course-service/{url}",
      "ServiceName": "CourseService",
      "LoadBalancerOptions": {
        "Type": "RoundRobin"
      }
    },
    {
      "DownstreamPathTemplate": "/api/{url}",
      "DownstreamScheme": "http",
      "UseServiceDiscovery": true,
      "UpstreamPathTemplate": "/user-service/{url}",
      "ServiceName": "UserService",
      "LoadBalancerOptions": {
        "Type": "RoundRobin"
      }
    }
  ],
  "GlobalConfiguration": {
    "ServiceDiscoveryProvider": {
      "Host": "localhost",
      "Port": 8500,
      "PollingInterval": 5000,
      "Type": "Consul",
      "ConfigurationKey": "API_Gateway"
    }
  }
}
```

图5-2 Ocelot网关配置信息

图中只包含配置文件中的部分信息，以课程微服务的网关路由配置为例，其中 UpstreamPathTemplate 称为上游路径模板，指的是请求发送至网关所对应的地址，DownstreamPathTemplate 为下游路径模板，指的是请求转发到课程服务所对应的地址，将 UseServiceDiscovery 启用服务发现设置为 true，实现负载均衡 LoadBalancerOptions 所使用的是轮询策略(RoundRobin)。当用户点击课程链接查看课程详情时，该请求的地址为网关地址，随后 Ocelot 网关会先去 Consul 注册中心查询课程服务注册信息，再根据轮询策略从课程服务集群中找到合适的服务实例，并将请求转发给该实例进行逻辑处理，最后返回的响应数据也将通过 Ocelot 网关统一发送至客户端。

5.2 课程管理微服务实现

5.2.1 课程管理功能时序图

当后台管理员登录系统，访问课程管理界面。在这里，如果管理员发起创建新课程请求，控制层会接收到这个请求，然后调用 `CourseService`、`ChapterService`、`VideoService` 等服务层的方法实现类来处理。这些方法实现类进一步利用仓储层的数据库增删改查操作与数据库交互，最终返回操作的结果。服务层负责创建新的课程，并将相关信息回传至控制层，最后在后台管理员界面上会展示出课程创建成功的提示信息。课程管理功能后台管理员时序图如图 5-3 所示。

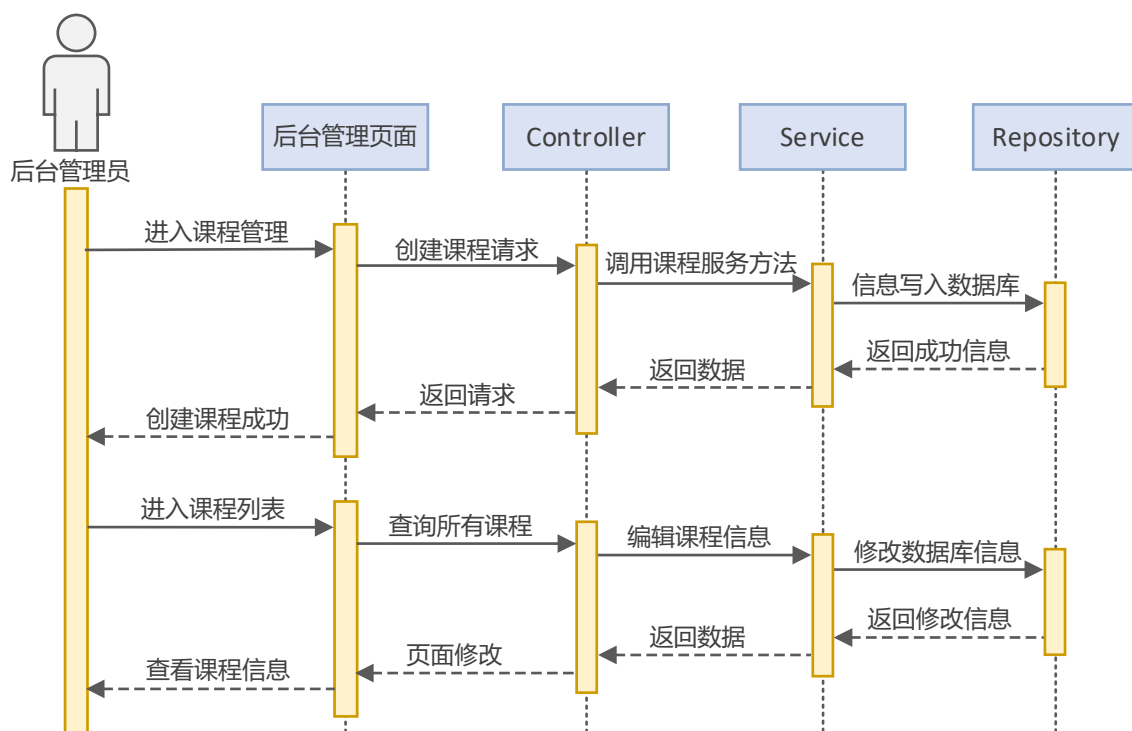


图5-3 课程管理功能后台管理员时序图

对于前台用户，他们在课程页面执行查看课程详情、视频播放等操作后，这些请求首先被控制层接收到，然后会调用服务层中的课程服务、章节服务和视频服务等具体实现来处理这些请求，根据用户查询的课程信息返回相应结果。当服务层执行学习课程、获取课程列表等功能之后，会将数据传递回控制层，使得前端用户界面能够加载课程简介、播放视频等内容。课程管理功能前台用户时序图如图 5-4 所示。

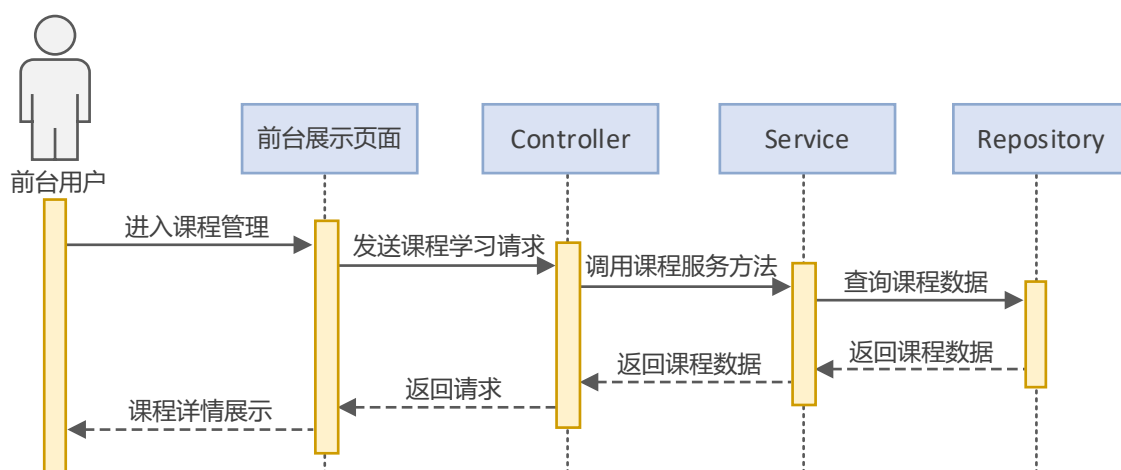


图5-4 课程管理功能前台用户时序图

5.2.2 课程管理类图设计与接口说明

在课程管理微服务中主要涉及三个实体类，分别为 Course 类、Chapter 类和 Video 类，这三个类分别对应数据库表中的课程表、章节表和视频表，该微服务的 UML 类图展示如图 5-5 所示，由于篇幅有限，图中仅展示这些类或接口的部分方法和属性。

用户在客户端发送课程服务的请求到后端服务器时，会先由控制层（Controller）进行处理，其中分别包含 CourseController、ChapterController 和 VideoController 三个控制器类。CourseController 为课程信息管理控制器，依赖与 ICourseService 接口，为前端提供了添加课程 SaveCourseInfo、发布课程大纲 PublishCourseById、删除课程 RemoveCourseById 以及获取课程详情信息 GetCourseById 等功能的 API 接口；ChapterController 为课程章节管理控制器，依赖于 IChapterService 接口，提供了添加课程章节信息 SaveChapter、获取课程章节列表 GetChapterList、删除课程某章节 RemoveChapterById 等功能的接口；VideoController 为课程视频管理控制器，该类依赖于 IVideoService 接口，针对课程提供了添加视频课时信息 SaveVideo、获取视频资料 GetVideoById、更新视频课时 UpdateVideoById 和删除视频课时 RemoveVideoById 等功能的接口。

控制层中的 API 接口需要通过调用服务层（Service）中的业务方法来实现具体的业务逻辑，在课程管理微服务中服务层主要包括 CourseService、ChapterService 和 VideoService 三个类。CourseService 是课程信息管理服务接口 ICourseService 的实现类，根据实际的业务逻辑对接口中的添加课程信息 SaveCourseInfo()、获取课程信息 GetCourse()、获取课程列表 GetCourseList()等方法进行具体实现；ChapterService 为课程章节管理服务接口 IChapterService 的实现类，根据业务逻辑对接口中的添加章节信息 SaveChapterInfo()、遍历获取课程章节列表 GetChapterList()、删除课程某章节信息 RemoveChapterById()等方法进行具体的逻辑实现；VideoService 作为课程视频管理服

务接口 `IVideoService` 的实现类，主要实现了获取视频资源列表 `GetVideoSourceList()`、通过视频编号删除视频 `RemoveVideoById()` 等方法的业务逻辑。

仓储层 (`Repository`) 主要作用是和服务层提供对数据库表进行操作的功能，其中 `CourseSerDbContext` 是课程服务的数据库上下文类，此类对服务中所用到的实体类进行声明；`CourseSerRepository` 为课程管理微服务仓储接口 `ICourseSerRepository` 的实现类，为服务层提供了对课程、章节和视频等数据操作的方法。

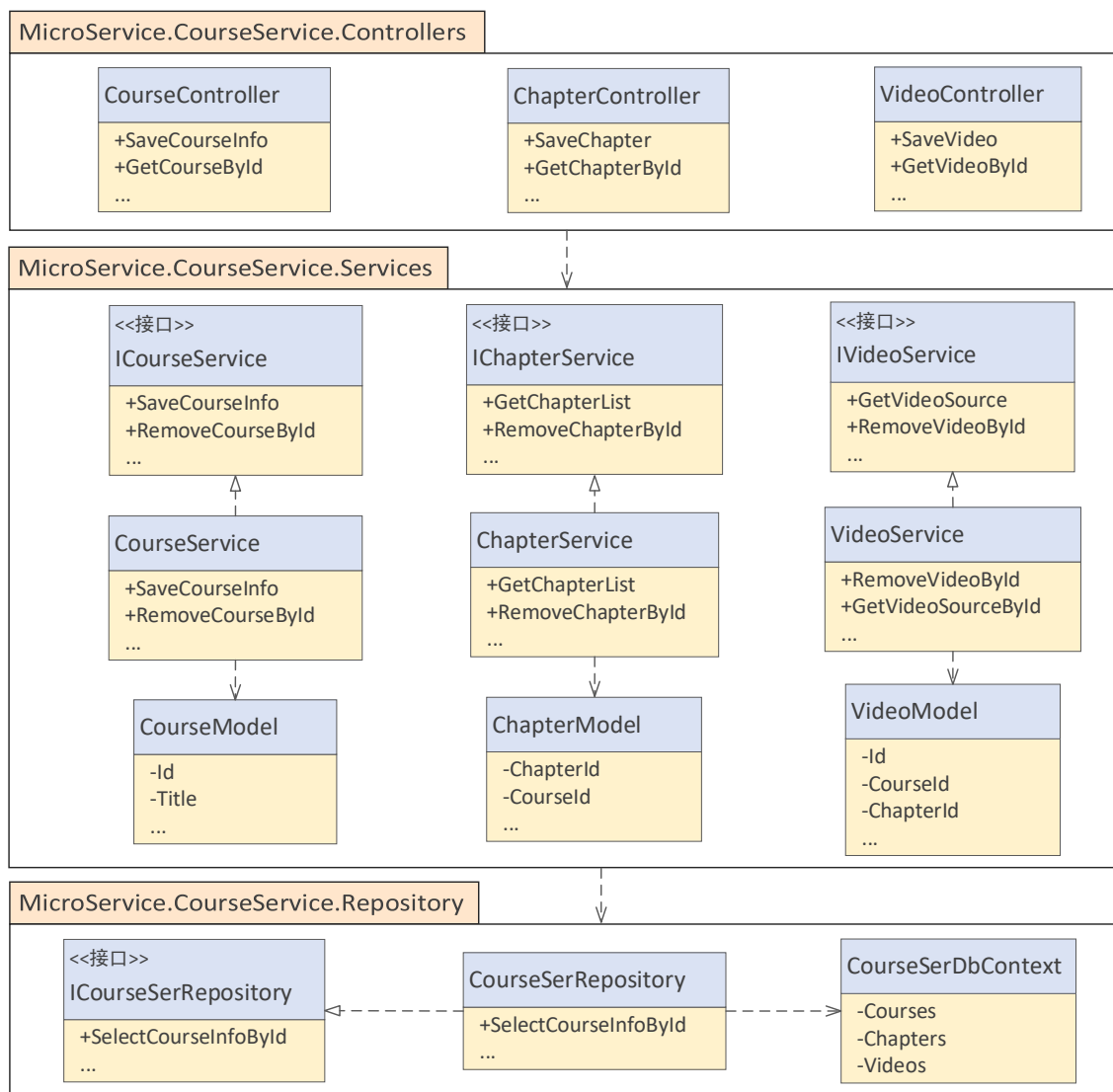


图5-5 课程管理微服务UML类图

课程创建和课程信息修改等课程管理功能的请求首先都会在 `Controller` 控制层中进行处理，需要在控制层声明相关的请求处理接口。课程管理微服务中的 `CourseController` 控制器 API 接口描述如表 5-2 所示。

`SaveCourseInfo` 接口用于处理后台管理员添加课程信息的请求，接收一个 `courseInfoForm` 实体类作为参数，并调用服务层中相应的方法来实现添加课程功能，并在课程表中保存该条数据信息。

GetCourseById 接口可以通过传递特定的课程 Id 返回课程的具体信息给前端, 请求方式为 GET。

RemoveCourseById 接口功能是通过接收一个课程 Id 来删除指定课程, 借助 DELETE 请求方式, 并调用服务层的 RemoveVideoByCourseId 方法删除课程视频。

PublishCourse 接口用于处理发布课程的请求, 接收一个待发布的课程 Id 作为参数, 采用 PUT 请求方式, 调用服务层的方法对课程表中的数据进行修改, 将课程发布状态改为 1 (已发布)。

GetPublishInfo 接口用于处理获取课程的发布信息的请求, 采用 GET 请求来获取这些数据, 其主要功能是查询课程发布的详细信息。

PageList 接口用于处理获取课程列表的请求, 通过 GET 请求来获取一组课程信息, 供前端页面展示, 将请求参数定义为 List 类型来表示要获取的课程信息。

GetCourseInfo 接口用于处理查看课程信息的请求, 采用 GET 请求, 当用户点击课程图标, 随后转到课程详情页面。

表5-2 课程管理服务CourseController接口表

序号	接口名称	请求方式	方法参数	接口功能说明
1	Course/SaveCourseInfo	POST	courseInfoForm	添加课程信息
2	Course/GetCourseById	GET	courseId	Id查询课程信息
3	Course/RemoveCourseById	DELETE	courseId	删除课程
4	Course/PublishCourse	PUT	courseId	发布课程
5	Course/GetPublishInfo	GET	courseId	获取课程发布信息
6	Course/PageList	GET	courseList	获取课程列表
7	Course/GetCourseInfo	GET	courseId	查看课程信息

课程管理服务中的 ChapterController 控制器的 API 接口主要用于处理后台用户发出的章节管理请求, 其接口的相关信息如表 5-3 所示。

SaveChapter 接口的作用是处理添加并保存课程章节信息的请求, 请求方式为 POST, 接收一个章节实体类作为方法参数, 调用服务层的新增章节信息实现方法, 并在数据库表中创建新的章节记录。

GetChapter 接口用于获取特定课程章节信息, 请求方式为 GET, 接收章节 Id 作为参数, 调用服务层和仓储层中的方法从数据库中获取数据, 并将结果返回至前端页面进行渲染展示。

UpdateChapter 接口用于更新课程章节信息, 请求方式为 PUT, 将章节类对象作为参数传递, 调用服务层中的方法实现对数据的修改。

RemoveById 接口用于处理删除章节的请求, 请求方式为 DELETE, 根据章节 Id

对课程某章节进行删除操作，同时对数据库表中对应数据进行修改。

GetChapterList 接口用于获取课程章节列表信息，请求方式为 GET，调用服务层中的对应方法，采用循环遍历的方式得到课程章节列表，返回一个以章节实体类为泛型的 `List<Chapter>` 集合，通过前端页面进行渲染展示。

表5-3 课程管理服务ChapterController接口表

序号	接口名称	请求方式	方法参数	接口功能说明
1	Chapter/SaveChapter	POST	class	新增课程章节信息
2	Chapter/GetChapter	GET	id	获取课程章节信息
3	Chapter/UpdateChapter	PUT	class	更新章节信息
4	Chapter/RemoveById	DELETE	id	删除课程章节
5	Chapter/GetChapterList	GET	courseid	查询课程章节列表

课程管理服务中的 VideoController 控制器的 API 接口用于处理后台用户操作课程视频的请求，其 API 接口的相关信息如表 5-4 所示。

SaveVideo 接口用于处理添加视频课时的请求，请求方式为 POST，接收一个视频类对象作为参数，为某课程添加视频内容，通过调用服务层中具体的实现方法完成该功能，并在数据库表中添加新的视频数据。

GetVideo 接口用于处理获取视频信息的请求，请求方式为 GET，通过传递一个视频 Id 作为参数并调用对应方法，从数据库中获取该视频数据，之后将该数据返回至前端进行数据渲染。

UpdateVideo 接口用于更新视频信息，请求方式为 PUT，接收视频 Id 作为参数对课程大纲中的某视频课时信息进行修改，并在数据库表中更新修改内容。

RemoveById 接口用于处理删除视频的请求，请求方式为 DELETE，接收视频 Id 作为参数来删除课程大纲中特定的课时视频。

表5-4 课程管理服务VideoController接口表

序号	接口名称	请求方式	方法参数	接口功能说明
1	Video/SaveVideo	POST	class	新增视频课时信息
2	Video/GetVideo	GET	id	获取视频信息
3	Video/UpdateVideo	PUT	id	更新视频信息
4	Video/RemoveById	DELETE	id	删除视频课时信息

5.2.3 课程管理功能页面展示

当前台用户通过网站地址进入系统时，用户不需要登录账号就可以直接查看网站首页，并展示首页课程如图 5-6 所示。



图5-6 前台首页展示

在课程管理系统中，后台管理员需先发布课程，包括设定课程大纲，然后课程就能被前端用户浏览。发布的课程页面设计如图 5-7 所示，其中管理员需填写课程标题、课程定价、类别以及总课时数等关键信息。

添加课程

基本信息

资料附件

课程标题

晨曦BIM智能翻模案例教学

课程类别

晨曦BIM智能工具

价格

0

是否优惠

☐ 是 ☒ 否

优惠价格

0

封面

注：支持jpg, png, bmp图片文件，建议图片使用大小为450x300

课时数

1

发布时间

2024-02-11 15:26:14

课程描述

Hiragino S

16px

A B I U ABC

段落

晨曦BIM智能翻模是款新型的智能翻模工具。抛开传统的图层提取概念，运用人工智能技术结合施工顺序及制图规范自动识别图纸内容，一体化完成图纸分割与整理、构件识别、生成revit模型等过程。将标准化、有规律的步骤交由软件自动完成，帮助用户简化操作步骤、提高建模效率。同时与晨曦BIM算量对接，完成土建、钢筋快速出量。

保存

图5-7 发布课程页面

后台管理员在后台管理系统中添加并发布课程之后，可以在课程管理页面中查询该课程是否发布成功，并且可以查看本课程的基本信息如下图 5-8 所示。



图5-8 后台查询课程基本信息

新增课程方法代码如下，AddCourse 方法的参数值类型 AddCourseCommand 的实例是由前端管理员用户输入的课程信息组合之后，再通过多次封装得到的，其中的 Model 属性中保存的就是要新增的课程信息，然后通过数据库映射对象的 Add 方法将课程信息添加到数据库，最后通过 SaveChanges 方法对数据库修改进行保存。

```
public async Task<Result<CourseInformationModel>> AddCourse
(AddCourseCommand command, CancellationToken cancellationToken){
    var course = _mapper.Map<Course>(command.Model);
    _context.Add(course);//添加到数据库
    await _context.SaveChangesAsync();//保存数据库
    var couseMarket = _mapper.Map<CourseMarket>(command.Model);
    couseMarket.Id = course.Id;
    _context.Add(couseMarket);
    await _context.SaveChangesAsync();
    var result = _mapper.Map<CourseInformationModel>(command);
    return ResultFactory.Success(result);
}
```

后台用户在后台管理系统添加新课程之后，可以选择为新课程添加章节信息，主要包括编辑章节主题，添加对应本章节的课程视频内容，课程视频的添加在后文体现，编辑完成之后选择保存按钮即可生成课程章节，编辑课程章节页面如图 5-9 所示。

课程信息

课程名称

晨曦BIM智能翻模案例教学

章节/视频	播放量	排序	
第1章		0	+ 🗑
晨曦BIM智能翻模案例教学	20	0	🗑
==请选择课程视频==	0	1	🗑

保存

图5-9 编辑课程章节信息

5.3 视频管理微服务实现

视频管理服务使用阿里云视频点播(Video on Demand，简称 VOD)技术，并结合阿里云提供的一套用于视频点播业务的开发工具集 SDK 来实现该服务的具体功能，包括上传视频至视频点播服务器、删除云存储中的视频以及获取视频播放凭证等功能。

5.3.1 视频管理功能时序图

当后台管理员在构建课程大纲和编辑课时时，还需利用视频管理功能上传视频。通过前端的上传文件组件触发请求，调用文件上传接口，同时以文件流的形式将视频存储到阿里云服务器，并接收到一个 VideoId 作为成功上传的凭证，然后在课时设计界面，用户会看到上传成功的提示。

如果后台管理员想要删除课程，需将其视频一并移除，则需要使用阿里云视频云存储服务的删除视频功能来执行删除操作。后台管理员视频管理时序图如图 5-10 所示。

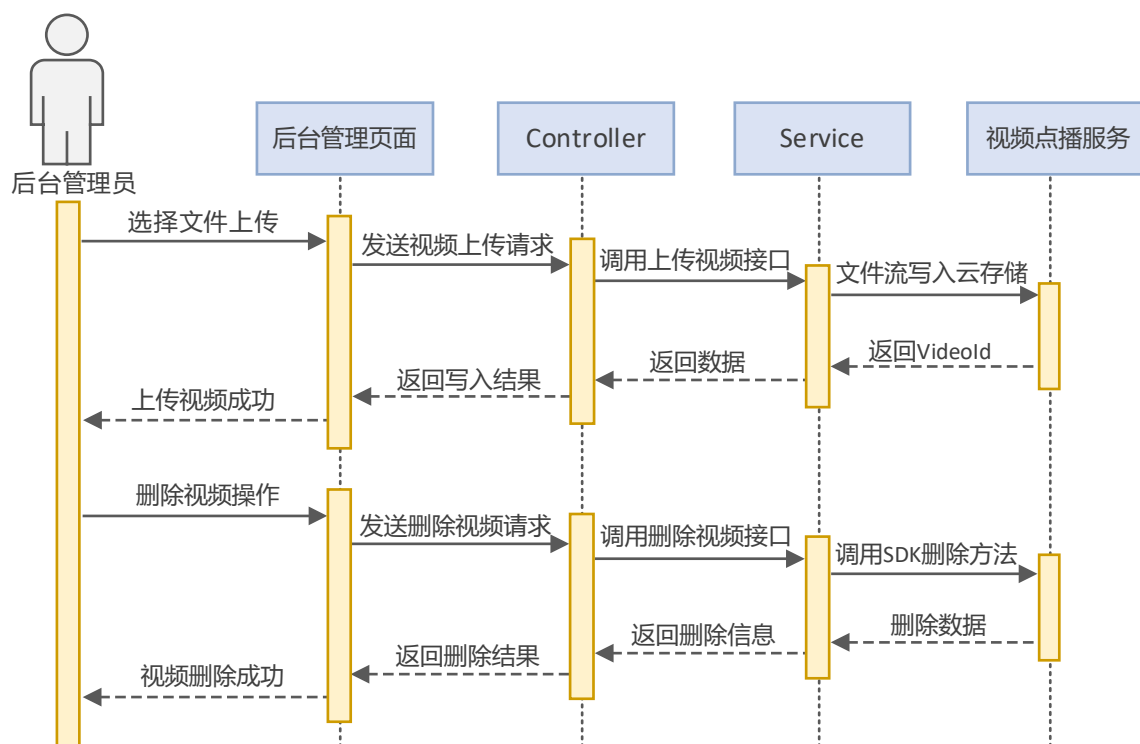


图5-10 后台管理员视频管理时序图

当前台用户浏览课程详情页并希望观看视频时，可以发起播放视频请求。此请求由控制层接收，然后调用服务层的视频点播实现方法，向阿里云申请播放凭证，获取播放权限，最后系统内嵌的阿里云 SDK 播放器利用该授权去播放用户所选的视频，从而实现完整的视频播放功能。前台用户点播视频时序图如图 5-11 所示。

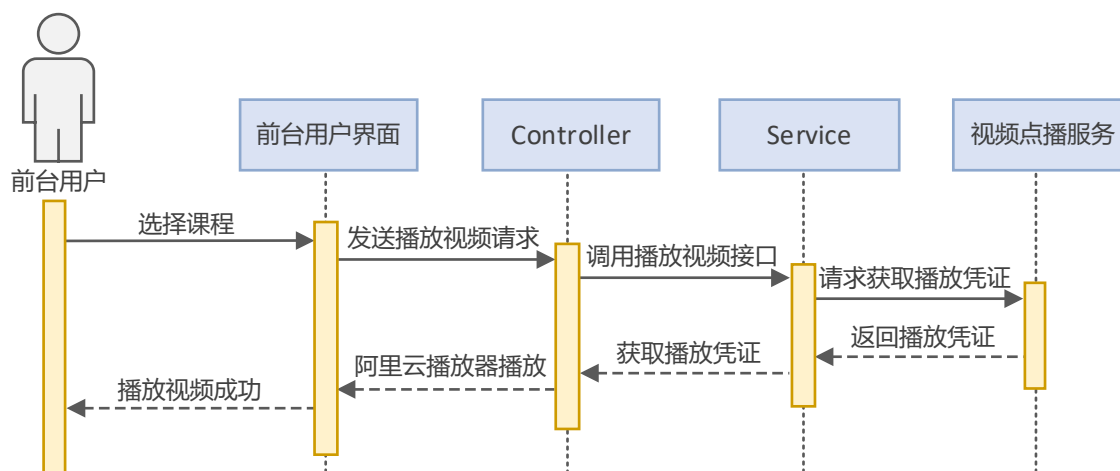


图5-11 前台用户点播视频时序图

5.3.2 视频管理类图设计与接口说明

视频管理微服务 UML 类图设计如图 5-12 所示，其中控制层中的包含视频管理控制器类 VideoController，该类提供了 UploadVideoSo 和 RemoveVideo 等视频管理接口，用于处理前端用户上传视频和删除视频的请求；这些 API 接口通过调用 IVideoService

接口中的方法，根据依赖注入和多态的方式由 `IVideoService` 的实现类 `VideoService` 来完成上传视频资源和删除视频的业务逻辑处理；其中 `VideoService` 与 `VideoModel` 和 `VodProperties` 模型类相关联，`VodProperties` 类包含调用阿里云视频点播 SDK 功能的参数配置信息，可用于实现上传和删除视频功能；在仓储层中 `OSSMediaDbContext` 为媒体服务数据库上下文类，其连接的是阿里云存储的数据库，云存储中主要保存图片和视频资源等媒体文件，`MediaSerRepository` 为媒体服务仓储接口 `IMediaSerRepository` 的实现类，为服务层中的 `VideoService` 提供视频查询等数据库操作。

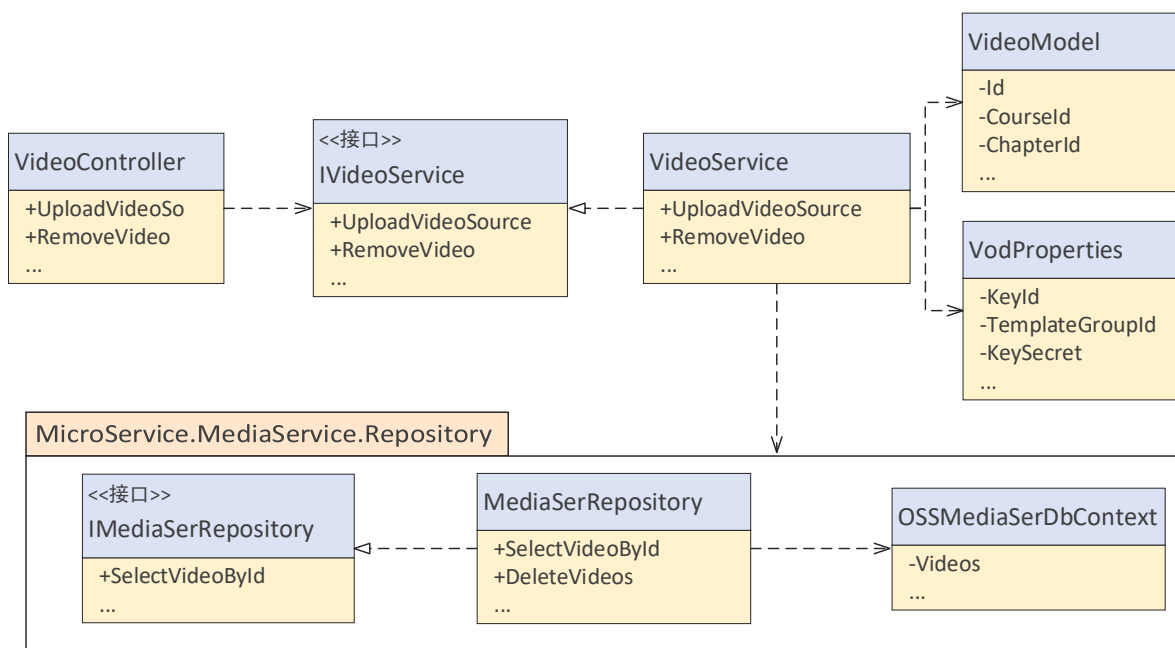


图5-12 视频管理微服务UML类图

视频管理服务中 `VideoController` 控制器的 API 接口主要用于处理用户操作视频文件的请求，其 API 接口的相关信息如表 5-5 所示。

`UploadVideo` 接口用于处理上传视频的请求，使用 POST 请求方式，调用服务层中的上传视频实现方法，利用阿里云视频点播 SDK 的上传文件流功能，将视频以文件流形式上传至阿里云存储服务器，并返回视频源地址。

`RemoveVideo` 接口用于处理删除视频文件的请求，请求方式为 DELETE，在服务层 `VideoService` 的相应方法中调用视频点播 SDK 中的删除功能来实现视频文件的删除操作，同时对云存储中的数据进行更新。

`DeleteVideoList` 接口功能是对视频文件进行批量删除，接收一个视频编号列表作为参数，调用 SDK 中的相关方法实现视频文件批量删除功能。

`GetPlayToken` 接口用于处理获取播放凭证的请求，通过调用服务层的具体实现方法，生成播放请求对象并设置源视频编号，再发送请求至视频点播服务器，收到响应数据后，前端页面播放器插件则可以通过该凭证播放相应视频。

表5-5 视频管理服务VideoController接口表

序号	接口名称	请求方式	方法参数	接口功能说明
1	Video/UploadVideo	POST	videoFileStream	上传视频
2	Video/RemoveVideo	DELETE	videoId	删除视频
3	Video/DeleteVideoList	DELETE	videoIdList	批量删除视频
4	Video/GetPlayToken	GET	videoSourceId	获取播放凭证

5.3.3 视频管理功能页面展示

在课程管理微服务中，当后台管理员在设定课程大纲，规划章节，添加课时时，可以方便地添加视频。一旦视频上传完成，前台用户选择相应课程就能播放这些视频内容。上传视频的页面如图 5-13 所示。先点击视频上传选项，挑选 MP4 格式的视频文件。后台管理员验证无误后，即可提交包含视频的课时信息，这时视频已安全上传至阿里云平台。

视频上传

基本信息

封面



注：支持jpg, png, bmp图片文件，建议图片使用大小为450x300

选择文件

更新视频

视频详情

智能翻模-案例视频教程.mp4

*如果要更新视频请选择文件，否则请清空；*仅支持.mp4、.movb、.avi格式的文件

视频名称

晨曦BIM智能翻模案例教学

所属课程

x 晨曦BIM智能翻模案例教学

排序

0

是否使用

☒ 是 ☐ 否

是否允许评论

☐ 是 ☒ 否

是否试听

☐ 是 ☒ 否

试听分钟数

1

保存

上传进度: 0 %

图5-13 上传视频页面展示

在课程详情页，用户可浏览课程大纲并选择观看视频。他们会通过申请播放授权来获取云端存储的视频，并利用阿里云播放器进行视频学习。课程详情页面如图 5-14 所示。



图5-14 视频详情页面

在阿里云播放器中，可以通过设置参数选项来添加播放功能，如设置视频倍速播放功能、视频预加载功能和发送弹幕的功能，这些功能能够提升用户的体验感。视频播放页面如图 5-15 所示。

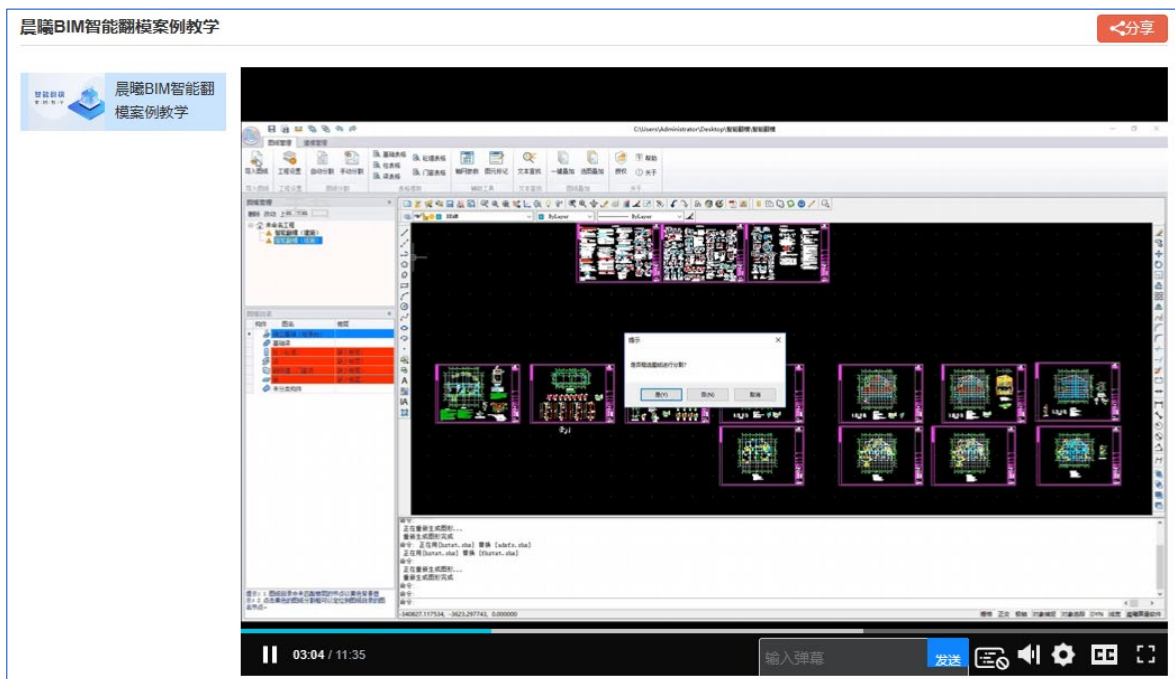


图5-15 视频播放器页面

上传视频文件部分代码如下，UploadVideo 方法中的参数 videoModel 保存着视频文件的文件名以及存储路径。通过创建一个 MediaFile 对象来接收视频在阿里云存储的保存路径等信息，然后再将该对象通过数据库关系映射添加到数据库的视频表中。

```
public async Task<Result<MediaFile>> UploadVideo (UploadVideoCommand
videoModel, CancellationToken cancellationToken){
    using var fileStream = videoModel.File.OpenReadStream();
    _ossClient.PutObject(new(_options.FileBucket, videoModel.SavePath,
fileStream));
    var media = new MediaFile{
        FileId = videoModel.SavePath,
        AccessPath =
        $"https://{_options.FileBucket}.{_options.Endpoint}/{request.SavePath}",
        FileName = videoModel.File.Name,
        StoragePath = videoModel.SavePath};
    await _context.AddAsync(media, cancellationToken);
    await _context.SaveChangesAsync(cancellationToken);
    var mediaProcess = new MediaProcess{
        AccessPath =
        $"https://{_options.FileBucket}.{_options.Endpoint}/{ videoModel.SavePath}",
        MediaFile = media.Id,
        MediaFileName = media.FileName,
        StorageBucket = _options.FileBucket};
    await _context.AddAsync(mediaProcess, cancellationToken);
    return ResultFactory.Success(media);
}
```

5.4 订单管理微服务实现

订单管理微服务整合了微信支付 SDK 和订单号生成等技术，主要负责订单和第三方支付的处理，包括创建订单和处理支付流程。当订单管理微服务完成后，课程管理微服务能远程调用它，执行订单管理和支付操作。

5.4.1 订单管理时序图

当用户决定购买课程，点击购买按钮，会调用控制器中的 API 接口来处理请求，接着调用服务层的新增订单实现方法。系统据此生成包含课程详情、支付金额、订单编号等的订单信息，并保存到数据库中，页面上会显示订单概览，并提供支付按钮进入支付流程。前台用户使用订单管理功能时序图如图 5-16 所示。

当点击支付按钮，系统会引导至微信支付的二维码界面。在控制层接收到支付请求后，它会启动 OrderService 调用微信支付的下单接口。随后，微信系统回应一个链

接参数 `code_url`，此参数将被用来构建二维码的地址，并且订单详情（包括课程名称和支付金额）会被嵌入其中。用户在扫描二维码并核对订单信息无误后，支付流程便得以完成，成功的交易信息会记录到订单支付表中。紧接着，用户页面会自动转至课程页面，通过订单管理功能，可以实现付费课程视频的播放。

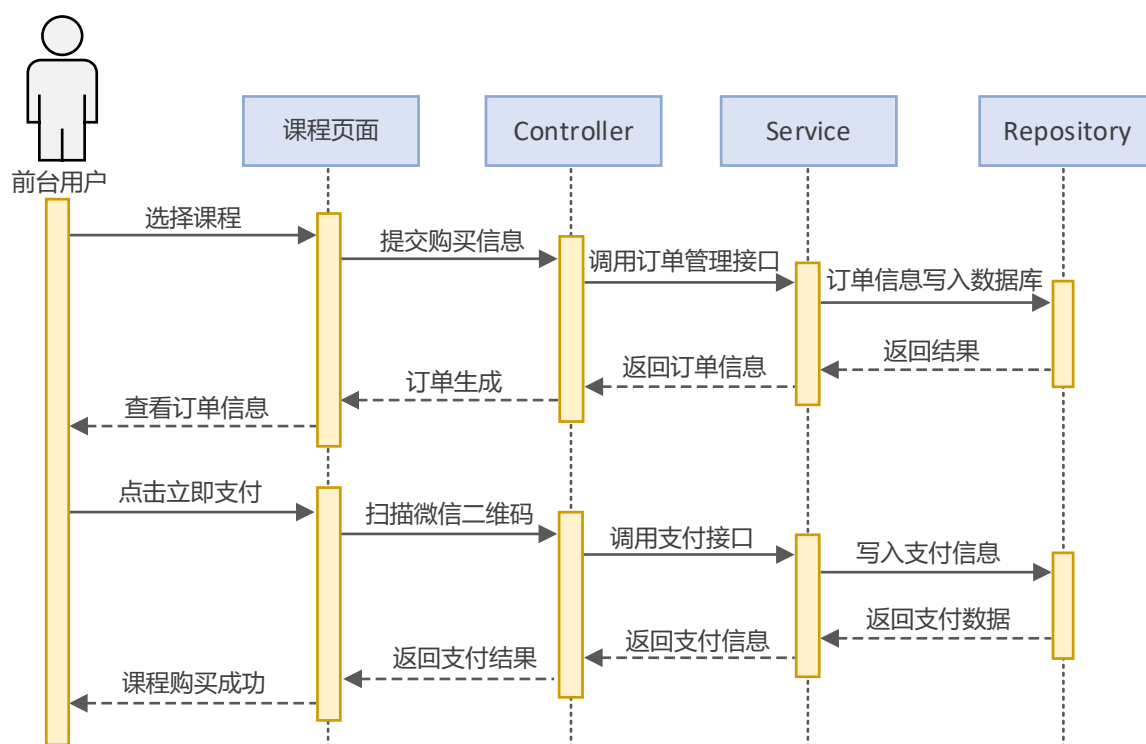


图5-16 前台用户使用订单管理功能时序图

5.4.2 订单管理类图设计与接口说明

订单管理微服务 UML 类图如图 5-17 所示，在 Controller 控制层中，`OrderController` 为订单管理控制器类，依赖于 `IOrderService` 接口，为前端提供生成订单 `SaveOrder`、获取订单信息 `GetOrderInfo` 等功能的 API 接口，调用 `IOrderService` 接口中相应的业务处理方法；`WXPayController` 为微信支付控制器类，依赖于 `IOrderService` 接口，主要用于处理前端用户发起的微信支付请求，调用 `IOrderService` 中的相应方法，并利用微信支付 SDK 得到订单交易信息，通过 `CreateWXNative` 方法生成支付二维码来实现微信支付功能。

在 Service 服务层中，`OrderService` 类作为 `IOrderService` 接口的实现类，关联订单 `OrderModel`、订单详情 `OrderDetailModel` 数据模型类，根据业务功能需求对 `IOrderService` 接口的生成订单 `SaveOrder`、编号查询订单信息 `GetOrderById`、用户编号查询订单 `SelectOrderByUid` 等方法进行实现；`WXPaySDK` 作为微信支付的第三方接口，提供了微信支付配置类 `WXPayConfig`、报告类 `WXPayReport`、微信支付请求类 `WXPayRequest` 等实现类，微信支付控制类就是通过调用该 SDK 接口来完成微信支付

的功能。

在 Repository 仓储层中 OrderSerDbContext 为订单数据库上下文类，其中的 Orders 和 OrderDetails 属性分别与数据库中的 t_orders 表和 t_orderdetails 表相对应，并且与 OrderSerRepository 类是依赖的关系，而该类为订单服务仓储接口 IOrderSerRepository 的实现类，为服务层中的 OrderService 提供订单查询等数据库操作。

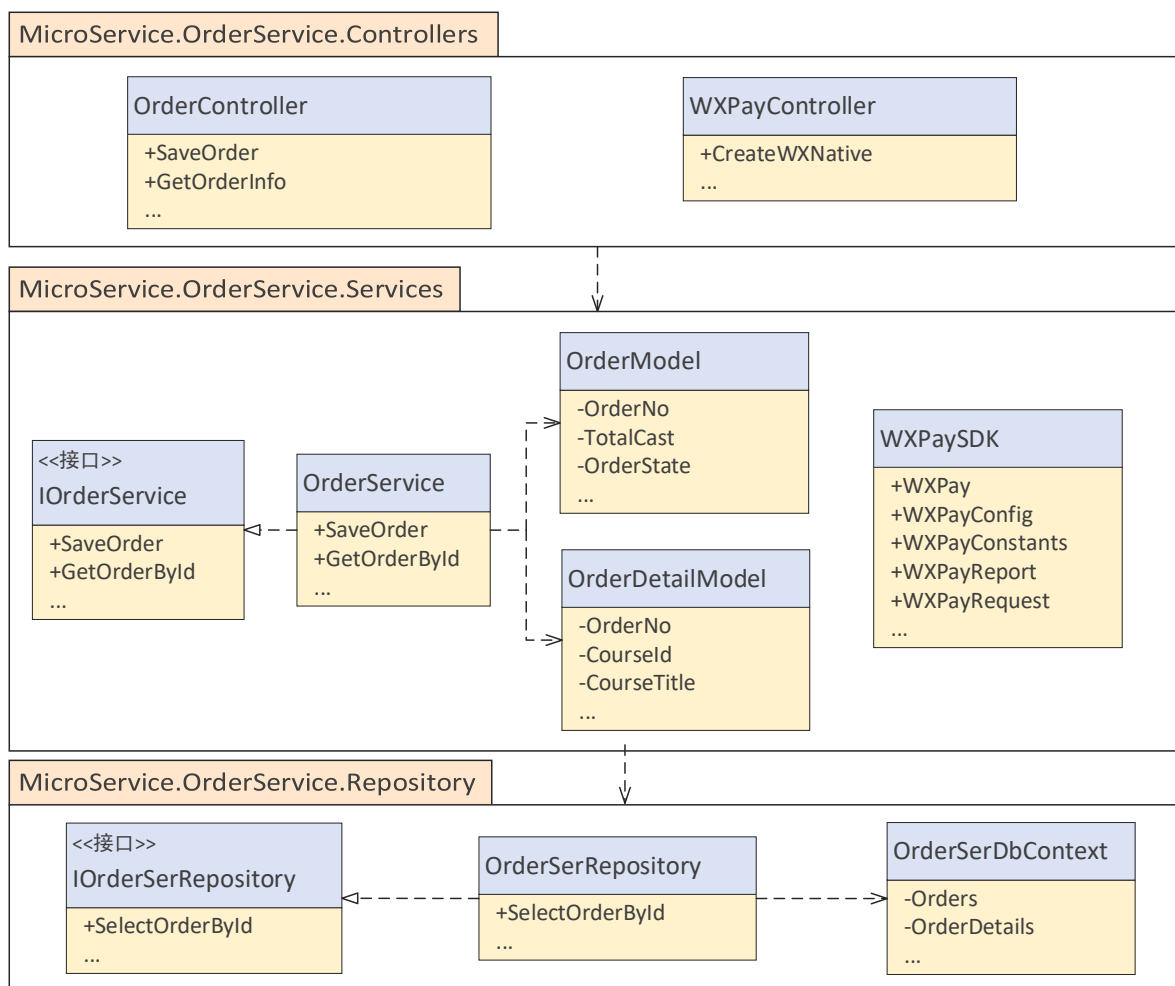


图5-17 订单管理微服务UML类图

订单管理服务中 OrderController 和 WXPAYController 两个控制器类的 API 接口是用于处理订单交易和支付相关的请求，其 API 接口的相关信息如表 5-6 所示。

SaveOrder 接口用于处理新增订单的请求，请求方式为 POST，接收两个参数分别为课程编号 courseId 和用户编号 userId，userId 是通过请求体的方式进行传递，代表用户身份信息。当用户发送购买请求时，调用服务层中的创建订单方法生成订单号，并将课程和用户信息写入数据库表中。

GetOrderInfo 接口用于获取订单信息，利用 userId 参数确定用户身份，再将订单编号传递给服务层中的 GetOrderById 方法获取到该订单信息。

GetOrderList 接口用于获取用户所有订单列表，将用户 Id 作为参数传递给服务层

的 `selectOrderByUserId` 方法来查询指定用户的购买记录。

`RemoveOrder` 接口用于处理取消订单的操作，请求方式为 `DELETE`，接收一个订单编号 `OrderId` 作为参数，调用服务层中删除订单的方法实现取消订单功能。

`CreateWXNative` 接口用于发起微信 Native 支付方式来完成订单支付，接收订单 `Id` 作为参数，调用 `WXPaySDK` 的统一下单接口，该 API 接口会返回一个包含预支付会话标识(`prepay_id`)等信息的 JSON 数据，根据返回的数据生成支付二维码。

表5-6 订单管理服务接口表

序号	接口名称	请求方式	方法参数	接口功能说明
1	Order/SaveOrder	POST	courseId、userId	新增订单
2	Order/GetOrderInfo	GET	courseId、userId	获取订单信息
3	Order/GetOrderList	GET	userId	获取用户已购课程信息
4	Order/RemoveOrder	DELETE	orderId	删除/取消订单
5	Order/QueryPayStatus	GET	orderId	查询订单状态
6	Pay/CreateWXNative	GET	orderId	发起微信Native支付

5.4.3 订单管理页面展示

前台用户选择付费课程，点击立即购买按钮，此时会进行订单生成操作，订单确认页面如图 5-18 所示。

订单信息



土建造价基础+BIM土建体验课程

单价: ¥ 0.01

数量: 1

总价: ¥ 0.01

学员信息

姓名

admin

手机

13311111111

QQ

1231231231

实付金额: ¥0.01

提交订单

图5-18 订单确认页面

在确认完成订单信息后，点击“提交订单”，将跳转至订单支付页面，选择微信支付后生成支付二维码。支付二维码如图 5-19 所示，扫码后微信支付如图 5-20 所示。



图5-19 支付二维码



图5-20 微信支付

前台用户可以登录系统之后在个人信息管理页面查看订单信息，其中会返回有关于该用户的所有历史订单，用户订单信息页面如图 5-21 所示。



图5-21 用户订单信息页面

创建订单并保存到数据库功能部分代码如下，CreateOrder 创建订单方法，首先判断系统用户是否已经存在对应的订单，如果有则返回提示信息；没有则继续将方法参数中的 orderModel 映射成 Order 对象，然后将该订单信息添加到数据库订单表中，最后保存对数据库的修改。

```
public async Task<Result<int>> CreateOrder(OrderCreateCommand orderModel,
Cancellation token cancellationToken)
{
    if (await _dbContext.Orders.AnyAsync(o => o.ExternalBusinessId ==
orderModel.ExternalBusinessId))
        return ResultFactory.Fail<int>("对应的订单已经存在，请勿重复创建");
    var order = _mapper.Map<Order>( orderModel);
    var orderItems =
_mapper.Map<IEnumerable<OrderItem>>( orderModel.OrderItems);
    await _dbContext.AddAsync(order); //添加订单到数据库订单表
    await _dbContext.AddRangeAsync(orderItems); //保存数据库修改
    return ResultFactory.Success(order.Id);
}
```

5.5 用户管理微服务实现

用户管理服务主要用于实现用户登录注册和用户信息管理两大功能，同时服务于前台用户和后台管理员。在实现用户注册功能时还需调用第三方短信服务发送短信验证码，用户信息管理包括前台用户对个人信息管理、后台管理员对用户数据进行修改删除等操作。

5.5.1 用户服务功能时序图

用户进行注册账户时，首先点击登录注册按钮，将跳转到注册登录页面。在该页面，用户需填写账户名称、手机号和密码等必要信息，然后提交注册请求。控制层会调用服务层接口，生成一个随机验证码并通过阿里云短信服务发送到用户的手机，并同步存储在 Redis 缓存中，设定一定时效。用户接收到短信后，需输入验证码以完成注册。注册成功后即可使用手机号进行登录。用户登录注册时序图如图 5-22 所示。

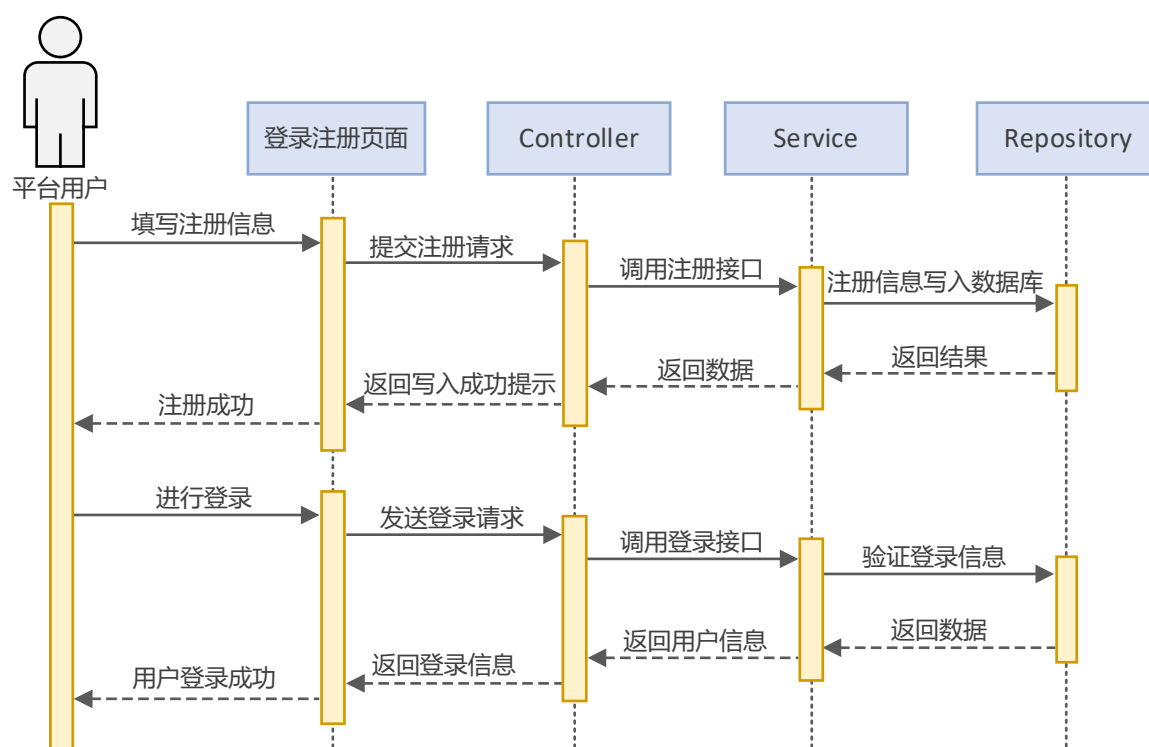


图5-22 用户登录注册时序图

5.5.2 用户服务类图设计与接口说明

用户管理微服务 UML 类图如图 5-23 所示，在控制层中 LoginController 作为用户登录注册控制器，为前端登录注册请求提供了注册 Register、登录 Login、获取短信验证码 GetSmsCode 等功能的 API 接口，与 ILoginService 接口是依赖关系，需要调用其中的方法来实现业务功能，并向前端发送响应结果；UserController 为用户个人管理控制器，为前端提供获取用户信息 GetUserInfo、更新用户信息 UpdateUserInfo 等功能的 API 接口，与 IUserService 接口是依赖关系，需要调用该接口中的业务处理方法。

服务层中的 LoginService 类是登录注册服务接口 ILoginService 的具体实现类，与登录 LoginModel 和注册 RegisterModel 数据模型类相关联，根据业务功能需求实现了用户注册 Register、登录 Login 等方法；UserService 是用户管理服务接口 IUserService 的实现类，与用户信息数据模型类 UserInfoModel 关联，基于业务需求实现了获取用户信息 GetUserInfo、更新用户信息 UpdateUserInfo 等方法。

在 Repository 仓储层中 AdminSerDbContext 为用户数据库上下文类，其中的 Users 属性与数据库中的用户表 t_users 相对应，并且与 UserSerRepository 类是依赖关系，而该类为用户服务仓储接口 IAdminSerRepository 的实现类，为服务层中的 UserService 提供用户信息查询等数据库操作。

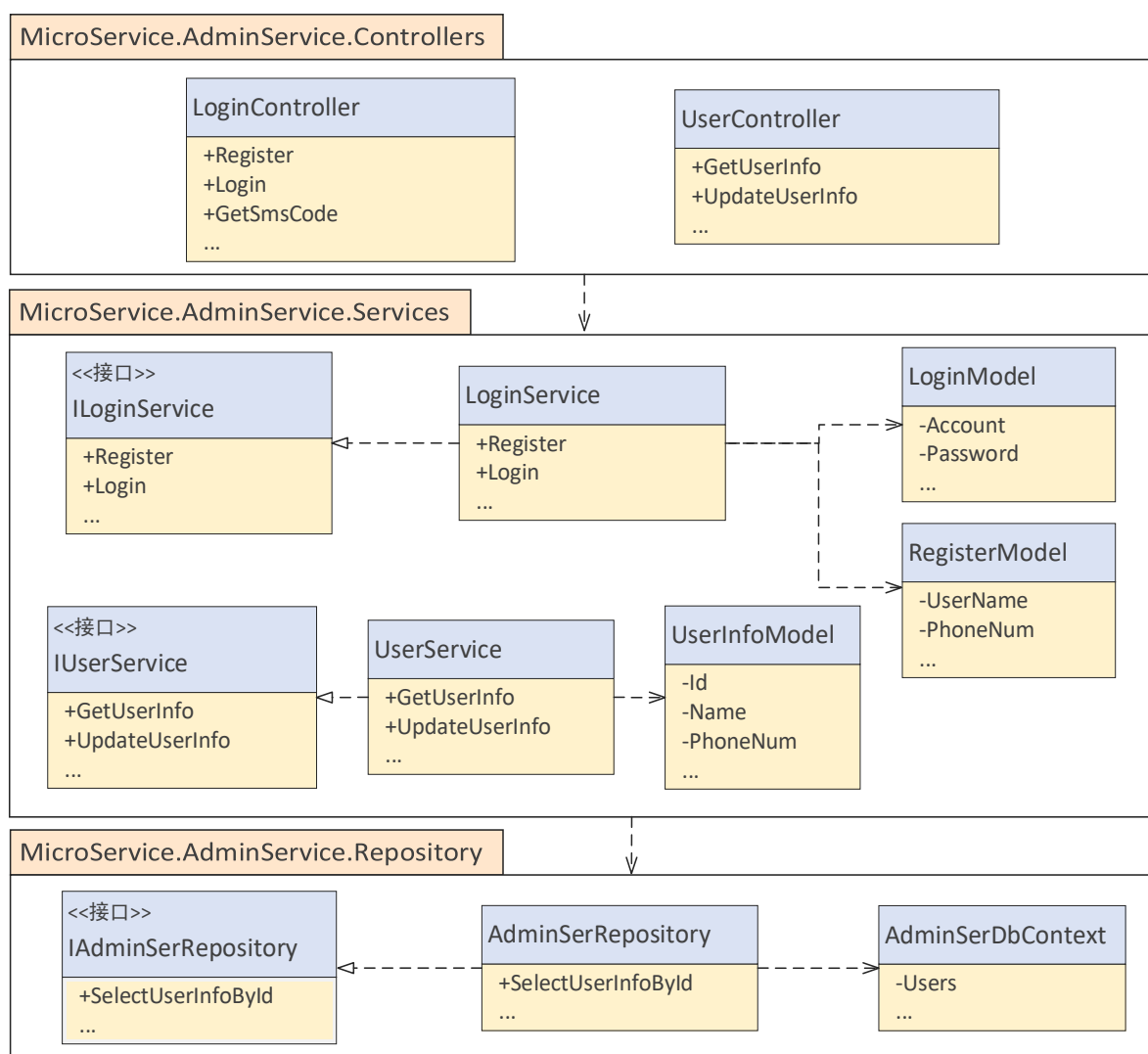


图5-23 用户管理微服务UML类图

用户管理服务中 `LoginController` 和 `UserController` 两个控制器类的 API 接口是用于登录注册和用户信息管理请求的处理，其 API 接口的相关信息如表 5-7 所示。

Register 接口用于处理新用户注册账号的请求，请求方式为 **POST**，将前端用户输入的注册信息 `registerInfo` 以请求体的方式发送至服务器端的注册接口，然后调用服务层中注册用户的实现方法，验证注册信息中的数据是否合法，同时检查用户名是否存在；验证通过后将执行注册相关的业务逻辑，生成用户 `Id` 并将用户信息存储到数据库中。

Login 接口用于实现用户登录功能，用户输入手机号码和密码后利用 **POST** 请求将数据以请求体的形式发送到服务器端的登录接口，再调用服务层中的用户登录实现方法，该方法会验证用户信息并生成 `jwtToken` 参数，该参数会作为响应数据保存至用户浏览器；后续用户的请求若包含有效的 `jwtToken` 则会被正常处理，否则系统将判断用户未登录或 `token` 过期，并引导用户重新登录。

Logout 接口用于处理用户登出操作，终止用户会话并清理相关资源。在服务端接收到登出请求时会先验证 jwtToken 的有效性，然后直接将该令牌设置为失效状态并返回登出操作的结果。

GetSmsCode 接口用于处理获取短信验证码的请求，请求方式为 GET，接收用户从注册页面输入的手机号 phoneNum 作为参数。在服务层调用 SendCode 方法，该方法会校验手机号的有效性，然后生成一个随机验证码，并通过阿里云短信服务发送该验证码到用户手机；同时验证码会被保存在 Redis 缓存中并设置过期时间，使用户可以使用此验证码完成短信注册流程。

GetUserInfo 接口用于获取登录用户的订单信息，利用 userId 参数确定用户身份，调用服务层中的 GetUserById 方法获取到用户个人信息，并返回至客户端用户界面进行展示。

UpdateUserInfo 接口用于修改用户个人信息，请求方式为 PUT，接收用户 Id 作为参数并调用服务层中相应的修改用户信息方法，同时在数据库表中更新修改内容。

表5-7 用户管理服务接口表

序号	接口名称	请求方式	方法参数	接口功能描述
1	Login/Register	POST	registerInfo	注册用户
2	Login/Login	POST	loginInfo	登录
3	Login/LoginOut	GET	无	登出
4	Login/GetSmsCode	GET	phoneNum	获取短信验证码
5	User/GetUserInfo	GET	userId	获取用户信息
6	User/UpdateUserInfo	PUT	userId	更新/修改用户信息

5.5.3 用户服务页面展示

前台用户使用登录注册功能时，若用户没有注册过账号则可以点击立即注册跳转至注册界面，此时需要填写手机号等相关信息，之后通过发送短信验证码进行手机号验证。注册页面显示如图 5-24 所示，登录页面如图 5-25 所示。

用户注册

已有账号, [马上登录](#)

手机号码:

13479511111

密码:

.....

确认密码:

.....

真实姓名:

圭吾

QQ:

1231231231

验证码:

请输入短信验证码

获取验证码

点击注册

图5-24 注册页面

用户登录

联系方式:

13479511111

密码:

.....

登 录

还不是会员? [立即注册](#)

[忘记密码?](#)

图5-25 登录页面

在用户进入系统，完成注册操作，并登录成功后，可以查看个人信息页面，其中会显示用户头像、名称、性别等信息。用户个人信息页面展示如图 5-26 所示。

个人信息



更改

移除

姓名 *

圭吾

性别 *

☒ 男 ☐ 女

联系电话 *

13479511111

QQ *

1231231231

电子邮箱

保存

图5-26 用户个人信息页面

用户注册登录功能的部分代码如下,注册用户时,首先判断是否有同名账户存在,然后生成验证码并存储到 Redis 中,判断用户输入的验证码是否与 Redis 数据库中的一致,最后将用户信息保存到数据库用户表中,完成用户注册操作。用户在登录页面填入账户和密码,信息验证通过后会返回 JwtToken 令牌,用于后续访问系统资源。

```
//用户注册
public async Task<Result<string>> Register(RegisterCommand request,
Cancellation token cancellationToken)
{
    if (await _userManager.Users.AnyAsync(u => u.UserName == request.Name))
    { return ResultFactory.Fail<string>("同名账号已经存在");}
    String checkCode = RandomUtils.getFourBitRandom(); //生成验证码
    smsService.send(mobile,checkCode); //发送验证码
    //存储验证码
    redisTemplate.opsForValue().set(mobile, checkcode, 5,DateTime.Minutes);
    if (request.code != redisTemplate. opsForValue().get(checkcode))
    return ResultFactory.Fail<string>("验证码错误"); //判断验证码是否输入正确
    var result = await _userManager.CreateAsync(new IdentityUser { UserName =
request.Name }, request.Password);
    return result switch{
        null or { Succeeded: false } => ResultFactory.Fail<string>("账号注册失败"),
        _ => ResultFactory.Success(request.Name) };
}

//用户登录
public async Task<Result<string>> Login(LoginCommand request, Cancellation token
cancellationToken){
    var result = await _signinManager.PasswordSignInAsync(request.UserName,
request.Password, false, true);
    if (result.IsLockedOut)
        return ResultFactory.Fail<string>("失败次数太多, 帐号已经被锁定");
    if (result.Succeeded is false)
        return ResultFactory.Fail<string>("登录失败");
    var user = await _userManager.Users.SingleAsync(u => u.UserName ==
request.UserName);
    var tokenHandler = new JwtSecurityTokenHandler();
    var tokenDescriptor = new SecurityTokenDescriptor(){
        Expires = DateTime.UtcNow.Add(_options.Value.Expire),
        Issuer = _options.Value.Issuer,
```

```
Audience = _options.Value.Audience,
SigningCredentials = new SigningCredentials(new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_options.Value.Credentials)),
SecurityAlgorithms.HmacSha256Signature),
Claims = new Dictionary<string, object>{
    [nameof(UserInfo.Id)] = user.Id,
    [nameof(UserInfo.Name)] = user.UserName,
}
};
var token = tokenHandler.CreateEncodedJwt(tokenDescriptor);
return ResultFactory.Success(token);
}
```

5.6 系统测试

5.6.1 系统测试环境

测试环境的选取直接影响了系统测试的准确性，为确保测试结果的可靠性，测试环境通常选择系统用户常用的设备环境，以模拟真实的使用场景，从而有效评估系统的性能和稳定性。具体测试环境如表 5-8 所示。

表5-8 测试环境配置

测试软硬件	软硬件版本型号
处理器	Intel(R) Core(TM) i5-11400H
内存	16G
操作系统	Windows 10
浏览器	IE、谷歌、火狐浏览器
模拟HTTP 请求工具	POSTMAN、JMeter

5.6.2 系统功能测试

功能测试，也被称为黑盒测试，是一种基于软件需求的测试方法。这种方法侧重于通过软件的用户界面或其他输入接口来验证其是否满足预定义的功能需求^[55]。在进行功能测试时，测试人员无需深入了解软件的内部逻辑或代码结构，而是通过模拟用户操作来观察系统的响应，进而评估软件系统是否达到了预期的功能目标。下面将对划分出各个微服务模块进行功能测试。

(1)用户管理微服务功能测试

测试用例涵盖了用户管理的多个方面,包括用户注册、登录、用户个人信息管理和权限管理等。通过执行这些测试用例,可以验证用户管理模块的各个功能是否能够满足用户需求。其具体的测试内容和测试结果如表 5-9 所示。

表5-9 用户管理服务测试

用例说明	用户管理服务用例
测试目的	检查用户管理模块功能能否正常运行
测试内容	1.用户跳转至注册页面,输入正确的手机号码、密码、确认密码,并获取短信验证码来注册新用户;随后跳转到登录页面,填写已注册的账号和密码来登录系统; 2.管理员登录后台管理系统给新用户分配角色,修改用户信息,最后将用户删除; 3.用户登录系统后点击头像查看个人信息,对姓名、性别、邮箱和用户头像等信息进行添加或修改。 用户注册时能够成功获取到短信验证码,成功注册新用户; 注册后在登录页面输入账户密码能够成功登录系统,并在首页右上角显示账号名;
预期结果	用户登录系统后能够成功查看到个人信息并对其进行修改; 管理员登录后台可以对用户信息进行增删改查操作;
测试结果	与预期结果一致
结论	测试通过

(2)课程管理微服务功能测试

课程管理微服务测试用例如表 5-10 所示,前台用户登录账户进入系统首页,点击选择某课程进入课程详情页面,成功显示出课程基本信息则说明课程管理展示页面功能正常。

表5-10 课程管理服务测试

用例说明	课程管理服务用例
测试目的	检查课程管理模块功能能否正常运行
测试内容	1.管理员在后台系统中点击添加课程,编辑课程大纲,添加课程章节,引入课程视频; 2.管理员通过课程标题查询课程记录,并删除该课程; 3.前台用户进入系统,首页展示课程列表信息,点击课程图标查看课程详情,点击课程分类查看某一类的课程信息; 4.在免费课程的详情页中点击“加入课程”按钮将该课程添加到“我的课程”中;随后查看我的课程中是否存在该课程。

续表5-10 课程管理服务测试

预期结果	管理员能成功对课程信息、课程章节以及课程视频数据进行增删改查；系统首页展示课程封面等信息，通过分类导航能成功查看某一类课程；前台用户将成功获得免费课程，并进行课程学习。
测试结果	与预期结果一致
结论	测试通过

(3)视频管理微服务功能测试

视频管理微服务测试用例如表 5-11 所示，前台用户登录系统之后，选择某免费课程，然后点击加入课程按钮，代表该课程已添加到我的课程中了，之后选择该课程的某一章节视频进行观看，若成功跳转到视频播放页则说明视频点播页面功能正常。

表5-11 视频管理服务测试

用例说明	视频管理服务用例
测试目的	检查视频管理模块功能能否正常运行
测试内容	1.前台用户点击课程章节视频，测试是否跳转到视频播放页面； 2.管理员调用视频管理方法，对阿里云视频点播服务器中的视频资源进行删除，并上传新的视频文件。
预期结果	课程视频能正常播放；成功上传视频文件到云服务器。
测试结果	与预期结果一致
结论	测试通过

(4)订单管理微服务功能测试

订单管理微服务测试用例如表 5-12 所示，前台用户登录系统之后，选择某付费课程，然后点击立即购买，随后跳转到订单页面，之后点击支付，若成功显示出付款吗则说明订单管理服务功能正常。

表5-12 订单管理服务测试

用例说明	订单管理服务用例
测试目的	检查视频管理模块功能能否正常运行
测试内容	1.用户登录系统后选择某付费课程进入课程详情页面，点击“立即购买”，查看是否生成订单信息； 2.在订单信息页面中点击“提交订单”，测试是否跳转到支付页面，然后选择微信支付方式，测试是否生成微信支付二维码； 3.用户在我的订单页面中查看所有历史订单信息。
预期结果	成功生成订单信息，提交订单后能顺利完成支付操作。
测试结果	与预期结果一致
结论	测试通过

5.6.3 非功能性测试

非功能性测试也是软件测试中的重要组成部分,它关注软件系统在性能、可靠性、易用性、安全性、兼容性、可维护性等方面的表现。这些属性通常不直接影响系统的业务逻辑,但对于系统整体质量和使用体验至关重要。本节将从性能和兼容性两方面对在线学习平台进行测试。

(1)性能测试

性能测试的主要目的是评估系统的稳定性、响应速度、吞吐量和资源利用率等关键性能指标,确保系统在实际应用中能够稳定高效地运行^[56]。从用户发出请求到系统返回响应的的时间,是评估系统性能的重要指标之一,可以借助浏览器的性能工具来获取请求页面资源的响应时间。本系统部分页面加载响应时间表如表 5-13 所示,系统中大部分页面加载耗时都在 1000ms 以内;而视频播放页面由于需要调用第三方的视频点播方法,同时还要加载视频播放器插件,所以页面加载耗时较长,但控制在 2000ms 以内,这种表现足够满足系统性能需求。

表5-13 页面加载响应时间表

用例编号	测试内容	第一次	第二次	第三次	平均
1	系统首页	832ms	922ms	853ms	869ms
2	用户登录页面	536ms	556ms	569ms	553ms
3	课程详情页面	821ms	895ms	915ms	877ms
4	生成订单信息	813ms	798ms	824ms	812ms
5	视频播放页面	1670ms	1476ms	1557ms	1568ms

(2)兼容性测试

基于 Web 技术实现的在线学习平台需要在浏览器中才能使用到该系统中的功能,浏览器兼容性测试是评估该系统兼容性的主要方式。测试人员分别使用 IE、谷歌和火狐三个不同内核的浏览器对系统进行访问,在客户端发送一系列请求,同时记录返回的数据,最后对三个浏览器的测试数据与预期结果进行比对得出测试结果。经过测试,结果达到预期水平,系统兼容性测试通过。测试内容如表 5-14 所示。

表5-14 浏览器兼容性测试

浏览器	内核	测试内容	测试结果
IE	Trident	在三个不同浏览器中访问系统,执行登录注册、管理个人信息、查看课程详情、加入课程、购买课程、观看课程视频等操作。	访问系统功能正常,未出现异常信息
谷歌	Blink		
火狐	Gecko		

(3)高并发压力测试

高并发压力测试是一种在模拟大量用户同时访问系统的情况下，验证系统在高负载情况下的稳定性、可靠性和性能表现，发现系统的性能瓶颈，以便进行优化和调整。本论文选用 Apache JMeter 作为系统的测试工具，对系统的请求响应时间、系统吞吐量和并发用户数等性能指标进行测试^[57]。该工具的用户界面如图 5-27 所示，其中线程数表示用户并发访问数量，Ramp-Up 时间表示从测试开始到达到最大并发用户数所需的时间，将其设置为 1 秒来模拟短时间内并发访问的增长。

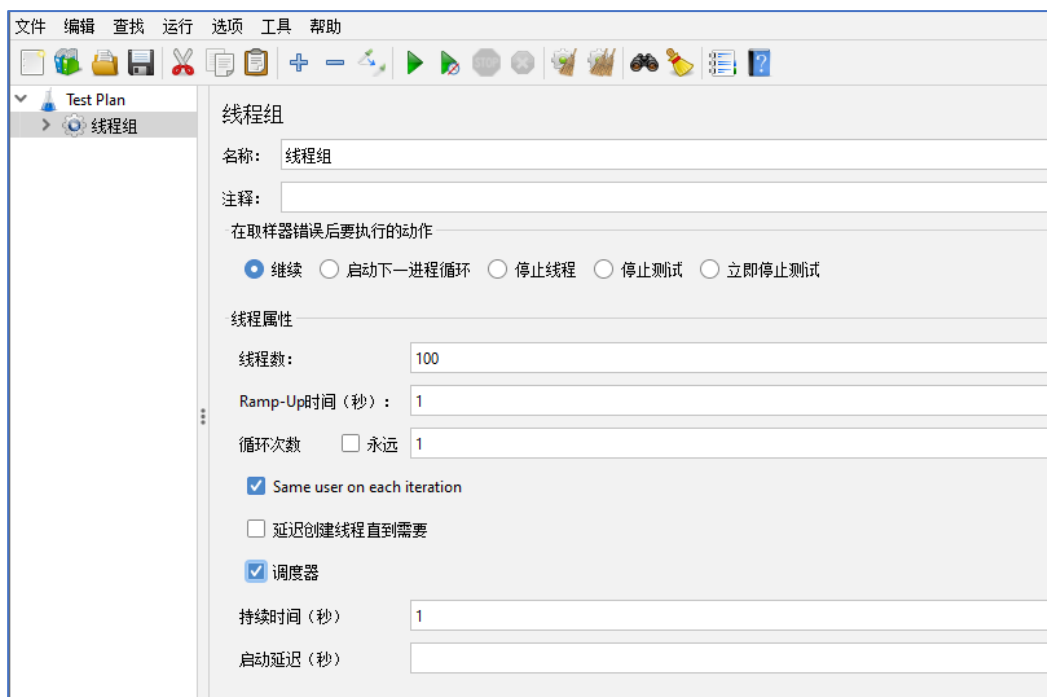


图5-27 JMeter用户界面

本次测试请求以查看课程详情为例，IP 地址和路径参数为 `http://localhost:5500/course-service/CourseDetail?qCourseID=125&qCourseClassId=8`，通过设置不同的线程数来模拟短时间内对课程详情信息的并发访问，分别对 100、500、1000、2000 和 3000 个并发线程数进行了多次性能测试，主要记录请求响应时间和吞吐量等重要指标，分析出系统的性能表现在何种情况下达到最优。其测试结果如表 5-15 所示。

表5-15 高并发压力测试结果

序号	并发线程数	平均响应时间	最小响应时间	最大响应时间	成功率	吞吐量/s
1	100	51ms	40ms	79ms	100%	96.8
2	500	58ms	42ms	94ms	100%	342.7
3	1000	64ms	45ms	103ms	100%	418.1
4	2000	96ms	40ms	218ms	100%	432.5
5	3000	723ms	40ms	6103ms	93.2%	106.8

测试表中响应时间表示服务器对各请求的处理耗时时间，这个时间越短说明系统性能越好；成功率表示在高并发情况下服务器能正确处理请求数量的百分比；吞吐量表示服务器每秒处理请求的数量，数量越大则系统并发能力越强。通过对表中的结果分析，当并发线程数在 2000 以下，系统平均响应时间能够控制在 100ms 以内，系统访问也并未出现异常情况，系统运行相对比较稳定。而当并发线程数达到 3000 时，访问成功率开始降低，响应时间明显增长，同时吞吐量也减少；最后通过减少并发访问量，且保证访问不出现异常的情况下，系统的并发量稳定在 2100 左右，说明系统具有一定的处理高并发能力。在后续系统上线之后，可以通过升级服务器硬件设施或 SQL 优化等方式来提升系统性能。

5.7 本章小结

本章节对基于微服务架构的在线学习平台实现进行了介绍，通过功能时序图来展示各微服务之间的交互过程，简单描述了各微服务中的一些核心方法，再通过页面展示和实际代码的呈现的方式来体现各个微服务模块所能完成的功能。最后为确保系统的正常运行，对系统进行功能测试和非功能性测试。

第六章 总结与展望

6.1 总结

本文设计并实现了一个基于微服务架构的在线学习平台，与慕课等多学科综合类课程系统不同，该平台专注于工程领域的学习内容，旨在为广大工程领域从业者提供一个便捷、高效的学习途径，同时拓展公司的教学培训业务。首先，本论文深入探讨了基于微服务架构的在线学习平台的研究背景及其重要意义，揭示了该领域的研究现状与发展趋势。通过对比分析国内外关于微服务架构和在线学习平台的研究现状，明确了选题的原因，使用微服务架构来开发在线学习平台是行业发展的主流所趋，这种架构模式能够提高系统的可伸缩性和高并发处理能力。

其次，介绍了微服务架构的基本概念，并分析了从单体架构到面向服务架构，再到微服务架构这一发展历程，然后对单体架构和微服务架构进行对比分析，以突出微服务架构的优势特点；并简要阐述了微服务的拆分准则。之后介绍了微服务相关技术组件，包括 Consul 服务注册、Ocelot 网关和 Apollo 配置中心，另外还对 Identity Server4 统一鉴权中心和 gRPC 通信机制进行原理分析，确定了本系统所使用到的技术栈。

然后，对在线学习平台进行需求分析，先经过可行性分析对系统实施的可能性进行全面评估。之后通过用户需求分析将系统用户划分为前台用户和后台管理员两种类型，采用 UML 用例图对这两类用户进行了详细的用例分析，并对用户角色进行了合理的权限分配。根据微服务架构的理念，按业务逻辑从系统中拆分出四个主要的微服务，分别为用户服务、课程服务、视频服务和订单服务，对这些服务模块进行详细的功能性分析，并在最后还对分析了系统的非功能性需求。

之后根据系统的需求分析对在线学习平台的概要设计进行阐述，先对在线学习系统整体架构进行设计；然后根据每个服务的功能模块图，对各微服务进行了更为精确的功能划分，并且对一些主要功能流程进行了分析。之后根据需求分析对系统数据库进行设计，分析出系统中各个实体之间的关系并生成 E-R 图，然后对各实体对应的数据库表结构进行物理设计，并对表中的字段做了约束。

最后详细阐述了在线学习平台的具体实现。先对系统的开发环境进行了介绍，然后通过功能时序图来展示各微服务之间的交互过程，了解它们之间的依赖关系。简单描述了各微服务中的一些核心方法，并通过页面展示和实际代码的呈现的方式来体现各个微服务模块所能完成的功能。最后为确保系统的正常运行，对系统进行了功能测试和非功能性测试，在功能测试方面，针对每个微服务模块设计了详细的测试用例，非功能性测试包括性能测试和稳定性测试，经过一系列的测试工作，全面验证了系统的质量和可靠性。

6.2 创新点

(1)架构创新

本文采用微服务架构对在线学习平台进行设计，实现了系统模块间的解耦，且每个微服务都可以独立开发和部署，提高了系统的可维护性和可扩展性。微服务架构允许每个服务使用不同的技术栈，可以选择最适合该服务的技术，这种技术多样性使得平台能够充分利用各种技术的优势，提高系统的整体性能。

(2)研究价值

基于微服务架构的在线学习平台设计与实践为在线教育领域的研究提供了新的方向和方法。通过对微服务架构的深入研究和应用实践，可以推动在线教育平台的创新发展。

(3)应用价值

微服务架构的应用使得在线学习平台更加灵活、可扩展和易于维护。这种架构模式不仅提高了平台的性能和稳定性，还降低了开发和运维成本，为在线教育平台的长期发展提供了有力支持。

6.3 展望

通过对基于微服务架构的在线学习平台的开发，让我对微服务架构有了更加深入的理解和掌握。该系统解决了传统单体架构系统可扩展性不强的问题，并且提高了系统高并发处理的能力，系统的总体表现达到预期目标，但仍有很大改进空间。

首先系统的页面设计还不够美观，用户体验有待进一步提升。后续考虑引入专业的 UI 设计师，对页面布局、色彩搭配和交互设计进行优化，使其更符合现代审美和用户习惯。

其次，该系统目前仅实现了部分核心功能，未来还可以在系统中扩展一些其他功能。例如，针对在线学习平台学习方式单一的问题，可以增加直播授课的功能。随着人工智能的火热，可以在系统中引入一些智能推荐算法等技术，帮助平台实现学习资源的智能推荐和个性化定制。

随着微服务架构在各类系统中的应用越来越广泛，许多微服务相关的开发组件一直在推陈出新，可以利用这些新兴的开发组件，更高效地实施微服务架构，提升系统的性能和稳定性。

参考文献

- [1] 杨晓宏,郑新,何继龄.教育战“疫”之中国智慧及对教育数字化转型的启示——在线教育视角[J].中国电化教育,2023(03):46-54.
- [2] 尚普咨询集团.在线教育行业 2023 年发展现状评价与趋势预测[EB/OL].www.zhuanlan.zhihu.com/p/634247282,2023-06-02.
- [3] 徐玲玲,徐牧.在线教育调研洞察报告[R].深圳:华为技术有限公司,2021-04-28.
- [4] 宋丽哲.数字化转型背景下在线教育智能决策机制的探索[J].中国信息技术教育,2024(06):77-79.
- [5] 李家怡.线上教学问题分析与线上教学平台发展模式研究[C].中国管理科学研究院教育科学研究所.教育理论与实践与实践网络研讨会论文集(三).2022:150-153.
- [6] 门舜,孟庆红.微服务技术在软件架构中的应用分析与研究[J].信息与电脑(理论版),2023,35(15):138-140.
- [7] 张墨涵,王雪英,沈学东,等.微服务架构技术与挑战[J].网络安全技术与应用,2023(02):3-4.
- [8] Tokmak A V, Akbulut A, Catal C. Boosting the visibility of services in microservice architecture[J]. Cluster Computing, 2023: 1-13.
- [9] Blinowski G, Ojdowska A, Przybyłek A. Monolithic vs. microservice architecture: A performance and scalability evaluation[J]. IEEE Access, 2022, 10: 20357-20374.
- [10] Chaieb M, Saied M A. Automate migration to microservices architecture using Machine Learning techniques[J]. arXiv preprint arXiv:2301.06508, 2023(01):546-597.
- [11] Hasan H M ,Osman H M ,Admodisastro I N , et al.From Monolith to Microservice: Measuring Architecture Maintainability[J].International Journal of Advanced Computer Science and Applications (IJACSA),2023,14(5):857-866.
- [12] Delowar M H ,Tangina S ,Sharmen A , et al.The role of microservice approach in edge computing: Opportunities, challenges, and research directions[J].ICT Express,2023,9(6):1162-1182.
- [13] Teresa M G ,Muhammad Y ,Javier T , et al.A transaction platform for microservices-based big data systems[J].Simulation Modelling Practice and Theory,2023,123:28-54.
- [14] Vahdati A, Ramsin R. Model-Driven Methodology for Developing Chatbots Based on Microservice Architecture[C].In Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering.2024:247-254.
- [15] Li Y, Wang C Z, Li Y, et al. Granularity decision of microservice splitting in view of maintainability and its innovation effect in government data sharing[J]. Discrete Dynamics in Nature and Society, 2020: 1-11.
- [16] 江郑,王俊丽,曹芮浩.一种基于微服务架构的服务划分方法[J].计算机科学,2021,48(12):17-23.
- [17] 王璐,姜宇轩,李青山.微服务故障检测研究综述[J].计算机学报,2023,46(11):2342-2369.
- [18] 杨乾龙,江凌云.基于机器学习的微服务负载均衡算法研究[J].计算机科学,2023,50(05):313-321.
- [19] 张卓,张上,项天旭.基于改进 TOPSIS 的动态权重负载均衡算法[J].计算机工程与设

计,2023,44(11):3222-3229.

[20] Wang P, Zhang X, Cao Z, et al. MADMM: microservice system anomaly detection via multi-modal data and multi-feature extraction[J]. Neural Computing and Applications, 2024: 1-19.

[21] Anshari M, Alas Y, Sabtu N P H, et al. Online Learning: trends, issues and challenges in the Big Data Era[J]. Journal of E-learning and Knowledge Society, 2016, 12(1):121-134.

[22] Saadatdoost R, Sim A T H, Yadegaridehkordi E. The Coursera Community Framework: exploring the MOOC as a Community of Practice[J]. Journal of e-Learning and Knowledge Society, 2023, 19(1): 53-66.

[23] Feng J, Zhao Y. Analysis of learning behaviour based on edX open data[C].Journal of Physics: Conference Series. IOP Publishing, 2021, 1738(1): 012132.

[24] Freeman C, Kittredge A, Wilson H, et al. The duolingo method for app-based teaching and learning[J]. Duolingo Research Report, 2023:1-19.

[25] 刘安然.中美高校慕课比较研究——以“中国大学 MOOC”与 Coursera 为例[J].高教探索,2021,9(9):88-94.

[26] 何国良,荆燕飞.基于“腾讯课堂”的在线教学实践[J].实验科学与技术,2021,19(3):56-61.

[27] 朱少威,王丹婷,王雪,等.基于 SWOT 方法的网络课程学习平台发展研究[J].Advances in Applied Mathematics,2023,12(6):2774-2780.

[28] 金建平.基于“实验楼”在线实训平台的课程教学研究与实践——以中职 Linux 课程为例[J].现代职业教育,2020,20:204-205.

[29] 赵迪, 段薇薇.广联达造价软件应用技术课程开发与研究[J]. 山西建筑, 2015, 41(26): 235-236.

[30] 肖静, 孙林惠泽, 程慧超. 面向开放教育的线上线下融合课堂框架的初步研究[J].亚洲开放大学协会学报,2019,14(2):134-146.

[31] 刘婷.JSP 技术发展前景研究[J].信息与电脑(理论版),2019,8:70-75.

[32] 张林.基于微服务架构的商对客模式电商网站的设计与实现[J].软件工程,2021,24(09):55-57.

[33] 薛云兰,黄嘉浩,邵桐杰.微服务架构的在线课程学习系统的研究与设计[J]. 计算机时代,2022(5):130-137.

[34] 克里斯·理查森. 微服务架构设计模式[M].机械工业出版社,2019.

[35] 倪小璐,王旭英,边俊凯,等.微服务软件架构设计模式及其应用[J].杭州师范大学学报(自然科学版),2021,20(4): 442-448.

[36] Söylemez M, Tekinerdogan B, Kolukisa Tarhan A. Challenges and solution directions of microservice architectures: A systematic literature review[J].Applied sciences,2022,12(11):5507-5546.

[37] 李春霞.微服务架构研究概述[J]. 软件导刊, 2019, 18(8): 1-3.

[38] 李鑫.微服务治理体系、架构及实践[M].北京:电子工业出版社,2020.

[39] Raj V, Bhukya H.Assessing the Impact of Migration from SOA to Microservices Architecture[J].SN Computer Science,2023,4(5):77-84.

[40] Gos K, Zabierowski W. The comparison of microservice and monolithic architecture[C].2020 IEEE XVth International Conference on the Perspective Technologies and Methods in MEMS Design (MEMSTECH). IEEE, 2020:150-153.

[41] 张义鑫,林磊,张宁,等.基于云平台和微服务架构的城市轨道交通 AFC 系统[J].都市快轨交

通,2021,34(06):131-139.

[42] 谢得麟,严跃宁,林亚忠.基于微服务架构的医院统一接入平台设计与应用[J].中国医疗设备,2021,36(2):89-91.

[43] Velepucha V, Flores P. A survey on microservices architecture: principles, patterns and migration challenges[J]. IEEE Access, 2023(8),11:88339-88358.

[44] 张善永.TechEmpower 21 轮 Web 框架性能评测--C#的性能和 Rust、C++并驾齐驱[EB/OL]. www.cnblogs.com/shanyou/p/16536009.html, 2022-07-30.

[45] 陆钢,陈长怡,黄泽龙.面向云网融合的智能云原生架构和关键技术研究[J].电信科学,2020,36(9): 67-74.

[46] Wang X, Zhang Y, Jia S, et al. Design and Implementation of a Microservices-based Medical Platform[J]. International Core Journal of Engineering, 2023, 9(6): 143-151.

[47] Gonzalez I. Building Microservice APIs Using GRPC[D]. California State University, Northridge, 2020(01):1-38.

[48] 张莉,蒲梦媛,刘奕君,等. 对软件工程中经验研究的调查[J]. 软件学报, 2018, 29(5): 1422-1450.

[49] 李智,邓杰,杨溢龙,等. 从信息物理融合系统问题模型到 UML 用例图的变换方法[J]. 计算机科学, 2020, 47(12): 65-72.

[50] 杨福军,丁涛,付眸,等. 基于 RBAC 的权限复杂性与可靠性控制模型研究[J]. 计算机应用与软件, 2022, 39(1): 30-38.

[51] 张黎娜,童敏. 基于 SSO 的开放大学智慧化校园建设的探索与实践[J]. 电脑知识与技术,2022,18(4):48-50.

[52] 杨婷,莫若玉,张秀娟,等.轻量级缓存策略的关系型数据库全文搜索加强与扩展[J].计算机应用,2023,43(08):2431-2438.

[53] 刘燕,刘斌. 智慧旅游信息平台数据库建设和架构设计[J]. 现代电子技术, 2021, 44(24): 74-78.

[54] 顾郁晨. 基于微服务架构的医疗健康管理系统的设计与实现[D].华东师范大学,2022.

[55] 王 旭.计算机软件测试技术与开发应用研究[J].工程技术创新与发展,2023,1(2):6-11.

[56] 许海楠.计算机应用系统性能测试技术及应用分析[J].网络安全技术与应用,2021,(09):14-15.

[57] 张辉.Web 系统的性能测试技术研究[J].计算机时代,2021,(10):28-31.